

Nanobot-Java 项目学习手册

一、项目概述

Nanobot-Java 是基于香港大学开源的 Nanobot (mini 版 OpenClaw) 项目进行的 Java 重写。它是一个轻量级的 AI Agent 终端应用，遵循 "Core stays small; extend at the edges" (核心保持精简，通过边缘扩展) 的设计理念。

项目位置: <https://github.com/ztkyaojiayou/mynanobot-java>

二、AI 开发演进与核心概念

阅读目标: 理解 AI 编程领域的发展脉络和关键术语，建立全局认知框架。

2.1 AI Native — 一个时代的范式转移

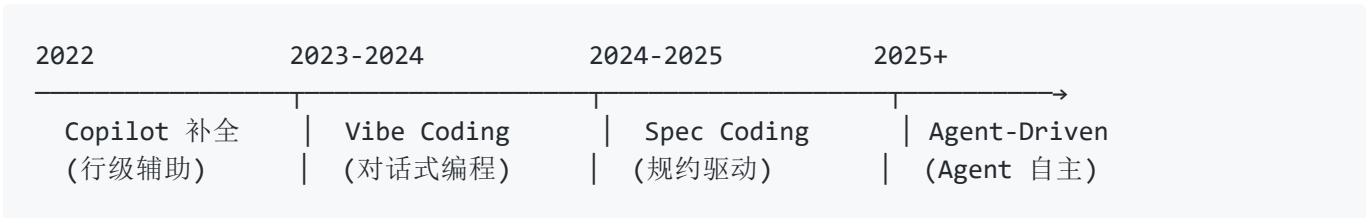
AI Native 指从设计之初就以 AI 为核心构建的应用，而非在现有产品上"嫁接" AI 功能。

	AI-Wrapped (AI 包装)	AI-Native (AI 原生)
AI 角色	辅助功能，可选模块	核心引擎，不可剥离
架构	传统三层 + LLM API 调用	以 Agent Loop 为中心的状态机
交互	表单/按钮为主，偶尔用 AI	自然语言是主交互界面
开发方式	人写代码，AI 辅助补全	AI 自主编码，人做 Code Review
示例	Notion AI、钉钉 AI 助手	Cursor、Claude Code、Devin、本项目

判断标准: 如果把 AI 组件从系统中移除，产品是否还能正常工作？能 → AI-Wrapped；不能 → AI-Native。

本项目的目标就是构建一个 AI-Native 的轻量级 Agent 框架。

2.2 AI 编程模式的演进: Vibe Coding → Spec Coding → Agent-Driven



2.2.1 Vibe Coding (对话式编程)

由 Andrej Karpathy 在 2025 年初提出：开发者用自然语言描述需求，AI 生成代码，人只管"vibe"（感觉）和验证结果。

用户："帮我写一个四则运算计算器，Vue3 + TypeScript"

AI：生成完整组件代码 → 用户运行 → 不满意 → "加个历史记录功能" → AI 修改

特点：

- ✓ 上手零门槛，会说话就能编程
- ✓ 原型验证极快，几分钟出一个 Demo
- ✗ 缺乏结构化约束，复杂项目难以维护
- ✗ AI 理解偏差会累积，迭代多轮后代码质量下降

2.2.2 Spec Coding (规约驱动编程)

Vibe Coding 的下一阶段演进。在写代码之前，先与 AI 合作产出一份详尽的**规格说明书** (Specification)，明确边界条件、数据结构、错误处理等，再让 AI 按 Spec 实现。

用户 + AI 讨论需求 → 产出 Spec.md (接口定义、数据模型、测试用例、边界条件)

→ AI 按 Spec 逐项实现

→ 自动化验证 vs Spec

特点：

- ✓ 有规约约束，减少 AI 理解偏差
- ✓ Spec 本身就是文档，可维护性强
- ✓ 支持"先出计划再执行"的工作流 (本项目 Plan Mode 的灵感来源)
- ✗ 前期讨论成本较高

2.2.3 Agent-Driven (Agent 自主驱动)

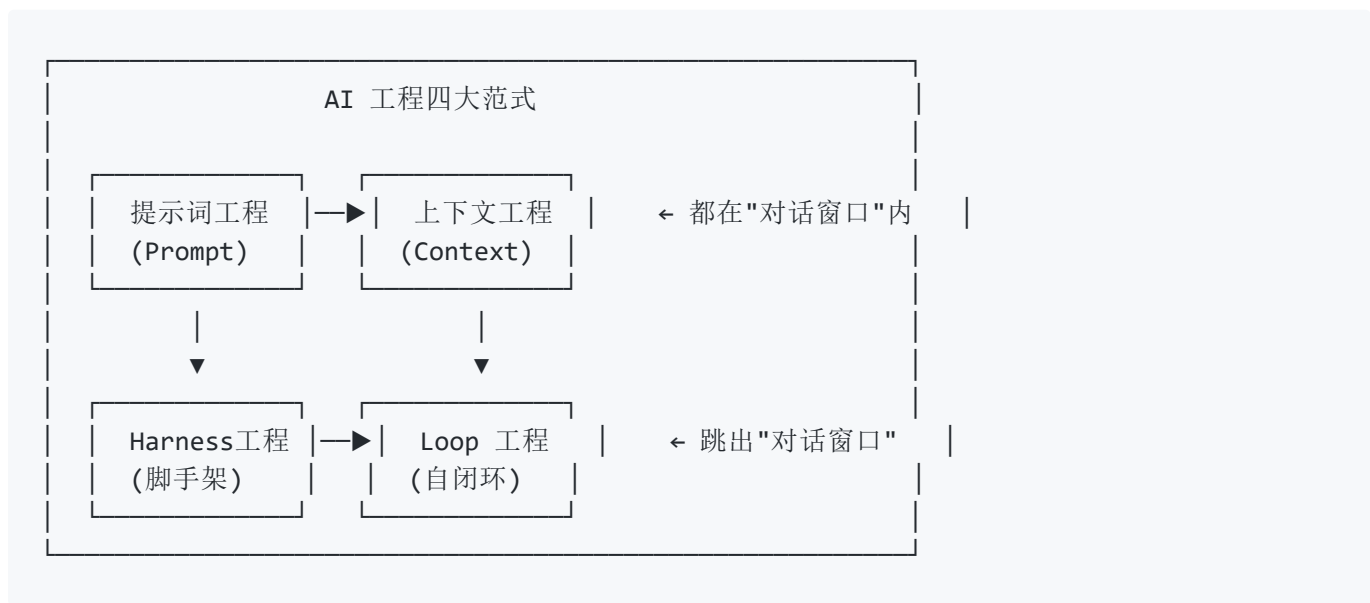
Spec Coding 的再下一阶段。AI Agent 不再被动等待指令，而是**主动**探索、规划、执行、验证循环。

Agent 接到目标: "实现用户注销功能"

- 自动探索项目结构 (`list_dir` / `glob` / `read_file`)
- 自动制定 Plan (需要改哪些文件)
- 自动逐个执行 (`edit_file` / `write_file`)
- 自动验证 (`mvn test` → 修复 → 再验证)
- 提交 PR

这正是本项目 Plan Mode + Agent Loop 的设计目标。

2.3 AI 工程的四大范式



2.3.1 第一层：提示词工程（Prompt Engineering）

时间：2022-2023 年，ChatGPT 时代早期。

核心问题：如何"问对问题"才能让 LLM 输出高质量结果？

技术	说明	示例
Zero-shot	直接提问，不给示例	"翻译成英文：你好"
Few-shot	给几个示例再提问	"输入:你好→Hello； 输入:再见→?"
Chain of Thought	要求逐步推理	"一步步思考后再回答"
Role Prompting	设定角色	"你是一个资深 Java 架构师..."
Structured Output	约束输出格式	"用 JSON 格式返回"

局限：提示词再精妙，LLM 的知识也被训练数据的截止日期锁死，无法获取实时信息、无法执行操作。

2.3.2 第二层：上下文工程（Context Engineering）

时间：2023-2024 年，RAG 和 Agent 兴起。

核心问题：如何把正确的信息在正确的时间喂给 LLM？

技术	说明	本项目的实现
RAG（检索增强生成）	从知识库检索相关文档注入上下文	<code>web_search</code> / <code>web_fetch</code>
System Prompt 设计	系统级指令控制 Agent 行为	<code>BuildState</code> → SOUL + NANOBOT.md + Rules + Plan Mode
Memory（记忆系统）	跨会话的信息持久化	<code>Dream</code> （提取+检索）+ <code>Consolidator</code> （压缩）
会话历史压缩	上下文窗口有限，需要压缩旧消息	<code>CompactState</code> + <code>Consolidator</code>
项目记忆注入	NANOBOT.md / CLAUDE.md 提供项目级上下文	<code>/init</code> 命令 + <code>BuildState</code> 自动加载

核心理念：LLM 的能力 = 模型本身的能力 × 你喂给它的上下文质量。上下文工程的目标是把"对的上下文"在"对的时间"塞进有限的 Context Window。

2.3.3 第三层：Harness 工程（脚手架工程）

时间：2024-2025 年，Agent 基础设施爆发。

核心问题：如何为 LLM 构建一个"可以做事"的运行时环境？

技术	说明	本项目的实现
Tool Use / Function Calling	LLM 调用外部工具	Tool 接口 + 17 个内置工具
MCP (Model Context Protocol)	标准化工具接入协议	MCPServer → MCPToolWrapper
Agent Loop	LLM 调用的控制循环	AgentLoop 状态机 + AgentRunner
Safety Guards	工具执行前的安全检查	PathGuard / CommandGuard / NetworkGuard
Permission System	工具调用的权限管控	PermissionManager (4 种模式)
Multi-Channel	多入口接入 (CLI/HTTP/WS)	V1 ChannelServer / V2 Spring Boot / V3 CliChannel
Sandbox / Workspace	执行环境隔离	工作区路径约束

核心理念：LLM 是一个"大脑"，但不能"动手"。Harness 工程就是给这个大脑装上"手"（工具）、"眼睛"（检索）、"安全帽"（权限）、"方向盘"（Agent Loop）。

2.3.4 第四层：Loop 工程（自闭环工程）★ 最新

时间：2025 年开始兴起。

核心问题：如何让 Agent 拥有自我纠正和自我改进的能力，形成"执行→验证→修复"的闭环？

传统 Agent (Harness 级):

LLM 输出 → 执行 → 结束

Loop 工程 (Loop 级):

LLM 输出 → 执行 → 观察结果 → 失败? → 分析原因 → 重新执行

↑_____|

模式	说明	示例
Self-Debugging	Agent 运行自己的代码，捕获错误，自我修复	"跑一下测试 → 失败了 → 分析 stack trace → 修改代码 → 再跑"
Reflection	Agent 定期反思自己的输出质量	"我上次的回答遗漏了边界条件，这次补上"
Adversarial Review	多个 Agent 互相审查	Claude Code 的 /code-review 多维度交叉验证
Plan → Execute → Verify	三步闭环	本项目的 /mode plan + 执行 + 验证
Multi-Agent Loop	子 Agent 并行执行，主 Agent 汇总	Subagent + SpawnTool

本项目的 Loop 工程体现：

- `AgentRunner.runInternal()`：递归循环，工具失败重试 3 次
- Plan Mode： `/plan` → 探索 → 出计划 → `/plan approve` → 执行
- Task 工具： `task_create` → 分步执行 → `task_update` 标记完成
- 子 Agent 系统： `spawn` → 分发任务 → 收集结果 → 汇总

2.4 关键名词速查表

术语	英文	一句话解释
AI 原生	AI Native	以 AI 为核心引擎构建的应用，非嫁接
对话式编程	Vibe Coding	自然语言描述需求，AI 生成代码
规约驱动编程	Spec Coding	先产 Spec 再让 AI 按规约实现
Agent 驱动	Agent-Driven	AI 主动探索→规划→执行→验证
提示词工程	Prompt Engineering	设计有效提问以获得高质量输出
上下文工程	Context Engineering	管理喂给 LLM 的信息（RAG、System Prompt、Memory）
脚手架工程	Harness Engineering	构建 Agent 运行时（工具、安全、循环控制）
自闭环工程	Loop Engineering	Agent 自我验证、纠错、改进的闭环
大语言模型	LLM	驱动 Agent 的核心 AI 模型
检索增强生成	RAG	从知识库检索信息增强 LLM 回答
思维链	Chain of Thought	让 LLM 逐步推理而非直接给答案
函数调用	Function Calling	LLM 调用外部 API/工具的能力
MCP 协议	Model Context Protocol	AI-工具通信的标准化协议
Agent 循环	Agent Loop	LLM 调用的控制循环（感知→思考→行动）
流式输出	Streaming / SSE	LLM 逐 token 实时输出内容
Token	Token	LLM 处理文本的最小单位（约 0.75 个英文单词）
嵌入向量	Embedding	文本的数值化表示，用于语义搜索
上下文窗口	Context Window	LLM 一次能处理的最大信息量
系统提示词	System Prompt	控制 Agent 行为的最高优先级指令
幻觉	Hallucination	LLM 生成似是而非的虚假信息

三、从0到1自实现一个Agent系统

阅读目标：跟着本章从零开始，逐步构建出一个完整的 AI Agent。每一步都有可直接复制粘贴运行的代码。读完本章，你会拥有一个自己能写代码、能搜索网页、能记住上下文的 Agent。

3.0 准备工作：5分钟搭好项目骨架

在开始写 Agent 之前，先创建一个标准 Maven 项目。

```
mini-agent/  
├─ pom.xml # Maven 配置  
└─ src/main/java/com/tutorial/  
    └─ (每步新增的 Java 文件)
```

pom.xml — 只需要一个 JSON 库依赖：

```
<?xml version="1.0" encoding="UTF-8"?>  
<project xmlns="http://maven.apache.org/POM/4.0.0"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
http://maven.apache.org/xsd/maven-4.0.0.xsd">  
    <modelVersion>4.0.0</modelVersion>  
    <groupId>com.tutorial</groupId>  
    <artifactId>mini-agent</artifactId>  
    <version>1.0-SNAPSHOT</version>  
    <properties>  
        <maven.compiler.source>17</maven.compiler.source>  
        <maven.compiler.target>17</maven.compiler.target>  
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>  
    </properties>  
    <dependencies>  
        <dependency>  
            <groupId>com.fasterxml.jackson.core</groupId>  
            <artifactId>jackson-databind</artifactId>  
            <version>2.16.1</version>  
        </dependency>  
    </dependencies>  
    <build>  
        <plugins>  
            <plugin>  
                <groupId>org.codehaus.mojo</groupId>  
                <artifactId>exec-maven-plugin</artifactId>  
                <version>3.1.0</version>  
            </plugin>  
        </plugins>  
    </build>  
</project>
```


创建好目录和 pom.xml 后，执行 `mvn compile` 确认环境正常：

```
mkdir -p mini-agent/src/main/java/com/tutorial
# 把上面的 pom.xml 放到 mini-agent/ 目录下
cd mini-agent
mvn compile      # 应该显示 BUILD SUCCESS
```

为什么选 Jackson? Java 自带的 HttpClient 处理 HTTP，Jackson 解析 LLM 返回的 JSON。这两个依赖覆盖了 Agent 开发 90% 的需求。

3.1 第1步：最小可行Agent — 10行代码跑起来

一个 Agent 的本质是什么？**接收用户输入** → **调用 LLM** → **返回结果**。

完整可运行代码 — 复制到 `src/main/java/com/tutorial/Step1MinimalAgent.java`：

```

package com.tutorial;

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;

/**
 * 第1步：最小可行 Agent
 * 一条消息发给 DeepSeek，打印返回结果。
 *
 * 运行方式：
 * export DEEPSEEK_API_KEY=sk-xxxxxxx
 * mvn exec:java -Dexec.mainClass="com.tutorial.Step1MinimalAgent"
 */
public class Step1MinimalAgent {
    public static void main(String[] args) throws Exception {
        // ---- 1. 准备 API Key ----
        String apiKey = System.getenv("DEEPSEEK_API_KEY");
        if (apiKey == null || apiKey.isBlank()) {
            System.err.println("❌ 请设置环境变量 DEEPSEEK_API_KEY");
            System.err.println("    export DEEPSEEK_API_KEY=sk-xxxxxxx");
            System.exit(1);
        }

        // ---- 2. 构建请求体 ----
        String body = """
        {
            "model": "deepseek-chat",
            "messages": [
                {"role": "system", "content": "你是一个编程助手，用中文回答。"},
                {"role": "user", "content": "用 Java 写一个 Hello World"}
            ]
        }""";

        // ---- 3. 发送 HTTP 请求 ----
        HttpClient client = HttpClient.newBuilder()
            .connectTimeout(Duration.ofSeconds(10))
            .build();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://api.deepseek.com/chat/completions"))
            .header("Authorization", "Bearer " + apiKey)
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(body))
            .build();

        // ---- 4. 解析并打印结果 ----
        HttpResponse<String> response = client.send(

```

```

        request, HttpResponse.BodyHandlers.ofString());

// 简单提取 content（正式项目用 Jackson 解析）
String json = response.body();
int start = json.indexOf("\"content\":");
if (start != -1) {
    start += 11; // 跳过 "content":
    int end = json.indexOf("\"", start);
    String content = json.substring(start, end)
        .replace("\\n", "\n")
        .replace("\\\"", "\"");
    System.out.println("👤 Agent 回答: \n" + content);
} else {
    System.out.println("原始响应: \n" + json);
}
}
}

```

运行：

```

export DEEPSEEK_API_KEY=sk-xxxxxxx # 替换为你的 API Key
cd mini-agent
mvn exec:java -Dexec.mainClass="com.tutorial.Step1MinimalAgent"

```

预期输出：

👤 Agent 回答：
下面是一个简单的 Hello World 程序：

```

public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}

```

🎉 **恭喜！你已经有了一个“能对话”的 Agent。** 但它还只会“说话”，不会“做事”——不能读文件、搜网页、写代码。下一步给它装上“手”。

📄 本项目对应源码：`providers/impl/DeepSeekProvider.java` — 实际版本支持流式、重试、错误处理。

3.2 第2步：加上工具调用 — 让 Agent 能“动手”

LLM 可以返回 `tool_calls`，告诉程序“我想执行 `read_file`”。程序执行后把结果返回给 LLM，LLM 再基于新信息继续回复。这就是 Agent 的“手”。

3.2.1 设计 Tool 接口

```
// Tool.java
package com.tutorial;

import java.util.List;
import java.util.Map;
import java.util.concurrent.CompletableFuture;

public interface Tool {
    String getName();           // 工具名，如 "read_file"
    String getDescription();    // 功能描述，LLM 据此判断何时调用
    Map<String, Object> getParameters(); // 参数 JSON Schema
    CompletableFuture<String> execute(Map<String, Object> params);
}
```

3.2.2 实现第一个工具：read_file

```
// ReadFileTool.java
package com.tutorial;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.Map;
import java.util.concurrent.CompletableFuture;

public class ReadFileTool implements Tool {
    @Override
    public String getName() { return "read_file"; }

    @Override
    public String getDescription() { return "读取指定文件的内容"; }

    @Override
    public Map<String, Object> getParameters() {
        return Map.of(
            "type", "object",
            "properties", Map.of(
                "path", Map.of("type", "string", "description", "文件路径")
            ),
            "required", List.of("path")
        );
    }

    @Override
    public CompletableFuture<String> execute(Map<String, Object> params) {
        return CompletableFuture.supplyAsync(() -> {
            try {
                String path = (String) params.get("path");
                return Files.readString(Path.of(path));
            } catch (IOException e) {
                return "❌ 读取失败: " + e.getMessage();
            }
        });
    }
}
```

3.2.3 核心：工具调用循环

LLM 返回 `tool_calls` 时，执行工具并把结果注入对话历史，**递归**调用 LLM 直到它返回纯文本。

完整可运行代码 — 放到 `src/main/java/com/tutorial/Step2ToolCalling.java`：

```

package com.tutorial;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;
import java.util.*;

/**
 * 第2步：加上工具调用
 *
 * 运行方式：
 * export DEEPSEEK_API_KEY=sk-xxxxxxx
 * echo "Hello World" > /tmp/test.txt
 * mvn exec:java -Dexec.mainClass="com.tutorial.Step2ToolCalling"
 */
public class Step2ToolCalling {
    private static final ObjectMapper mapper = new ObjectMapper();
    private static final String API_KEY = System.getenv("DEEPSEEK_API_KEY");
    private static final HttpClient client = HttpClient.newBuilder()
        .connectTimeout(Duration.ofSeconds(30)).build();
    private static int iteration = 0;

    // 工具注册表
    private static final Map<String, Tool> tools = Map.of(
        "read_file", new ReadFileTool()
    );

    public static void main(String[] args) throws Exception {
        if (API_KEY == null || API_KEY.isBlank()) {
            System.err.println("❌ 请设置 DEEPSEEK_API_KEY");
            System.exit(1);
        }

        List<Map<String, Object>> messages = new ArrayList<>();
        messages.add(msg("system", "你是编程助手。可以调用 read_file 工具读取文件。"));
        messages.add(msg("user", "帮我读一下 /tmp/test.txt 的内容"));

        String result = runLoop(messages);
        System.out.println("\n📄 最终回答: \n" + result);
    }

    /** 递归循环: LLM → 工具执行 → 注入结果 → 再调 LLM */
    static String runLoop(List<Map<String, Object>> messages) throws Exception {
        if (++iteration > 10) return "⚠️ 达到最大迭代次数";

        String responseJson = callLLM(messages);
    }

```

```

        JsonNode root = mapper.readTree(responseJson);
        JsonNode choice = root.path("choices").get(0);
        JsonNode msg = choice.path("message");

        // 情况1: LLM 想调工具
        if (msg.has("tool_calls")) {
            JsonNode toolCalls = msg.get("tool_calls");

            // 把 LLM 的 tool_calls 请求加入历史
            messages.add(jsonToMap(msg));

            for (JsonNode tc : toolCalls) {
                String toolName = tc.path("function").path("name").asText();
                String argsJson = tc.path("function").path("arguments").asText();
                String callId = tc.path("id").asText();

                System.out.println("🔗 调用工具: " + toolName + "(" + argsJson + ")");

                @SuppressWarnings("unchecked")
                Map<String, Object> args = mapper.readValue(argsJson, Map.class);
                Tool tool = tools.get(toolName);
                String result = tool != null
                    ? tool.execute(args).get()
                    : "❌ 未知工具: " + toolName;

                // 工具结果加入历史
                messages.add(Map.of(
                    "role", "tool",
                    "tool_call_id", callId,
                    "content", result
                ));
                System.out.println("    结果: " + result.substring(0,
                    Math.min(100, result.length())) + "...");
            }

            return runLoop(messages); // 递归!
        }

        // 情况2: LLM 返回纯文本 → 结束
        return msg.path("content").asText("");
    }

    /** 调用 DeepSeek API */
    static String callLLM(List<Map<String, Object>> messages) throws Exception {
        // 构建 tools 参数 (告诉 LLM 有哪些工具可用)
        List<Map<String, Object>> toolDefs = new ArrayList<>();
        for (Tool t : tools.values()) {
            toolDefs.add(Map.of(
                "type", "function",
                "function", Map.of(
                    "name", t.getName(),

```

```

        "description", t.getDescription(),
        "parameters", t.getParameters()
    ));
}

Map<String, Object> body = new HashMap<>();
body.put("model", "deepseek-chat");
body.put("messages", messages);
body.put("tools", toolDefs);

HttpRequest req = HttpRequest.newBuilder()
    .uri(URI.create("https://api.deepseek.com/chat/completions"))
    .header("Authorization", "Bearer " + API_KEY)
    .header("Content-Type", "application/json")

    .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(body)))
    .build();

return client.send(req, HttpResponse.BodyHandlers.ofString()).body();
}

static Map<String, Object> msg(String role, String content) {
    return new HashMap<>(Map.of("role", role, "content", content));
}

@SuppressWarnings("unchecked")
static Map<String, Object> jsonToMap(JsonNode node) {
    return mapper.convertValue(node, Map.class);
}
}

```

运行:

```

echo "Hello World" > /tmp/test.txt
mvn exec:java -Dexec.mainClass="com.tutorial.Step2ToolCalling"

```

预期输出:

 调用工具: read_file({"path":"/tmp/test.txt"})
结果: Hello World

 最终回答:
/tmp/test.txt 的内容是: Hello World

💡 发生了什么？

1. 用户说"帮我读一下 /tmp/test.txt"
2. LLM 理解意图 → 返回 `tool_calls=[{read_file, path=/tmp/test.txt}]`
3. 程序执行 `read_file` → 拿到 "Hello World"
4. 把结果注入对话历史，递归调 LLM
5. LLM 基于结果生成最终回答

📄 本项目对应源码： `tools/Tool.java` + `core/AgentRunner.java:runInternal()` — 实际版本支持并行执行、超时、重试。

3.3 第3步：System Prompt — 让 Agent 知道"自己是谁"

System Prompt 是控制 Agent 行为的最高优先级指令。没有它，LLM 不知道自己是 my-nanobot。

```
// BuildState.java 的核心逻辑
String systemPrompt = """
    你是 mini-agent，一个基于 Java 的 AI 编程助手。
    你可以使用工具来读文件、写代码、执行命令。

    【行为准则】
    - 回答用中文
    - 代码块标注语言 (``java)
    - 修改文件前先 read_file 了解现状
    - 不确定时向用户确认，不要猜测

    【项目上下文】
    %s

    【编码规范】
    - Java 17, Maven 构建
    - 4 空格缩进，不用 Tab
    """,formatted(loadNanobotMd());
```

System Prompt 的组成部分（本项目 BuildState 的实际结构）：

① SOUL - 身份定义（名称+性格+能力）	← IdentityManager
② NANOBOT.md - 项目记忆（自动/手动）	← /init 生成
③ Rules - 编码/安全规则	← RuleManager 加载
④ Plan Mode 指令 - 只读+出计划	← 仅 /plan 模式下注入
⑤ 工具列表 - 由 API tools 参数提供	← 自动生成

设计原则：

原则	说明
先身份后能力	先告诉 LLM"你是谁", 再说"你能做什么"
具体优于抽象	"用 4 空格缩进" 优于 "代码要规范"
按需注入	Plan Mode 指令只在只读模式注入，不浪费 token
分层加载	核心身份固定，项目规范可变，工具列表自动生成

📄 本项目对应源码：`core/state/BuildState.java` — 含加载身份、NANOBOT.md、Plan Mode 提示词、Rules。

3.4 第4步：消息总线 — 让 Agent 处理并发

当多个用户同时发消息时，不能直接同步处理——需要一个队列。



```

public class MessageBus {
    // 有界队列（防 OOM）
    private final BlockingQueue<InboundMessage> inboundQueue =
        new ArrayBlockingQueue<>(100);

    // 发布消息（队列满时阻塞）
    public void publishInbound(InboundMessage msg) throws InterruptedException {
        inboundQueue.put(msg);
    }

    // 消费消息（超时返回 null）
    public InboundMessage consumeInbound(long timeout, TimeUnit unit)
        throws InterruptedException {
        return inboundQueue.poll(timeout, unit);
    }
}

```

InboundMessage 结构：

```

public class InboundMessage {
    String sessionId;    // 会话 ID
    String channel;      // 来源通道（cli / api / websocket）
    String content;      // 用户输入
    Map<String, Object> metadata; // requestId、streamMode 等
}

```

设计要点：

- 使用 `ArrayBlockingQueue`（有界）而非 `LinkedBlockingQueue`（无界），防止生产者过快撑爆内存
- 单线程消费保证同一会话的消息串行处理
- 通过 `sessionId` 隔离不同用户/会话

📄 本项目对应源码：`bus/MessageBus.java`

3.5 第5步：流式输出 — 让 Agent 不“冷场”

用户不想等 30 秒才看到完整回复。流式输出让文字逐 token 出现。

LLM 的 SSE 流格式：

```
data: {"choices":[{"delta":{"content":"你"}}]}
data: {"choices":[{"delta":{"content":"好"}}]}
data: {"choices":[{"delta":{"content":", 我"}}]}
data: [DONE]
```

实现要点:

```
// 发送流式请求
HttpRequest req = HttpRequest.newBuilder()
    .uri(URI.create("https://api.deepseek.com/chat/completions"))
    .header("Authorization", "Bearer " + apiKey)
    .POST(HttpRequest.BodyPublishers.ofString(
        mapper.writeValueAsString(Map.of(
            "model", "deepseek-chat",
            "messages", messages,
            "stream", true // ← 开启流式
        ))
    )).build();

// 逐行读取 SSE 流
HttpResponse<InputStream> resp = client.send(req,
    HttpResponse.BodyHandlers.ofInputStream());

BufferedReader reader = new BufferedReader(
    new InputStreamReader(resp.body()));

String line;
while ((line = reader.readLine()) != null) {
    if (line.startsWith("data: ")) {
        String data = line.substring(6);
        if ("[DONE]".equals(data)) break;
        JsonNode node = mapper.readTree(data);
        String delta = node.path("choices").get(0)
            .path("delta").path("content").asText("");
        System.out.print(delta); // 实时打印 ← 用户体验关键
    }
}
```

DeepSeek 流式的特殊性: 工具调用参数是逐个字符返回的（不保证完整 JSON），需要自己拼接。

```
第1帧: tool_calls[0].function.arguments = "{\"pa"
第2帧: tool_calls[0].function.arguments = "th\":"
第3帧: tool_calls[0].function.arguments = "\"/tmp"
...累积拼接成完整 JSON: {"path":"/tmp/test.txt"}
```

📄 本项目对应源码： `providers/impl/DeepSeekProvider.java` 的 `ToolCallAccumulator` 内部类 + `chatStream()`。

3.6 第6步：安全与权限 — 让 Agent 不越界

Agent 能执行 Shell 命令、写文件——这些能力必须有权限控制。

工具调用 → PathGuard(路径检查) → CommandGuard(命令检查)
→ NetworkGuard(URL检查) → PermissionMode(模式判定)
→ 交互确认(1=允许/2=信任/3=拒绝)

4 种权限模式（对标 Claude Code）：

```
public enum PermissionMode {
    PLAN,           // 只读（/plan 模式）
    DEFAULT,        // 读放行，写需确认
    ACCEPT_EDITS,   // 读+文件编辑放行，Shell 仍需确认
    BYPASS;         // 全部放行（谨慎使用）

    public boolean allowsTool(Tool tool) {
        return switch (this) {
            case PLAN      -> tool.isReadOnly(); // 只允许只读工具
            case DEFAULT   -> tool.isReadOnly(); // 只读自动放行
            case ACCEPT_EDITS -> !tool.isExclusive(); // 编辑也放行
            case BYPASS    -> true;                // 全放行
        };
    }
}
```

三个 Guard（守卫层）：

Guard	检查内容	示例
PathGuard	路径必须在工作区内	禁止读 <code>/etc/passwd</code>
CommandGuard	禁止危险命令	禁止 <code>rm -rf /</code> 、 <code>shutdown</code>
NetworkGuard	禁止内网地址	禁止访问 <code>127.0.0.1</code> 、 <code>192.168.*</code>

📄 本项目对应源码： `security/PermissionManager.java` + `security/PermissionMode.java`
+ `security/guard/`。

3.7 第7步：会话管理 — 让 Agent 记住上下文

Agent 不能每次对话都"失忆"。会话持久化让用户下次回来还能接着聊。

存储结构：

```
.nanobot/sessions/
├── cli-1784128421194/
│   ├── history.jsonl      ← 消息历史（每行一条消息的 JSON）
│   └── metadata.json      ← 会话元数据（名称、创建时间）
├── cli-1784207282340/
│   ├── history.jsonl
│   └── metadata.json
```

history.jsonl 格式：

```
{"role":"system","content":"你是 mini-agent..."}
{"role":"user","content":"帮我读一下 test.txt"}
{"role":"assistant","content":"好的，让我来读取..."}
{"role":"assistant","content":null,"tool_calls":[{"id":"call_1","function":
{"name":"read_file","arguments":{"path":"test.txt"}}}]}
{"role":"tool","tool_call_id":"call_1","content":"Hello World"}
```

核心操作：

操作	实现
追加消息	FileWriter 追加模式写入 JSONL（不重写全文件）
加载历史	流式读取 JSONL → List<Map>
重命名	修改 metadata.json 的 name 字段
删除	递归删除会话目录
列表	遍历 sessions/ 目录，读 metadata.json

```
// 增量追加（无需全量重写）
public void appendMessage(String sessionId, Map<String, Object> msg) {
    Path file = getHistoryPath(sessionId);
    try (FileWriter fw = new FileWriter(file.toFile(), true)) {
        fw.write(mapper.writeValueAsString(msg) + "\n");
    }
}
```

📄 本项目对应源码： `session/SessionStore.java` （存储层） + `session/SessionManager.java` （业务层）。

3.8 第8步：状态机重构 — 用 State 模式组织代码

当 Agent 的处理流程变复杂时（恢复历史→检查命令→构建上下文→调 LLM→保存→响应），一堆 if-else 会变得难以维护。State 模式是解药。

状态转换链：

RESTORE → COMPACT → COMMAND → BUILD → RUN → SAVE → RESPOND → DONE

State 接口：

```
public interface AgentState {  
    TurnState execute(TurnContext ctx);  
}
```

每个状态一个类：

```
// RestoreState.java - 加载会话历史  
public class RestoreState implements AgentState {  
    public TurnState execute(TurnContext ctx) {  
        List<Map> history = sessionStore.loadHistory(ctx.getSessionId());  
        ctx.getMessages().addAll(history);  
        return TurnState.COMPACT; // → 下一个状态  
    }  
}  
  
// RunState.java - 调 LLM + 执行工具  
public class RunState implements AgentState {  
    public TurnState execute(TurnContext ctx) {  
        String response = agentRunner.run(ctx);  
        ctx.setResponse(response);  
        return TurnState.SAVE;  
    }  
}
```

State 模式的优势：

- 每个状态独立文件，职责清晰
- 新增状态只需实现接口并注册，不碰已有代码（开闭原则）

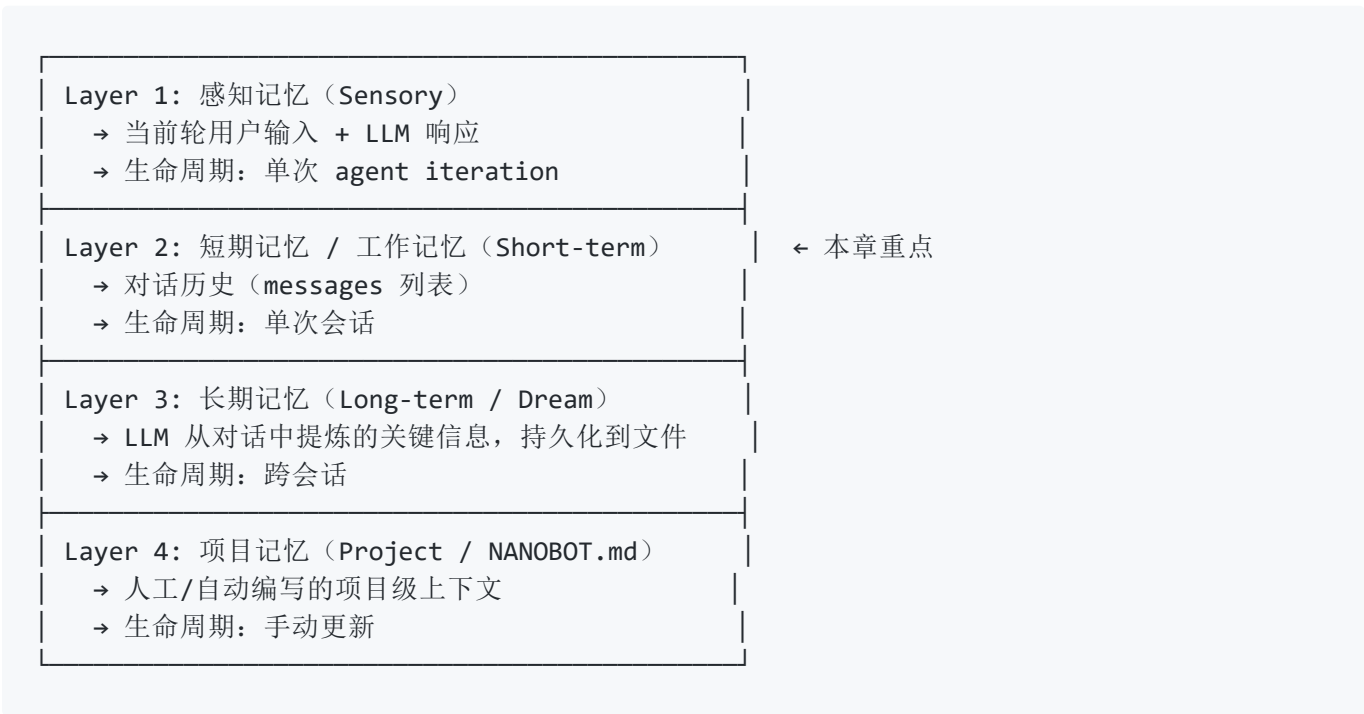
- 本项目的 AgentLoop 从 973 行精简到 579 行

📄 本项目对应源码：`core/state/` 目录下的 8 个 StateHandler 类 + `core/AgentLoop.java`。

3.9 第9步：记忆系统 — 突破上下文窗口限制

LLM 的 Context Window 有限（DeepSeek 约 128K tokens）。对话长了以后，要么截断（丢失早期信息），要么压缩（提炼要点）。

四层记忆模型：



记忆压缩 (短期→长期)：


```
// Consolidator.java 的核心逻辑
public String consolidate(List<Map<String, Object>> messages) {
    int estimatedTokens = estimateTokens(messages);
    if (estimatedTokens < contextWindow * 0.9) {
        return null; // 还没到 90%，不需要压缩
    }

    // 把旧消息发给 LLM，让它总结成一段 system 消息
    String prompt = ""
        请将以下对话历史压缩为一段精炼的摘要（保留关键决策和上下文）：
        "" + formatMessages(messages.subList(0, messages.size() / 2));

    String summary = callLLM(prompt);
    // 用摘要替换被压缩的旧消息
    messages.subList(0, messages.size() / 2).clear();
    messages.add(0, Map.of("role", "system", "content", "【对话摘要】" + summary));
    return summary;
}
```

📄 本项目对应源码： `memory/Consolidator.java` + `memory/Dream.java` + `memory/MemoryLoader.java`。

3.10 第10步：生产化 — 让 Agent 稳定运行

Demo 和产品之间的差距在于“异常情况处理”。

```
// AgentRunner.java 的生产化保护层

// ① 工具超时 + 重试
CompletableFuture<Object> future = tool.execute(params);
try {
    return future.get(90, TimeUnit.SECONDS); // 最多等 90 秒
} catch (TimeoutException e) {
    if (retryCount < 3) {
        Thread.sleep(1000); // 等 1 秒再试
        return executeToolWithRetry(tool, params, retryCount + 1);
    }
    return "✗ 工具执行超时，已重试 3 次";
}

// ② 降级兜底：连续 3 次工具失败 → 不带工具直接让 LLM 回答
if (consecutiveFailures >= 3) {
    log.warn("连续 3 次工具失败，触发降级兜底");
    return callLLMWithoutTools(messages);
}

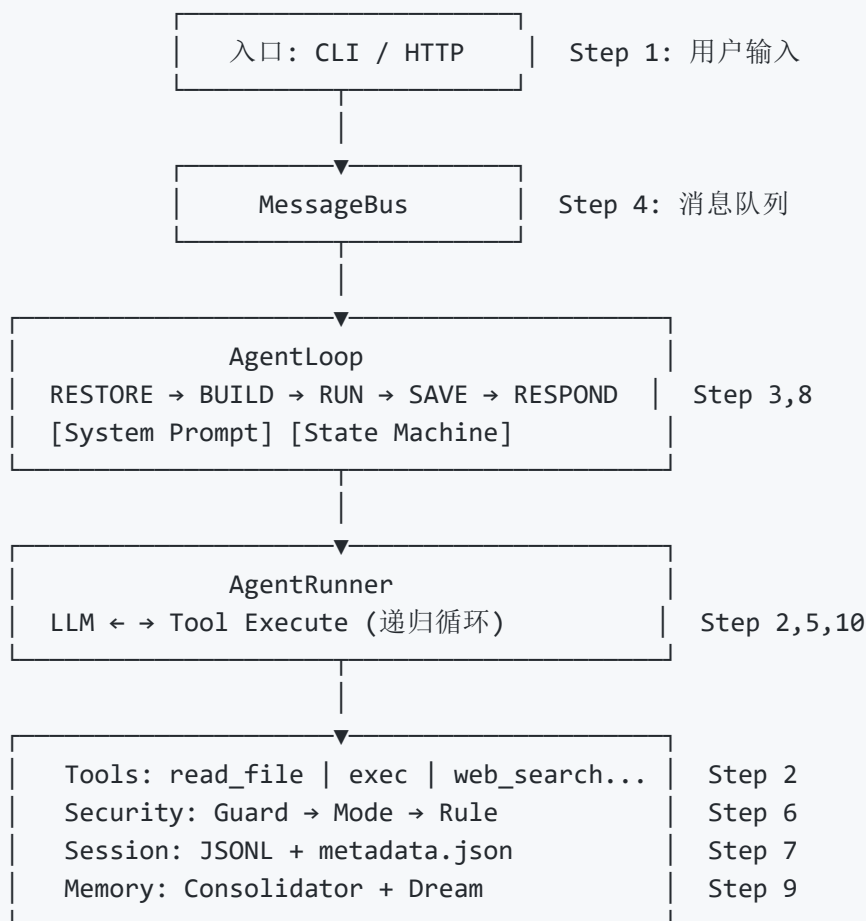
// ③ 优雅关闭
Runtime.getRuntime().addShutdownHook(new Thread(() -> {
    log.info("正在关闭...");
    messageBus.drain(); // 排空队列
    executor.shutdown(); // 关闭线程池
    executor.awaitTermination(5, TimeUnit.SECONDS);
}));
```

生产化检查清单：

机制	说明
☑ 工具超时	每次工具调用设 90 秒超时
☑ 自动重试	失败后重试 3 次，间隔 1 秒
☑ 有界队列	ArrayBlockingQueue(100) 防 OOM
☑ 优雅关闭	shutdown hook 排空队列 + 关线程池
☑ 流式中断	Esc 键停止当前回复
☑ 降级兜底	连续失败后不带工具直接回答
☑ 费用上限	maxCost 防 API 费用失控
☑ 迭代上限	maxToolIterations 防死循环

3.11 完整架构一览 + 你学到了什么

经过 10 步演进，你从一个 10 行的 HTTP 请求，逐步构建出了一个完整的 Agent 系统：



你学到了：

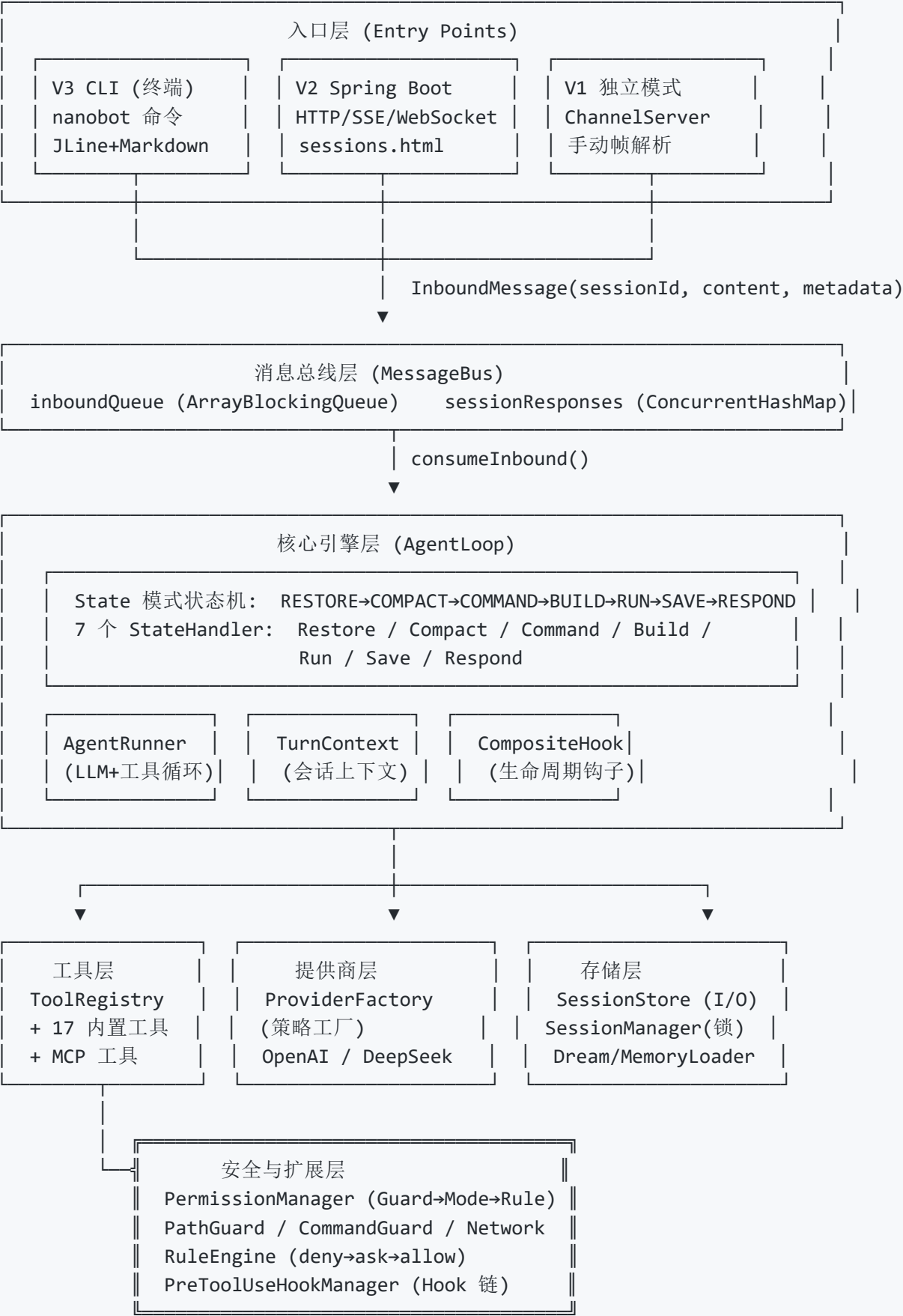
1. **不依赖任何 AI 框架**，纯 Java 手搓 Agent（只需 Jackson + JDK HttpClient）
2. **State 模式**的实际应用——状态机组织复杂流程
3. **从 Demo 到生产**的完整路径——超时、重试、降级、优雅关闭
4. **DeepSeek API 的调用细节**——流式参数拼接、tool_calls 解析
5. **Agent 的核心设计模式**——消息总线解耦、策略工厂选择 Provider、Repository 分离 I/O

📖 接下来建议阅读 **第四~五章**（架构详解），那里会对每个模块做深度剖析。也可以跳到 **第六章**（主流框架对比）了解生态全貌，或跳到 **第七章**（nanobot CLI 实战）直接使用本项目。

四、架构概览

阅读目标：先建立全局地图，了解系统的分层结构和核心组件，再看下一章的逐个模块详解。

4.1 整体架构分层



4.2 核心组件职责

组件	职责	文件位置
NanobotRunner	Spring Boot 启动器，组件初始化编排	NanobotRunner.java
NanobotApplication	V2 Spring Boot 入口 (HTTP/SSE/WS)	v2/NanobotApplication.java
NanobotCliApplication	V3 CLI 入口 (类 Claude Code 体验)	v3/NanobotCliApplication.java
CliChannel	CLI 交互通道，JLine 终端 + Markdown 渲染	v3/cli/CliChannel.java
AgentLoop	状态机引擎 (State 模式)，委托 7 个 StateHandler	core/AgentLoop.java
AgentRunner	LLM 调用循环，处理工具调用	core/AgentRunner.java
TurnContext	会话上下文，存储消息和状态	core/TurnContext.java
TurnState	状态枚举，定义状态机节点	core/TurnState.java
AgentState	State 模式接口 (8 个实现类)	core/state/
TaskStore	会话级任务追踪，持久化 JSON	core/TaskStore.java
MessageBus	消息总线，异步队列通信	bus/MessageBus.java
ToolRegistry	工具注册中心，支持只读过滤	tools/ToolRegistry.java
Tool	工具接口，定义工具契约	tools/Tool.java
LLMProvider	LLM 提供商接口 (详见 5.22)	providers/LLMProvider.java
ProviderFactory	策略工厂，按模型名匹配 Provider (详见 5.22)	providers/ProviderFactory.java

组件	职责	文件位置
SessionManager	会话业务层（锁管理+协调）	session/SessionManager.java
SessionStore	会话存储层（纯文件 I/O）	session/SessionStore.java
Config / ConfigLoader	配置加载和管理	config/
AgentHook / CompositeHook	钩子系统（Chain of Responsibility）	core/hook/
Command / CommandRegistry	命令系统（Command 模式）	command/
MCPManager	MCP 服务器管理和工具注册	mcp/MCPManager.java
MCPClient / StdioMCPClient / HttpMCPClient	MCP 客户端实现（Template Method）	mcp/
CronScheduler	定时任务调度器	cron/CronScheduler.java
Consolidator	对话压缩器（token 预算监控 → LLM 摘要）	memory/Consolidator.java
Dream	长期记忆系统（ ☒ 提取+检索+BUILD 注入，完整闭环）	memory/Dream.java
MemoryLoader	长期记忆文件 I/O（加载/解析/保存 MEMORY.md）	memory/MemoryLoader.java
PermissionManager	权限编排器（Guard→Mode→Rule 管道）	security/PermissionManager.java
PathGuard / CommandGuard / NetworkGuard	守卫层（Strategy 模式）	security/guard/

组件	职责	文件位置
RuleEngine	规则引擎, deny→ask→allow 优先级 链	<code>security/rule/RuleEngine.java</code>
MarkdownRenderer	CLI 终端 Markdown 渲染	<code>v3/tui/MarkdownRenderer.java</code>

4.3 模块结构

```

nanobot-java/
├── scripts/                                # 启动脚本
│   ├── nanobot                            # Mac/Linux 启动脚本
│   ├── nanobot.bat                        # Windows 启动脚本
│   └── start.sh / stop.sh                # 服务启停脚本
├── src/main/java/com/nanobot/
│   ├── NanobotRunner.java                # Spring Boot 组件初始化
│   ├── bus/                              # 消息总线
│   ├── command/                          # 命令系统 (Command 模式)
│   │   ├── Command.java
│   │   ├── CommandContext.java
│   │   ├── CommandRegistry.java
│   │   └── impl/                          # /exit, /help, /init, /mode, /resume
│   ├── config/                           # 配置系统
│   ├── core/                             # 核心引擎
│   │   ├── AgentLoop.java                # 状态机引擎 (State 模式)
│   │   ├── AgentRunner.java              # LLM 调用循环
│   │   ├── TaskStore.java                 # 任务追踪
│   │   ├── TurnContext.java
│   │   ├── TurnState.java
│   │   ├── state/                        # State 处理器 (NEW)
│   │   │   ├── AgentState.java
│   │   │   ├── RestoreState.java / CompactState.java / CommandState.java
│   │   │   ├── BuildState.java / RunState.java
│   │   │   └── SaveState.java / RespondState.java
│   │   ├── hook/                         # 钩子系统
│   │   └── subagent/                     # 子 Agent
│   ├── cron/                             # 定时任务
│   ├── identity/                         # 身份系统 (SOUL/IDENTITY/USER)
│   ├── mcp/                              # MCP 协议
│   ├── memory/                           # 记忆存储
│   ├── providers/                        # LLM 提供商
│   │   ├── LLMPProvider.java
│   │   ├── ProviderFactory.java          # 策略工厂 (NEW)
│   │   └── impl/                          # OpenAI, DeepSeek
│   ├── rules/                            # 规则系统
│   ├── security/                         # 安全系统
│   │   ├── PermissionManager.java
│   │   ├── PermissionMode.java           # PLAN/DEFAULT/ACCEPT_EDITS/BYPASS
│   │   ├── guard/                        # PathGuard/CommandGuard/NetworkGuard
│   │   ├── hook/                         # PreToolUseHook
│   │   └── rule/                          # RuleEngine
│   ├── session/                          # 会话管理
│   │   ├── SessionManager.java           # 业务层
│   │   └── SessionStore.java             # 存储层 (NEW)
│   ├── skill/                            # 技能系统
│   ├── tools/                            # 工具系统 (17+ 内置工具)
│   │   ├── Tool.java / ToolRegistry.java
│   │   └── impl/                          # 文件、搜索、Shell、Web、Task、AskUser
│   └── v1/                               # V1 独立模式 (Nanobot.java + ChannelServer)

```

```

|   |   | v2/                                # V2 Spring Boot (NanobotApplication + REST + WS)
|   |   | | controller/                      # ChatController, SessionController, HealthController
|   |   | | | websocket/                     # NanobotWebSocketEndpoint
|   |   | | v3/                              # V3 CLI 模式 (NanobotCliApplication)
|   |   | | | cli/                          # CliChannel (JLine 终端 + Markdown 渲染)
|   |   | | | tui/                          # MarkdownRenderer
|   |   | src/main/resources/
|   |   | | config/config.yaml
|   |   | | logback.xml / logback-cli.xml
|   |   | | static/                         # sessions.html (会话管理前端)

```

4.4 核心接口设计

4.4.1 Tool 接口

工具是 Agent 与外部世界交互的核心方式：

```

public interface Tool {
    String getName();                // 工具名称
    String getDescription();         // 工具描述
    JsonNode getParameters();        // 参数 JSON Schema
    boolean isReadOnly();            // 是否只读
    boolean isExclusive();           // 是否独占执行
    CompletableFuture<Object> execute(Map<String, Object> params); // 执行方法
}

```

4.4.2 LLMPProvider 接口

统一的 LLM API 调用接口：

```

public interface LLMPProvider {
    String getName();
    CompletableFuture<LLMResponse> chat(List<Message> messages, List<JsonNode> tools);
    CompletableFuture<LLMResponse> chatStream(List<Message> messages,
                                              List<JsonNode> tools,
                                              Consumer<String> onDelta);
}

```

4.4.3 AgentHook 接口

生命周期钩子扩展点（Chain of Responsibility 模式）：

```
public interface AgentHook {
    default CompletableFuture<Void> beforeIteration(AgentHookContext ctx) {...}
    default CompletableFuture<Void> beforeExecuteTools(AgentHookContext ctx) {...}
    default CompletableFuture<Void> afterIteration(AgentHookContext ctx) {...}
    default String finalizeContent(AgentHookContext ctx, String content) { return
content; }
    String getName();
}
```

4.4.4 AgentState 接口 (State 模式)

```
public interface AgentState {
    TurnState execute(TurnContext ctx);
}
// 实现类: RestoreState, CompactState, CommandState, BuildState, RunState, SaveState,
RespondState
```

4.4.5 Command 接口 (Command 模式)

```
public interface Command {
    String name();
    default List<String> aliases() { return List.of(); }
    String description();
    boolean execute(CommandContext ctx, String input);
}
// 实现类: ExitCommand, HelpCommand, InitCommand, ModeCommand, ResumeCommand
```

4.4.6 ProviderStrategy 接口 (Strategy 模式)

```
public interface ProviderStrategy {
    boolean supports(String model);
    LLMProvider create(Config config, String model);
}
// ProviderFactory 遍历注册的策略, 首个 match 的创建 Provider
```

4.4.7 MCP 相关接口

MCPClient 接口 - MCP 客户端接口:

```
public interface MCPClient {
    CompletableFuture<MCPResult> callTool(String toolName, Map<String, Object>
arguments);
    CompletableFuture<List<MCPToolInfo>> listTools();
    CompletableFuture<MCPResult> readResource(String uri);
    CompletableFuture<MCPResult> getPrompt(String promptName, Map<String, Object>
arguments);
    void close();
    boolean isConnected();
    String getServerName();
}
```

MCPToolWrapper 类 - 将 MCP 工具包装为 Nanobot Tool:

```
public class MCPToolWrapper implements Tool {
    private final MCPClient client;
    private final MCPToolInfo toolInfo;
    private final String qualifiedName; // mcp_<server>_<tool>

    @Override
    public CompletableFuture<Object> execute(Map<String, Object> params) {
        return client.callTool(toolInfo.getName(), params)
            .thenApply(result -> result.toString());
    }
}
```

五、核心模块详解

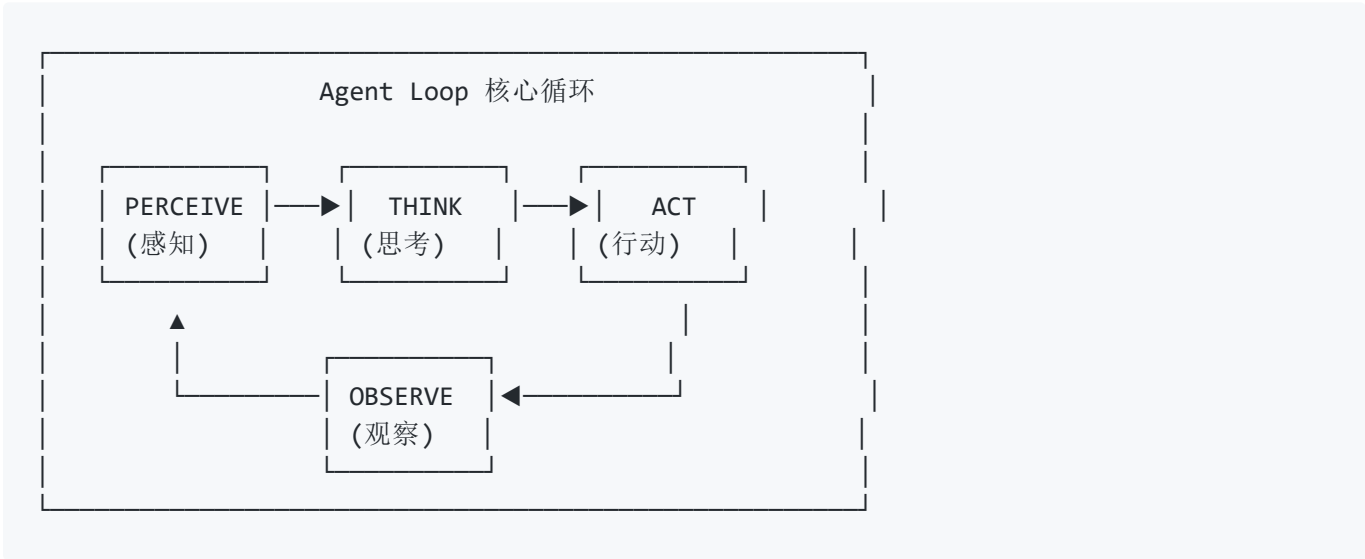
阅读目标：以下按"从能跑到跑得好"的顺序逐一剖析每个核心模块的设计与实现。建议先通读 5.1（Agent Loop）建立主线，其他模块按需深入。

5.1 Agent Loop — 核心循环 ★

Agent Loop 是 AI Agent 的心脏。理解它就理解了 Agent 是如何"活着"的。

5.1.1 什么是 Agent Loop

Agent Loop 是 AI Agent 的核心控制循环，遵循 **感知→思考→行动→观察** 的基本模式：



一次循环的完整过程：

阶段	对应组件	做什么
PERCEIVE	MessageBus.consumeInbound()	从消息队列取出用户输入
THINK	AgentRunner.run()	调用 LLM，解析响应（文本 or 工具调用）
ACT	Tool.execute()	并行执行 LLM 请求的工具（读文件/搜索/Shell...）
OBSERVE	executeTools() → messages.add()	工具结果注入消息历史，准备下一轮
LOOP	runInternal(iteration+1)	递归回到 THINK，直到 LLM 不再请求工具

5.1.2 双层架构



AgentLoop 职责（状态机层）：

- 恢复会话历史、压缩过长的上下文

- 构建 System Prompt (身份+NANOBOT.md+PlanMode+Rules)
- 调度 RUN 状态, 保存结果, 发送响应
- 管理生命周期钩子和流式回调

AgentRunner 职责 (执行引擎层) :

- 调用 LLM API (chat/chatStream)
- 解析 tool_calls, 执行工具 (并行) , 收集结果
- 递归循环直到 LLM 返回纯文本
- 防护: 迭代上限、费用上限、降级兜底

5.1.3 关键设计决策

决策	模式	理由
双层分离	AgentLoop(状态机) + AgentRunner(循环)	状态管理和 LLM 调用独立演进
递归非循环	runInternal() 递归	每轮自然携带更新后的 messages
State 模式	7 个独立 StateHandler	新增状态只需实现接口+注册
工具并行执行	CompletableFuture.allOf	同轮多工具同时跑, 减少延迟
流式双路径	SSE直推 + MessageBus	覆盖 HTTP 同步和 WebSocket 两种场景
单线程消费	单 daemon 线程	保证同一会话消息串行, 防并发冲突

5.1.4 循环终止条件

AgentRunner 在以下任一条件满足时停止循环:

1. LLM 返回纯文本 (无 tool_calls) → 正常结束
2. 达到 maxToolIterations (默认 100) → 配额耗尽
3. 达到 maxTurns → 轮次上限
4. 费用超 maxCost → 预算用尽
5. 用户取消 (Esc 中断) → 手动停止
6. 连续 3 次工具失败 → 降级兜底, 强制不带工具回答

5.1.5 完整循环示例

用户: "帮我查今天天气, 然后保存到 weather.txt"

▼ PERCEIVE: MessageBus 收到消息

▼ THINK [iteration=0]: 调用 LLM

LLM 返回: tool_calls=[web_search("今天天气"), write_file("weather.txt", ...)]

▼ ACT: 并行执行 web_search + write_file

web_search → "北京今天晴, 25°C"

write_file → 写入成功

▼ OBSERVE: 工具结果注入 messages

▼ THINK [iteration=1]: 再次调用 LLM (messages 含搜索结果+写入结果)

LLM 返回: "已查询天气并保存到 weather.txt, 北京今天晴, 25°C"

(无 tool_calls → 循环结束)

▼ RESPOND: 发送响应给用户

5.2 状态机流程 (已重构为 State 模式)

AgentLoop 采用 State 模式 管理消息处理, 每个状态对应一个独立的 AgentState 实现类:



State 模式优势:

- 每个状态独立文件 (core/state/*.java), 职责清晰
- 新增状态只需实现 AgentState 接口并注册
- AgentLoop 从 973 行精简到 579 行

状态转换表:

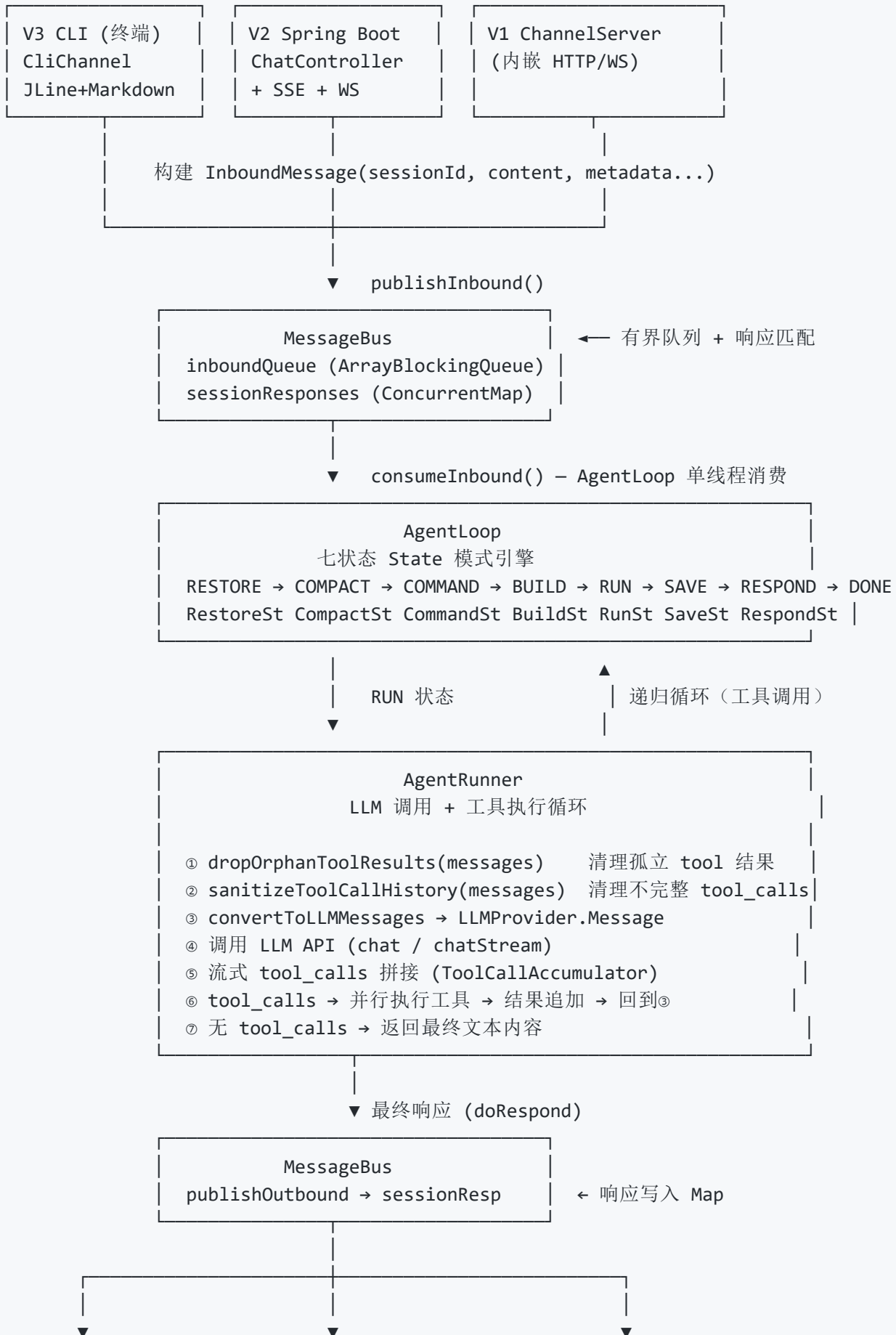
当前状态	事件	下一状态	说明
RESTORE	ok	COMPACT	加载会话历史
COMPACT	ok	COMMAND	压缩历史消息
COMMAND	dispatch	BUILD	分发普通消息
COMMAND	shortcut	DONE	处理快捷命令（如 <code>/stop</code> ）
BUILD	ok	RUN	构建 LLM 上下文
RUN	ok	SAVE	执行 LLM 调用和工具
SAVE	ok	RESPOND	保存会话状态
RESPOND	ok	DONE	发送响应

5.3 核心消息处理全链路详解

本节目标：完整梳理一条用户消息从进入系统到生成响应的全链路，理解每一层的职责、数据流转和关键代码路径。

5.3.1 端到端流程图

【消息入口层】— 三种渠道



HTTP 同步响应	SSE 流式推送	WebSocket 推送
waitForSessionResp	StreamRespCallback	StreamRespCallback
(按 requestId 匹配)	(onStreamData 直推)	(onStreamData 直推)

5.3.2 阶段一：消息入口 —— 三种渠道

系统支持三种渠道接收用户消息，最终都构建 InboundMessage 发布到 MessageBus 。

渠道 A：Spring Boot REST API (controller/ChatController.java)

端点	模式	机制
POST /api/chat	同步	发布消息后 waitForSessionResponse() 阻塞等待（默认 120s），按 requestId 精确匹配
POST /api/chat/stream	流式 SSE	注册 StreamResponseCallback 到 AgentLoop，通过 SseEmitter 实时推送每个 token

渠道 B：Jakarta WebSocket (websocket/NanobotWebSocketEndpoint.java)

- 端点： ws://host:port/ws
- @OnMessage 解析 JSON → type 为 "chat" → 构建 InboundMessage
- WebSocket 默认启用流式模式 (streamMode: true)
- 流式响应通过 StreamResponseCallback → session.getBasicRemote().sendText()

渠道 C：内嵌 HTTP 服务器 (channels/ChannelServer.java)

- 独立模式下的内嵌 HTTP 服务器 (com.sun.net.httpserver.HttpServer)
- ChatHandler 处理 POST /api/chat，支持流式 SSE 和同步两种模式
- WebSocketHandler 处理 /ws 的 WebSocket 升级握手， WebSocketConnection 管理帧读写

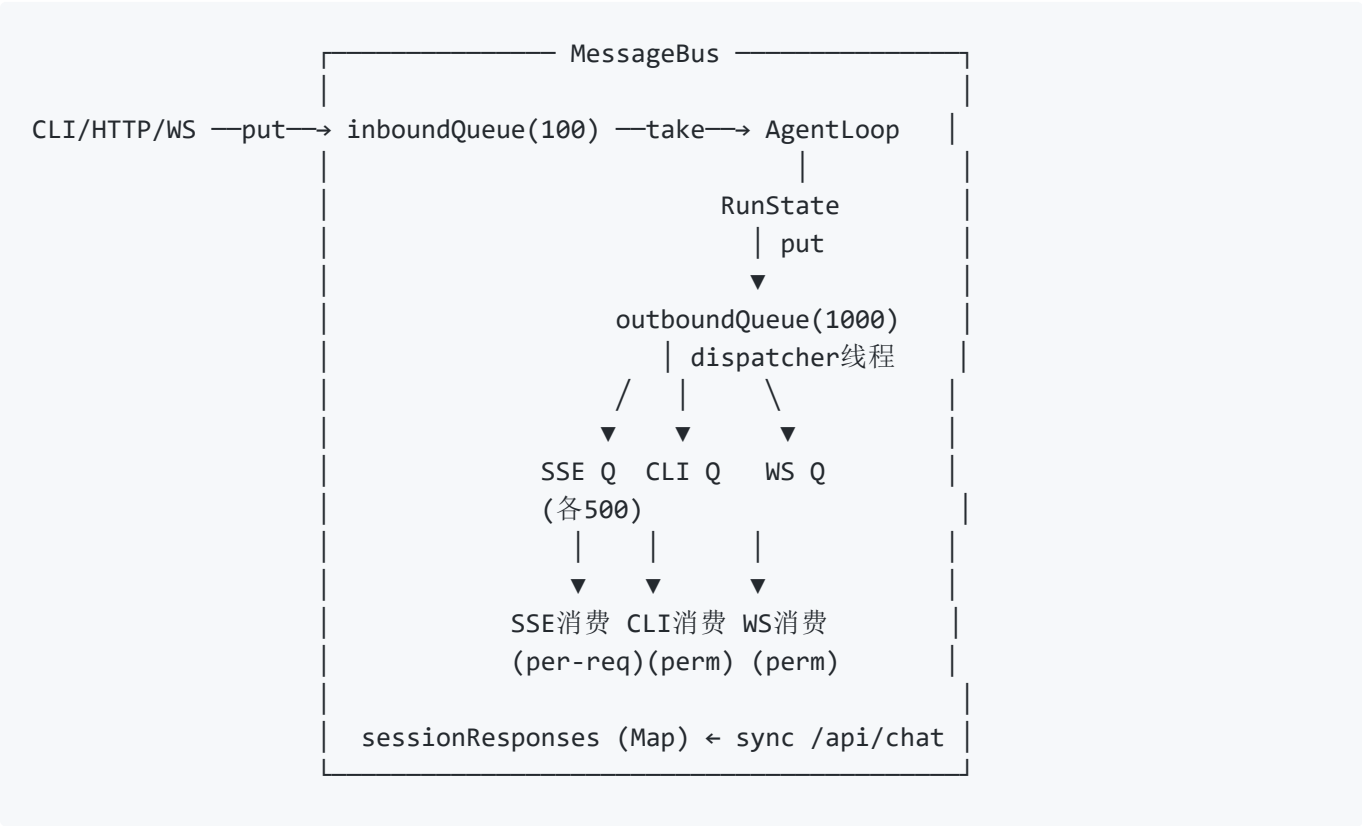
InboundMessage 核心字段：

字段	说明
channel	来源通道: "api", "websocket", "http" 等
senderId	发送者 ID
sessionId	会话标识, 用于会话隔离
content	用户文本内容
metadata	元数据 Map, 包含 requestId, streamMode, useSearch, connectionId 等
sessionKey	会话键, 格式 "{channel}:{chatId}", 支持 sessionKeyOverride 覆盖

5.3.3 阶段二：MessageBus —— 异步消息总线 (含 Outbound 发布-订阅)

文件: bus/MessageBus.java

MessageBus 是系统的**中枢神经**, 三队列架构实现完全的生产者-消费者解耦:



队列	容量	生产者	消费者	用途
inboundQueue	100	CLI/HTTP/WS 入口	AgentLoop 单线程	所有用户消息异步入队
outboundQueue	1000	RunState (每个流式token)	Dispatcher daemon 线程	流式 token 扇出到各订阅者
subscriberQueues	500/个	Dispatcher	各通道独立 consumer 线程	SSE/CLI/WS 各自独立消费
sessionResponses	Map	RespondState	ChatController 轮询	sync /api/chat 请求-响应匹配

核心API：

方法	说明
<code>publishInbound(msg)</code>	阻塞写入入站队列（ <code>put</code> ），队列满时背压阻塞
<code>consumeInbound(timeout)</code>	带超时阻塞消费，AgentLoop 主循环使用
<code>publishToOutboundQueue(msg)</code>	RunState 发布流式 token，阻塞 <code>put</code>
<code>subscribeToOutbound()</code>	获取独立 subscriberQueue，返回 <code>BlockingQueue<OutboundMessage></code>
<code>unsubscribeFromOutbound(q)</code>	移除订阅者（SSE 连接关闭/CLI 退出时调用）
<code>publishOutbound(msg)</code>	sync /api/chat 专用：写入 sessionResponses Map
<code>waitForSessionResponse(sid, rid, timeout)</code>	sync /api/chat 专用：轮询 sessionResponses 按 requestId 匹配

设计要点：

- 有界队列防 OOM：inbound 100、outbound 1000、subscriber 500
- Dispatcher 用 `offer(msg, 100ms)` 扇出——某 subscriber 慢不会阻塞其他
- AgentLoop 不持有任何消费者引用——完全通过 Queue 解耦
- sessionResponses 过期清理：cleanup daemon 每 2 分钟删除超过 5 分钟未被 `waitForSessionResponse` 取走的残留条目

消息清理链路：

层级	清理机制	时效
outboundQueue	dispatcher poll() 移除引用	≤1s
subscriberQueue	consumer poll() 移除引用	≤500ms
零订阅者	publishToOutboundQueue 直接 return	即时
unsubscribe	Queue 引用移除，GC 回收残留	即时
msg 对象本身	所有 Queue 释放引用后 GC 回收	取决于 GC
sessionResponses 正常	waitForSessionResponse 匹配后 it.remove()	即时
sessionResponses 残留	cleanup daemon 每 2 分钟清理 >5min 条目	≤7min

注：Queue 之间传递的是对象引用，不是拷贝。outboundQueue.poll() 只是将引用从队列数组中移除——msg 对象本身仍在堆上，被 dispatcher 局部变量、各 subscriberQueue 依次持有。整个过程只有一份数据。

5.3.4 阶段三：AgentLoop —— 七状态状态机引擎

文件：core/AgentLoop.java

AgentLoop 作为消息消费者，是状态机的主引擎。采用 State 模式（详见 5.2）：`RESTORE` → `COMPACT` → `COMMAND` → `BUILD` → `RUN` → `SAVE` → `RESPOND` → `DONE`。每轮完整循环处理一条入站消息。

5.3.5 阶段四：AgentRunner —— LLM 调用 + 工具执行循环

文件：core/AgentRunner.java

这是 Agent “智能” 的核心——LLM 调用和工具执行的递归循环：



防护机制 → 详见第 3.10 节 (生产化)

5.3.6 阶段五: LLM Provider 调用

文件: `providers/impl/DeepSeekProvider.java`

Provider 层封装了 LLM API 的调用细节:

- 同步调用 `chat()` → 解析 `choices[0].message.content` + `tool_calls`
- 流式调用 `chatStream()` → 逐行读 SSE → `ToolCallAccumulator` 拼接工具参数 → 通过回调实时推送 delta
- 消息转换 `convertToLLMMessages()` → 将内部 Map 格式转为 Provider 的消息对象

DeepSeek 的 ToolCallAccumulator: 流式返回的 `tool_calls` 参数是碎片化的字符片段, 需要累积拼接成完整 JSON 后解析。

5.3.7 阶段六: 响应返回 —— Outbound 发布-订阅

流式和同步走不同路径:

- **流式 (SSE/CLI/WS)**: `RunState` → `publishToOutboundQueue` → `Dispatcher` 扇出 → 各通道 `subscriberQueue` → 独立 `consumer` 线程 `poll` + 过滤 → 推送给用户

- **同步 (/api/chat) :** RespondState → publishOutbound → sessionResponses Map → waitForSessionResponse() 轮询匹配

详见 5.4 流式输出机制。

5.3.8 完整数据流示例

用户输入: "帮我写一个 Hello World 并保存到 Hello.java"

1. V3 CLI → CliChannel.handleInput("帮我写一个...")
 - InboundMessage(sessionId=..., content="帮我写一个...")
 - MessageBus.publishInbound(msg)
2. AgentLoop.run() → MessageBus.consumeInbound(1s)
 - RESTORE: 加载历史 → COMPACT: 无需压缩 → COMMAND: 无 / 命令
 - BUILD: 构建 System Prompt + tools 列表
 - RUN: 委托 AgentRunner.run()
3. AgentRunner.runInternal(messages, tools, 0)
 - callLLM(messages, tools)
 - LLM 返回 tool_calls:
[write_file(path="Hello.java", content="public class Hello {...}")]
 - 执行 write_file → 文件写入成功
 - messages.add(tool 结果消息)
 - 递归 runInternal(messages, tools, 1)
4. AgentRunner.runInternal(messages, tools, 1)
 - callLLM(messages, tools)
 - LLM 返回纯文本: "已创建 Hello.java, 包含 Hello World 程序"
 - 无 tool_calls → 循环结束
 - 返回内容给 AgentLoop
5. SAVE → RESPOND:
 - 流式模式: StreamResponseCallback → CliChannel → 终端输出
 - 同步模式: MessageBus.publishOutbound() → waitForSessionResponse()

5.3.9 三种运行模式对比

维度	V1 独立模式	V2 Spring Boot	V3 CLI
通信协议	内嵌 HTTP + WebSocket (手动帧解析)	REST + SSE + Jakarta WebSocket	stdin/stdout (纯终端)
流式输出	SSE (内嵌服务器)	SSE (Spring SseEmitter) + WS	终端直接打印 (MarkdownRenderer)
会话管理	内存	SessionStore (JSONL 文件)	SessionStore (JSONL 文 件)
并发处理	单线程 Channel Loop	Spring Boot 线程池 + MessageBus	单线程 (CLI 天然单用 户)
适用场景	学习、调试	生产部署、Web 集成	开发者日常使用

5.3.10 关键设计决策与要点

决策	说明
异步解耦	消息入口与 AgentLoop 通过 MessageBus 解耦，入口只负责发消息，不关心处理逻辑
统一消息模型	所有渠道 (CLI/HTTP/WS) 转换为统一的 InboundMessage，核心引擎不感知渠道差异
按 requestId 精确匹配	HTTP 同步模式下，sessionResponses 按 sessionId → requestId 两级匹配
流式发布-订阅	RunState → Outbound Queue → Dispatcher 扇出 → 各通道 subscriberQueue 独立消费
递归执行循环	AgentRunner 用递归而非 while 循环，更自然地表达"每轮处理新消息"的语义
单线程消费	AgentLoop 单 daemon 线程消费入站消息，避免并发修改同一会话的 messages 列表

5.4 流式输出机制 ★

5.4.1 流式 vs 非流式

模式	用户体验	实现复杂度	适用场景
非流式	等 5-30 秒后一次性显示	低（一次 HTTP 请求）	后台任务、API 调用
流式	逐字实时显示	中（SSE 解析 + 回调）	聊天、CLI、Web UI

5.4.2 数据流（发布-订阅架构）



5.4.3 关键组件

组件	职责
DeepSeekProvider.chatStream()	发送流式请求，逐行读 SSE
ToolCallAccumulator	按 index 累积碎片化 tool_calls 参数 → 构建完整 JSON
RunState.buildOnDelta()	每个 token 包装为 OutboundMessage → publishToOutboundQueue()
MessageBus.dispatcher	daemon 线程：从 outboundQueue poll → 逐个 offer 到各 subscriberQueue
subscriberQueue	每个通道独立持有，消费者线程 poll 消费

5.4.4 DeepSeek 流式参数特殊性

DeepSeek 的流式 tool_calls 参数是**逐个字符返回**的：

```
第1帧: tool_calls[0].function.arguments = "{ \"pa\"
第2帧: tool_calls[0].function.arguments = \"th\": \"
第3帧: tool_calls[0].function.arguments = \"\"/tmp\"
...累积拼接成: { \"path\": \"/tmp/test.txt\" }
```

`ToolCallAccumulator` 按 index 将参数片段追加到 `StringBuilder`，流结束后解析完整 JSON 构建 `ToolCall` 对象。

5.4.5 架构演进：回调模式 → 发布-订阅 ★

本节记录了一次重要的架构升级——从“AgentLoop 持有回调列表遍历广播”到“MessageBus Outbound Queue 扇出 Pub-Sub”。

旧架构（回调模式，v1）：

```
AgentLoop.streamResponseCallbacks (CopyOnWriteArrayList)
|
|   RunState.buildOnDelta():
|   for (callback : callbacks) {
|       callback.onStreamData(sessionId, requestId, delta); // 同步遍历
|   }
|
| — SSE callback (每请求注册/注销)
| — CLI callback (常驻)
| — WS callback (常驻)
```

新架构（发布-订阅，v2，当前）：

```
RunState → publishToOutboundQueue(delta)
|
▼
outboundQueue(1000) → dispatcher daemon → 扇出到各 subscriberQueue
|
| — SSE subscriber queue → consumer thread → emitter.send()
| — CLI subscriber queue → consumer thread → System.out.print()
| — WS subscriber queue → consumer thread → session.sendText()
```

对比：

维度	回调模式（旧）	发布-订阅（新）
AgentLoop 感知消费者	是——持有 callback 列表	否——只 put Queue
新增通道成本	改 AgentLoop 代码 + 注册回调	订阅 subscriberQueue 即可
消费者错误隔离	一个回调抛异常影响后续遍历	完全隔离，dispatcher 内部 try/catch
背压	无——慢回调拖慢所有消费者	有——subscriberQueue 满时丢弃该订阅者消息
内存管理	SSE 需手动 removeCallback	unsubscribe + 终止线程即可
耦合度	高——AgentLoop ↔ 具体消费者	低——AgentLoop ↔ Queue ↔ 消费者
可测试性	需 mock AgentLoop	独立测试 subscriberQueue 消费逻辑
延迟	约 0μs（直接内存调用）	约 2-20μs（两次 put + 唤醒）
LLM token 间隔	20-100ms	20-100ms（Queue 开销可忽略）

为什么升级？

- 1. **设计一致性**：MessageBus 的 Inbound 已经是 Queue 模式（生产者-消费者），Outbound 却用回调——不伦不类。统一为 Queue 模式让架构更清晰。
- 2. **AgentLoop 职责单一化**：AgentLoop 的核心是"处理消息的状态机"，不是"管理流式订阅者"。删除 50+ 行回调管理代码后，AgentLoop 更聚焦。
- 3. **与 Python 原版对齐**：港大 nanobot-python 的 MessageBus 本就是 Inbound+Outbound 完整总线设计。Java 版最初移除 Outbound 是因为只有同进程消费者，但由此引入了回调耦合。恢复 Outbound 的同时用 Fan-Out 解决了"多消费者不能共享 BlockingQueue"的问题。
- 4. **为未来通道做铺垫**：如果将来要接 Telegram/Discord Adapter，新架构下只需 `subscribeToOutbound()` + 启动 consumer 线程，完全不用动 AgentLoop。

适用场景判断：

场景	推荐模式	原因
同进程、消费者稳定（1-3个）	发布-订阅	解耦清晰，开销可忽略
同进程、消费者频繁变化（每次请求新建）	发布-订阅	unsubscribe 自动清理，无泄漏
极高吞吐（10万+ token/s）	回调	避免 Queue 开销（但 Agent 场景不存在）
分布式（消费者在独立进程）	发布-订阅 + 外部 MQ	Queue 可替换为 Redis/Kafka

5.5 工具系统 ★

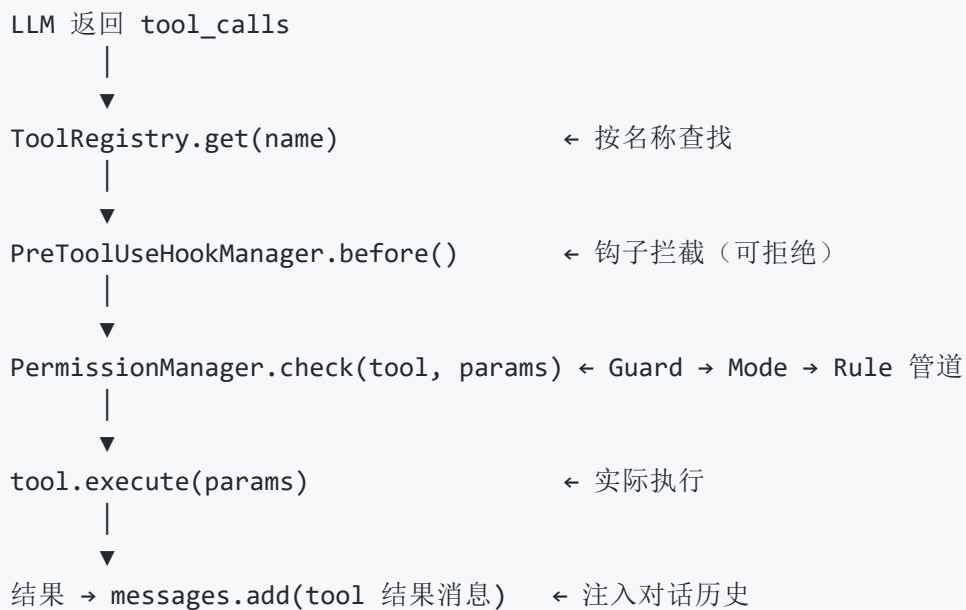
5.5.1 设计理念

- **接口简洁**：6 个方法，新工具只需实现接口
- **自动发现**：`@ToolDef` 注解标记，`ToolScanner` 扫描注册
- **安全过滤**：`PermissionManager` + `PreToolUseHookManager` 执行前拦截
- **Plan Mode 过滤**：只读模式下仅暴露 `isReadOnly()=true` 的工具

5.5.2 17 个内置工具 ★

工具	类	只读	说明
<code>read_file</code>	ReadFileTool	✓	读取文件内容，支持行范围和 offset/limit
<code>write_file</code>	WriteFileTool	✗	写入文件内容
<code>edit_file</code>	EditFileTool	✗	精确字符串替换编辑
<code>list_dir</code>	ListDirTool	✓	列出目录内容
<code>glob</code>	GlobTool	✓	按模式匹配文件 (<code>**/*.java</code>)
<code>grep</code>	GrepTool	✓	按正则搜索文件内容
<code>exec</code>	ExecTool	✗	执行 Shell 命令
<code>web_search</code>	WebSearchTool	✓	网页搜索
<code>web_fetch</code>	WebFetchTool	✓	获取网页内容
<code>get_current_time</code>	GetCurrentTimeTool	✓	获取当前时间（解决模型训练数据时间偏差）
<code>ask_user</code>	AskUserTool	✗	向用户提问
<code>task_create</code>	TaskCreateTool	✗	创建任务追踪
<code>task_list</code>	TaskListTool	✓	列出当前任务
<code>task_update</code>	TaskUpdateTool	✗	更新任务状态
<code>spawn</code>	SpawnTool	✗	创建子 Agent 并行执行任务
<code>spawn_check</code>	SpawnCheckTool	✓	检查子 Agent 执行结果
<code>mcp_*</code>	MCPToolWrapper	取决于 MCP 工具	MCP 服务器工具（动态加载）

5.5.3 工具执行流程



5.5.4 参数默认值

`@ToolDef(paramDefaults = {"maxResults": "5"})` — 注解方式提供默认参数，LLM 未传参时自动填入。

5.5.5 Plan Mode 工具过滤

```
/mode plan → PermissionManager.mode = PLAN
            → ToolRegistry.getPermittedTools() 只返回 isReadOnly()=true 的工具
            → LLM 看不到写工具 → 不会尝试写入
```

5.6 身份系统 (Identity)

文件: `identity/IdentityManager.java`

身份系统定义了 Agent 的"人格", 通过 YAML 配置文件定义:

配置加载优先级: SOUL.md > IDENTITY.md > USER.md
(核心性格) (角色定位) (用户偏好)

层	文件	内容
SOUL	SOUL.md	Agent 的核心性格、价值观、说话风格
IDENTITY	IDENTITY.md	角色定位、能力范围、限制
USER	USER.md	用户偏好、习惯、上下文

这三种身份在 `BuildState` 中被注入 System Prompt。

5.7 System Prompt 构建 ★

文件: `core/state/BuildState.java`

5.7.1 构建流程

```
BuildState.execute(ctx)
├─ ① 注入身份 - SOUL + IDENTITY + USER (IdentityManager)
├─ ② 加载项目记忆 - NANOBOT.md (自动生成或手动编辑)
├─ ③ 注入规则 - Rules (RuleManager 加载的编码/安全规则)
├─ ④ Plan Mode 覆盖 - 如果是 PLAN 模式，注入只读+出计划指令
└─ ⑤ 工具列表 - 由 ToolRegistry.getPermittedTools() 动态生成
```

5.7.2 设计原则

原则	说明	反例
先身份后能力	先告诉 LLM "你是谁", 再说 "你能做什么"	✗ 一上来就列工具, LLM 不知道自己的角色
具体优于抽象	"用 4 空格缩进"	✗ "代码要规范"
按需注入	Plan Mode 指令只在只读模式注入	✗ 所有模式都加载, 浪费 token
分层加载	身份固定, 项目规范可变, 工具自动生成	✗ 全部写死在代码里

5.7.3 最终结构

① SOUL - 身份定义（名称+性格+能力）	← IdentityManager
② NANOBOT.md - 项目记忆（自动/手动）	← /init 命令生成
③ Rules - 编码/安全规则	← RuleManager 加载
④ Plan Mode 指令 - 只读+出计划	← 仅 /plan 模式下注入
⑤ 工具列表 - 由 API tools 参数提供	← 自动生成

5.8 Plan Mode — 先计划后执行 ★

文件： `security/PermissionMode.java` + `core/state/BuildState.java` + `command/impl/ModeCommand.java`

5.8.1 workflow全景



5.8.2 三层协同机制



5.8.3 提示词注入机制

当处于 `/mode plan` 时，`BuildState` 会将以下指令注入 System Prompt:

"你当前处于 PLAN MODE（规划模式）。规则：

1. 你只能使用只读工具（`read_file`, `glob`, `grep`, `list_dir` 等）

2. 不能使用任何写工具（`write_file`, `edit_file`, `exec` 等）

3. 分析完成后，输出一个详细的实现计划

4. 计划格式：需求理解 → 影响范围 → 实现步骤 → 注意事项

5. 用户审批后会说 `/plan approve`，届时会切换到执行模式"

5.8.4 审批执行流程

`/plan` → Agent 只读分析出计划 → 用户审查

└ 不满意 → "第三步不对，应该先改接口" → Agent 更新计划

└ 满意 → `/plan approve` → 切换到 DEFAULT 模式 → 执行

5.8.5 与 Claude Code 对标

特性	Claude Code	nanobot
进入规划	<code>/plan</code>	<code>/plan</code> 或 <code>/mode plan</code>
审批执行	用户确认	<code>/plan approve</code>
权限实现	只读工具白名单	<code>isReadOnly()</code> + <code>ToolRegistry.getPermittedTools()</code>
提示词注入	内部机制	<code>BuildState</code> 显式注入

5.9 上下文管理系统 ★

上下文管理负责让 Agent 在"记住足够信息"和"不超出 Context Window"之间取得平衡。

5.9.1 为什么需要上下文管理

LLM 的 Context Window 有限 (DeepSeek 约 128K tokens) 。当对话历史超过窗口时:

- 截断早期消息 → 丢失重要上下文
- 不处理 → API 报错

上下文管理系统通过**持久化** + **压缩** + **选择性加载**解决这个问题。

5.9.2 会话持久化

文件: `session/SessionManager.java` + `session/SessionStore.java`

Repository 模式 — I/O 与业务逻辑分离:

```
SessionManager (业务层)
├── 锁管理、并发控制、协调逻辑
└── SessionStore (存储层)
    └── 纯文件 I/O (JSONL 读写、metadata 管理)
```

存储路径: `{workspace}/sessions/{sessionId}/history.jsonl`

写入策略 — 增量追加:

每条消息立即追加到 JSONL 文件末尾, 不重写整个文件。并发写通过 `SessionManager` 的锁机制保证安全。

读取策略: 从 JSONL 流式读取全部消息 → `List<Map<String, Object>>`。

5.9.3 消息历史生命周期

```
用户消息 → InboundMessage
├── BuildState 注入 System Prompt
├── 追加历史消息 (从 JSONL 加载)
├── 追加当前用户消息
├── RUN 状态: LLM 调用 → 追加 assistant 消息
│   └── 如果有工具调用 → 追加 tool_call + tool 结果消息
├── 最终追加 assistant 文本回复
└── SaveState 保存到 JSONL
```

5.9.4 Context Window 与 Token 预算

每次 LLM 调用前检查：

```
estimatedTokens = sum(len(msg.content) / 4) ← 粗略估算：1 token ≈ 4 字符
```

```
if estimatedTokens > contextWindowTokens * 0.9:  
    → 触发 CompactState（详见 5.10 记忆系统）
```

5.9.5 记忆压缩 (Consolidator)

当 token 数超过阈值时自动触发，把旧消息发给 LLM 做摘要，替换原文。

手动触发： `/compact` 命令可随时手动压缩，无视 token 阈值。

详见 5.10 记忆系统。

5.9.6 长期记忆 (Dream)

完整闭环（提取→存储→检索→注入）：

- **自动提取（异步 + 节流）**：SaveState fire-and-forget → `dream.extractAndStore()`。每轮对话检查字符增量，增量不足 5000 字则跳过，避免频繁调 LLM
- **手动提取（同步）**：`/remember` 命令可随时手动提取，无视增量阈值，执行 `.join()` 等待 LLM 返回结果
- **注入（同步）**：BuildState → `dream.retrieve(userMessage, 5)` → 关键词匹配 → 注入 system prompt

详见 5.10 记忆系统。

5.9.7 NANOBOT.md — 项目级上下文

`/init` 命令自动分析项目并生成，每次对话自动加载。详见 5.10 记忆系统。

5.10 记忆系统 ★

记忆系统解决的核心问题：LLM 本身是无状态的，Agent 如何在跨对话、跨会话的时间维度上“记住”重要信息。

5.10.1 四层记忆模型

四层记忆模型

Layer 1: 感知记忆 (Sensory Memory)

- └ 内容: 当前轮用户的输入、LLM 的即时响应
- └ 存储: 内存变量
- └ 生命周期: 单次 agent iteration (毫秒~秒)

Layer 2: 短期记忆 / 工作记忆 (Short-term / Working)

- └ 内容: 对话历史 (messages 列表)
- └ 存储: 内存 List<Map> + JSONL 文件持久化
- └ 生命周期: 单次会话 (分钟~小时)

Layer 3: 长期记忆 (Long-term Memory / Dream)

- └ 内容: 从对话中提炼的关键信息 (偏好、决策、事实)
- └ 存储: .nanobot/memory/MEMORY.md (Markdown)
- └ 检索: 关键词匹配 + 重要性加权 (纯本地, 零延迟)
- └ 生命周期: 跨会话 (天~月)

Layer 4: 项目记忆 (Project Memory / NANOBOT.md)

- └ 内容: 项目架构、编码规范、常用命令
- └ 存储: 项目根目录 NANOBOT.md
- └ 生命周期: 手动更新 (周~月)

5.10.2 短期记忆 (工作记忆) — 对话历史

存储格式 (JSONL) :

```
{"role":"system","content":"你是 my-nanobot..."}
{"role":"user","content":"帮我写一个登录接口"}
{"role":"assistant","content":null,"tool_calls":[{"id":"call_1",...}]}
{"role":"tool","tool_call_id":"call_1","content":"文件写入成功"}
{"role":"assistant","content":"已创建 LoginController.java, 包含..."}
```

加载机制:

1. RestoreState : 从 JSONL 加载全部历史 → List<Map>
2. 每条新消息追加到列表 → 实时写入 JSONL (增量追加)
3. 下次对话时重新加载 → 完整上下文恢复

5.10.3 短期记忆压缩 (Consolidator)

文件: memory/Consolidator.java

触发条件: `estimatedTokens > contextWindowTokens * 0.9`

压缩流程:

1. 计算当前所有消息的估计 token 数
2. 取前 50% 的消息发给 LLM 做摘要
3. LLM 返回摘要文本（保留关键决策、上下文、未完成的任务）
4. 用摘要替换被压缩的消息（减少 token 占用）
5. 摘要作为 system 消息插入 messages 头部

压缩示例:

压缩前 (8000 tokens):

用户: "帮我分析这个项目的认证模块"

AI: "让我先读一下项目结构..." [大段分析]

用户: "重点关注 JWT 部分"

AI: "JWT 实现用了 jjwt 库..." [详细说明]

...

压缩后 (~500 tokens):

system: "【对话摘要】用户要求分析项目认证模块。

项目使用 Spring Security + jjwt 实现 JWT 认证，
token 有效期 24h，存储在客户端 localStorage。"

[最近 3 轮对话保留原文]

5.10.4 长期记忆 (Dream) — 完整闭环 ★

文件: `memory/Dream.java` + `memory/MemoryLoader.java`

✅ **完整闭环已实现** (2026-07-21) : 提取→存储→检索→注入，四步全通。

5.10.4.1 存储

`.nanobot/memory/MEMORY.md` ← MemoryLoader 读写

Markdown + YAML Frontmatter 格式，按关键词分组:

```
---
name: project-memory
description: 项目长期记忆
updated: 2026-07-21 13:39:00
---

# 长期记忆

## Auth
- 2026-07-21: 用户偏好 JWT 认证而非 Session

## Api
- 2026-07-21: 项目 API 前缀统一用 /api/v1
```

5.10.4.2 提取（异步 + 增量节流，fire-and-forget）

自动触发机制：每轮对话 SAVE 阶段触发，但受增量节流控制。

节流策略： `EXTRACTION_MIN_NEW_CHARS = 5000` (≈ 1500 tokens, 约 3-5 轮对话)

```
extractAndStore(sessionId, messages):
    currentChars = countTotalChars(messages)
    lastChars = lastExtractionCharCount[sessionId]
    newChars = currentChars - lastChars

    if newChars < 5000 && lastChars > 0:
        → 跳过, return 空列表      ← 零 LLM 调用, 零费用

    lastExtractionCharCount[sessionId] = currentChars
    → 继续提取...
```

场景	行为
新会话首次提取	不跳过 (<code>lastChars==0</code>) , 立即提取
后续轮次增量 < 5000 字	跳过, 不调 LLM
后续轮次增量 $\geq$ 5000 字	触发提取, 更新基线
<code>/remember</code> 手动命令	无视阈值, 强制触发

提取流程：

```

SaveState.execute()
└─ dream.extractAndStore(sessionId, messages) // CompletableFuture, 不阻塞
    └─ countTotalChars() → 增量检查 → 不足则跳过
        └─ analyzeMessages(messages) // 调 LLM 分析对话
            └─ LLM 返回 JSON: [{content, keywords, importance}]
                └─ parseMemoryResponse() // Jackson 解析
                    └─ storeMemory() × N // 存入 ConcurrentHashMap
                        └─ saveToMemoryFile() // 持久化到 MEMORY.md

```

Prompt 模板：

请从以下对话中提取值得长期记忆的信息：

user: 我喜欢用 JWT 做认证, 不喜欢 Session

assistant: 好的, 我来用 JWT 实现...

...

请以 JSON 格式输出记忆条目, 每个条目包含：

- content: 记忆内容
- keywords: 关键词数组
- importance: 重要性(0-1)

LLM 返回示例：

```

[
  {"content": "用户偏好 JWT 认证而非 Session", "keywords": ["auth", "jwt",
"preference"], "importance": 0.8},
  {"content": "项目 API 前缀统一用 /api/v1", "keywords": ["api", "convention"],
"importance": 0.7}
]

```

容错： LLM 返回非 JSON 时降级——将全文作为单条 importance=0.3 的记忆存储。

5.10.4.3 检索（同步，纯本地计算）

`dream.retrieve(query, limit)` 的检索策略——**不用 LLM，不用 embedding，零延迟零成本：**

对每条记忆计算相似度分：

score = 0

内容词重叠：query 分词后，每命中一个词
→ +0.2 每个词

关键词匹配：记忆的 keywords 在 query 中
→ +0.3 每个匹配

重要性加成：importance × 0.5
→ +0.0 ~ +0.5

总分 cap 在 1.0

排序 → 取 top N → 返回

特性	值
算法	关键词重叠 + 重要性加权
复杂度	$O(n \times w)$ (n=记忆数, w=query词数)
LLM 调用	无
延迟	<1ms (几百条记忆内)

优劣：

- ☒ 零延迟、零成本、简单可解释
- ☐ 无语义理解——"认证"和"JWT"不互通，"K8s"和"Kubernetes"不匹配
-  可升级为 embedding 向量检索（需引入向量数据库或本地 ONNX 模型）

5.10.4.4 注入 (BUILD 阶段)

`BuildState.appendMemories()` 在构建 system prompt 时调用检索 → 注入：

System Prompt 构成：

- | | |
|---------------------------|---------------|
| 1. 身份信息 (IdentityManager) | ← ★ 新增，最多 5 条 |
| 2. 联网搜索提示 | |
| 3. 长期记忆 (Dream.retrieve) | |
| 4. NANOBOT.md 项目记忆 | |
| 5. Plan Mode 规则 | |
| 6. Rules 规则 | |

注入格式：

【长期记忆 - 从过往对话中自动提取】

- 用户偏好 JWT 认证而非 Session
- 项目 API 前缀统一用 /api/v1

效果：下次对话时，即使用户不重提 JWT，LLM 也能从长期记忆中"回想"起这个偏好。

5.10.4.5 完整生命周期

启动时：

Dream 构造 → `MemoryLoader.loadFromFile(.nanobot/memory/MEMORY.md)`
→ 加载已有记忆到内存 `ConcurrentHashMap`

每次对话 BUILD 阶段：

`BuildState.appendMemories()`
→ `dream.retrieve(userMessage, 5)`
→ 关键词匹配排序 → 注入 system prompt

每次对话 SAVE 阶段：

`SaveState.triggerMemoryExtraction()`
→ `dream.extractAndStore(sessionId, messages)`
→ LLM 异步提取 → 写入内存 + 持久化文件

闭环示意：

对话1

|
|
|— SAVE: Dream 提取记忆
| "用户偏好 JWT"
| → MEMORY.md
|
|

对话2（几天后）

|
|
|— BUILD: Dream 检索记忆
| query="帮我加个认证"
| → 检索到 "用户偏好 JWT"
| → 注入 system prompt
| → LLM 直接给 JWT 方案，无需重问

阶段	触发	操作	方向
启动	Dream 构造	<code>loadFromMemoryFile()</code> 加载 MEMORY.md → 内存	磁盘 → 内存
BUILD	BuildState	<code>retrieve(query, 5)</code> → 注入 system prompt	内存 → LLM上下文
SAVE	SaveState	<code>extractAndStore()</code> → LLM分析 → 写 MEMORY.md	对话 → 磁盘



5.11 安全系统详解

文件： `security/PermissionManager.java` + `security/guard/` + `security/rule/`

架构：



交互确认 (DEFAULT 模式下触发写操作)：

[!] 工具调用需要确认：
工具：exec
参数：{command=mvn test}
原因：Shell 命令执行需要您的确认
1=允许 2=之后都放行 3=拒绝 [1/2/3]

5.12 命令系统 (Command)

文件: `command/Command.java` + `command/CommandRegistry.java` + `command/impl/`

Command 模式实现:

```
public interface Command {
    String name(); // 命令名, 如 "help"
    default List<String> aliases() { return List.of(); } // 别名
    String description(); // 功能描述
    boolean execute(CommandContext ctx, String input); // 执行逻辑
    default boolean isShortcut() { return false; } // 是否短路 (不进入 Agent Loop)
}
```

命令列表:

命令	别名	功能	短路
<code>/help</code>	—	列出所有可用命令	✓
<code>/exit</code>	<code>/q</code> , <code>/quit</code>	退出 CLI	✓
<code>/init</code>	—	分析项目生成 NANOBOT.md	✗
<code>/mode <模式></code>	<code>/plan</code>	切换权限模式	✓
<code>/plan approve</code>	—	审批计划并执行	✗
<code>/resume [key]</code>	—	列出/恢复历史会话	✓
<code>/clear</code>	—	清空会话上下文	✓
<code>/compact</code>	—	手动压缩对话历史 (LLM 摘要)	✓
<code>/remember</code>	—	手动提取长期记忆 (LLM 分析)	✓

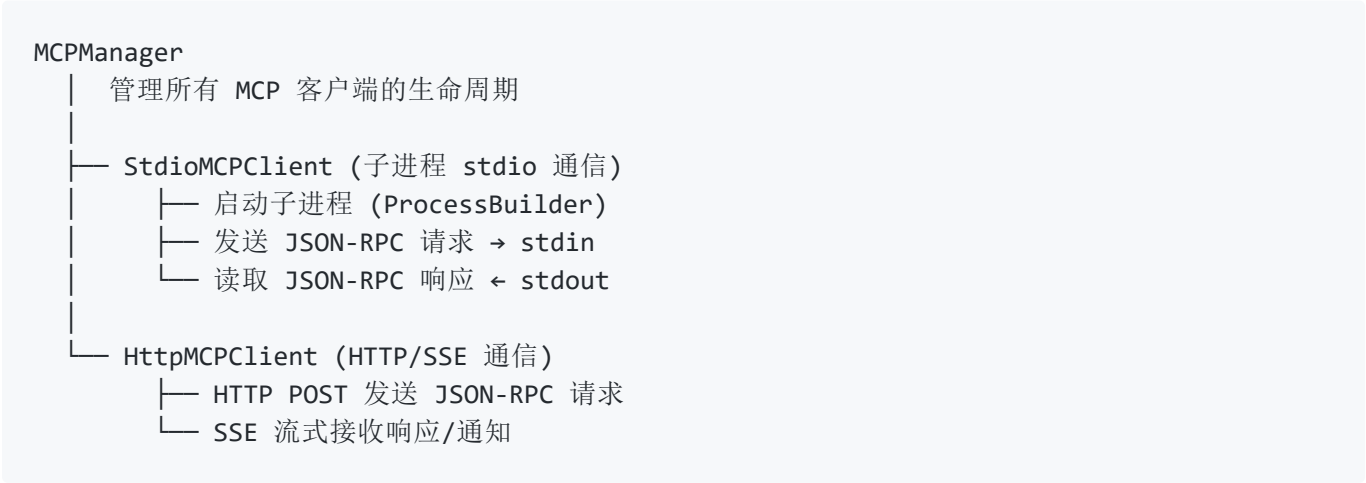
注意: `/compact` 和 `/remember` 虽标记为短路 (不进入 LLM 对话), 但内部会同步调用 LLM 完成压缩/提取, 然后直接返回结果。其余短路命令为零 LLM 调用。

短路机制: `isShortcut()=true` 的命令在 `CommandState` 直接处理, 不进 Agent Loop LLM 调用, 省 token。

5.13 MCP (Model Context Protocol) 系统

文件: `mcp/MCPManager.java` + `mcp/MCPClient.java` + `mcp/StdioMCPClient.java` + `mcp/HttpMCPClient.java`

5.13.1 MCP 架构



5.13.2 客户端实现层次 (Template Method 模式)



5.13.3 支持的传输方式

传输方式	实现类	适用场景
Stdio	<code>StdioMCPClient</code>	本地 MCP 服务器 (如 git-mcp)
HTTP + SSE	<code>HttpMCPClient</code>	远程/集中式 MCP 服务器

5.13.4 MCP 工具命名规则

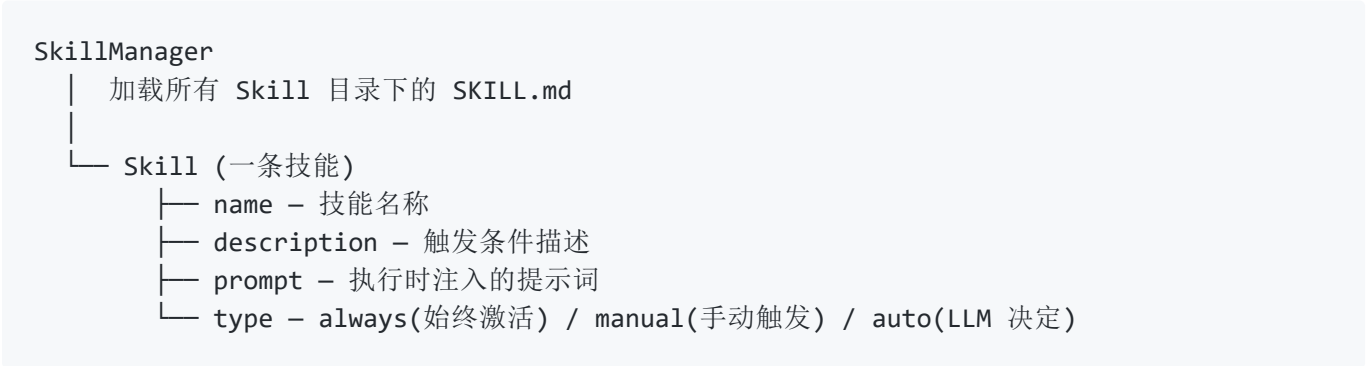
MCP 工具注册到 ToolRegistry 时自动加前缀: `mcp_{serverName}_{toolName}` (如 `mcp_git_commit`、`mcp_filesystem_read`)。

5.14 Skills 技能系统

文件: `skill/SkillManager.java`

5.14.1 Skills 架构

Skills 是可选的、LLM 可自主决定何时调用的"专业技能包"。每个 Skill 通过 `SKILL.md` 定义触发条件和提示词。



5.14.2 技能目录结构

```
skills/
├── code-review/SKILL.md      # 代码审查
├── deploy-verify/SKILL.md    # 部署验证
└── ...
```

5.14.3 SKILL.md 格式

```
# 技能名称
触发条件: ...
执行指令: ...
```

5.14.4 技能类型

类型	触发方式	示例
<code>always</code>	每次对话都激活	项目编码规范检查
<code>manual</code>	用户 <code>/skill <name></code> 触发	代码审查
<code>auto</code>	LLM 根据上下文自主决定	部署前运行测试

5.15 Rules 规则系统

文件: `rules/RuleManager.java`

5.15.1 Rules 架构

Rules 是必须遵守的约束条件（与 Skills 的"可选性"形成对比）。Rules 注入 System Prompt, Skills 注入对话上下文。

5.15.2 规则文件位置

多个位置按优先级加载：`项目根目录/rules/` → `~/.nanobot/rules/` → `classpath`

5.15.3 规则文件格式

```
# 规则标题
类型: always
优先级: high
---
规则内容...
```

5.15.4 规则类型

类型	说明
<code>always</code>	始终生效
<code>manual</code>	用户手动触发
<code>auto</code>	LLM 根据上下文自主激活

5.15.5 优先级机制

多规则冲突时按优先级链解决：`high > medium > low`。

5.16 多通道接入系统 (ChannelServer)

文件：`channels/ChannelServer.java` (V1) + `v2/controller/ChatController.java` (V2) + `v3/cli/CliChannel.java` (V3)

系统支持三种接入方式，从早期的手动帧解析到成熟的 Spring Boot REST API 到类 Claude Code 的 CLI 终端，体现了架构的演进过程。

详见 5.3.2（消息入口）和 5.3.9（三种运行模式对比）。

5.17 定时任务系统 (CronScheduler)

文件: `cron/CronScheduler.java`

支持 cron 表达式定时任务, 通过消息总线触发:

```
CronScheduler
├── ScheduledExecutorService 管理调度
└── 定时到 → 构造 InboundMessage → publishInbound()
    → AgentLoop 正常处理 (和用户消息走同一通道)
```

5.18 V3 CLI 模式 (CliChannel)

文件: `v3/cli/CliChannel.java` + `v3/tui/MarkdownRenderer.java`

核心特性:

- JLine 3 终端输入 (跨平台, 支持 Esc 键检测)
- Markdown **渲染** (代码块高亮、标题、列表、粗体/斜体)
- Esc **中断**流式回复
- 类 Claude Code 的**交互体验**

5.19 V2 Spring Boot 模式

文件: `v2/NanobotApplication.java` + `v2/NanobotConfig.java`

- Spring Boot 3.2 全功能模式
- REST API + SSE 流式 + Jakarta WebSocket
- `sessions.html` 会话管理前端
- 所有组件通过 `@Bean` 注入, `@Configuration` 统一编排

5.20 V1 独立模式

文件: `v1/Nanobot.java` + `channels/ChannelServer.java`

- 最早的实现, 内嵌 HTTP 服务器 + 手动 WebSocket 帧解析
 - 用于学习框架核心、验证概念
 - 不建议生产使用
-

5.21 子 Agent 系统 (Subagent)

文件: `core/subagent/AgentCoordinator.java` + `tools/impl/SpawnTool.java`

主 Agent 可动态创建子 Agent 执行子任务:

```
主 Agent (SpawnTool.spawn)
|
├─> searcher (搜索助手)
├─> coder (编程助手)
├─> summarizer (总结助手)
├─> calculator (计算助手)
|
└─> 子 Agent 独立执行 → 通过 SubagentCommunication 汇报结果
```

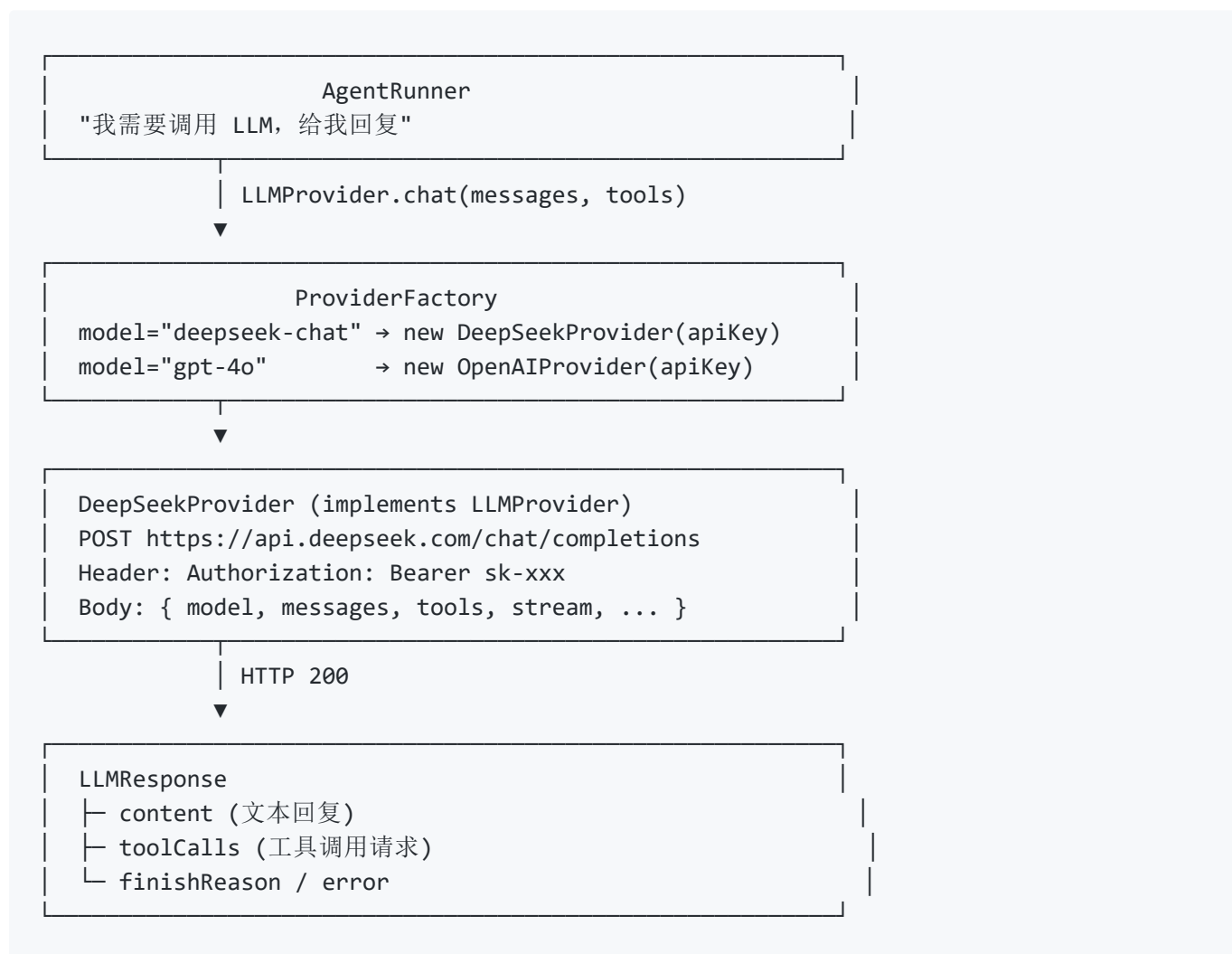
子 Agent 可注册不同类型 (各有不同能力标签), 由 `AgentCoordinator` 统一管理和调度。

5.22 LLM Provider 体系 — DeepSeek API 集成详解 ★

为什么单开一章: LLM Provider 是 Agent 与大模型交互的唯一通道。理解 DeepSeek API 的请求/响应格式、流式解析、工具调用回环, 是后续对接任何新模型的基础。

文件: `providers/LLMProvider.java` + `providers/ProviderFactory.java` + `providers/impl/DeepSeekProvider.java`

5.22.1 架构总览



设计要点:

1. **接口统一**: `LLMProvider` 定义 `chat()` / `chatStream()` 两个核心方法, 所有 `Provider` 实现同一套接口
2. **策略匹配**: `ProviderFactory` 按 `model` 名前缀匹配 `Provider` (`deepseek*` → `DeepSeek`, `gpt-*` → `OpenAI`, 兜底 → `OpenAI` 兼容)
3. **API Key 解析优先级**: 配置文件 > 环境变量 (`DEEPSEEK_API_KEY` / `OPENAI_API_KEY`)
4. **异步 `CompletableFuture`**: 所有网络 IO 通过 `CompletableFuture.supplyAsync` 异步执行

5.22.2 消息格式 (OpenAI 兼容)

DeepSeek API 与 OpenAI 完全兼容, 请求/响应格式一致。

请求体结构:

```

{
  "model": "deepseek-chat",
  "max_tokens": 8192,
  "temperature": 0.7,
  "stream": true,
  "messages": [
    {"role": "system", "content": "你是 Nanobot, 一个 AI 助手..."},
    {"role": "user", "content": "帮我读一下 pom.xml"},
    {"role": "assistant", "content": null, "tool_calls": [
      {"id": "call_001", "type": "function",
        "function": {"name": "read_file", "arguments": "{\"path\":\"pom.xml\"}"}}
    ]},
    {"role": "tool", "tool_call_id": "call_001", "content": "<xml>..."},
    {"role": "assistant", "content": "你的 pom.xml 内容如下..."}
  ],
  "tools": [
    {"type": "function", "function": {"name": "read_file", "description": "...",
  "parameters": {...}}}
  ]
}

```

四种消息角色：

role	来源	说明
system	BuildState 构建	Agent 人格、规则、当前日期、NANOBOT.md
user	用户输入	用户发送的消息
assistant	LLM 返回	AI 回复，可能包含 tool_calls （LLM 决定调工具）
tool	ToolRegistry 执行结果	工具执行后的返回值，通过 tool_call_id 关联

关键细节：DeepSeek API 要求 assistant 消息中 tool_calls[].function.arguments 必须是 JSON 字符串（如 `"{\"path\":\"pom.xml\"}"`），不能是 JSON 对象。buildRequestBody() 中专门做了格式转换。

5.22.3 非流式调用（chat）

```

AgentRunner
├── provider.chat(messages, tools)
│   ├── buildRequestBody() : 构造 JSON
│   ├── POST /chat/completions
│   └── parseResponse() : 解析 JSON → LLMResponse
└── response.isToolCall() ?
    ├── YES → AgentRunner 执行工具 → 结果追加到 messages → 再次 chat()
    └── NO → 返回文本内容给用户

```

响应解析 (`parseResponse`) :

```

// 普通文本回复
{
  "choices": [{
    "message": { "content": "你的 pom.xml 是..." },
    "finish_reason": "stop"
  }]
}
// → LLMResponse.success(content, "stop", null)

// 工具调用回复
{
  "choices": [{
    "message": {
      "content": null,
      "tool_calls": [{
        "id": "call_001",
        "function": { "name": "read_file", "arguments": "{\"path\":\"pom.xml\"}" }
      }]
    }
  }]
}
// → LLMResponse.toolCalls([ToolCallRequest("call_001", "read_file",
{path:"pom.xml"})], model)

```

5.22.4 流式调用 (chatStream) ★ 核心难点

流式调用是 DeepSeek 集成中最复杂的部分，核心难点在于 **工具调用的 arguments 是逐字符流式返回的**，需要跨 SSE delta 拼接。

SSE 数据流示例 (每行一个 `data:` 事件) :

```

data: {"choices":[{"delta":{"content":"你"}}]}
data: {"choices":[{"delta":{"content":"好"}}]}
data: {"choices":[{"delta":{"content":"! "}}]}
data: {"choices":[{"delta":{"tool_calls":[{"index":0,"id":"call_001","function":{"name":"read_file"}}]}]}
data: {"choices":[{"delta":{"tool_calls":[{"index":0,"function":{"arguments":{"\pa"}}]}]}]}
data: {"choices":[{"delta":{"tool_calls":[{"index":0,"function":{"arguments":"th\":"}}]}]}
data: {"choices":[{"delta":{"tool_calls":[{"index":0,"function":{"arguments":"\pom"}}]}]}
data: {"choices":[{"delta":{"tool_calls":[{"index":0,"function":{"arguments":".xml\":"}}]}]}
data: [DONE]

```

ToolCallAccumulator — 按 index 分桶拼接：

```

delta 1: tool_calls[0] → id="call_001", name="read_file"
delta 2: tool_calls[0] → arguments="{\pa"
delta 3: tool_calls[0] → arguments="th\":"
delta 4: tool_calls[0] → arguments="\pom"
delta 5: tool_calls[0] → arguments=".xml\":"

```

|

▼ (按 index 分桶, argsBuilder.append 拼接)

```

accumulators[0]:
  id = "call_001"
  name = "read_file"
  argsBuilder = "{\path\":"pom.xml\":" ← 完整 JSON
  |
  ▼ (buildToolCalls: 按 index 排序 → 构建 ToolCallRequest)

```

```

List<ToolCallRequest> [
  {id: "call_001", name: "read_file", arguments: {path: "pom.xml"}}
]

```

为什么不直接用一个 `StringBuilder`：LLM 可能**并行调用多个工具**（`tool_calls` 数组有多项），每一项的 `arguments` 碎片交替到达。按 `index` 分桶保证每个工具的 `arguments` 独立拼接不混淆。

流式解析主循环（`chatStream()` 核心代码）：

```
// 逐行读取 SSE (Server-Sent Events)
try (BufferedReader reader = ...) {
    String line;
    while ((line = reader.readLine()) != null) {
        if (!line.startsWith("data: ")) continue; // 跳过空行和注释
        String json = line.substring(6);          // 去掉 "data: " 前缀
        if ("[DONE]".equals(json)) continue;      // 流结束信号

        JsonNode chunk = objectMapper.readTree(json);
        JsonNode delta = chunk.get("choices")[0].get("delta");

        if (delta.has("content")) {
            String text = delta.get("content").asText();
            fullContent.append(text);
            onDelta.accept(text); // 实时推送给前端
        }
        if (delta.has("tool_calls")) {
            toolCallMode = true;
            parseToolCallsFromDelta(delta.get("tool_calls"), accumulators);
        }
    }
}
// 流结束后: 从 accumulators 构建完整的 ToolCallRequest 列表
```

5.22.5 工具调用回环 (Tool Call Loop)

这是 Agent 区别于普通 Chatbot 的核心流程:

Round 1:

```
LLM → "我需要读文件" → tool_calls: [read_file("pom.xml")]
AgentRunner → ToolRegistry.execute("read_file", {path:"pom.xml"})
→ tool result 作为 tool 消息追加到 messages
```

Round 2:

```
LLM → (收到文件内容) → "文件内容是...建议你..." → content: "建议你..."
→ finish_reason="stop" → 输出给用户
```

AgentRunner 循环控制 (伪码) :

```

for (int i = 0; i < maxIterations; i++) {
    LLMResponse response = provider.chat(messages, tools);

    if (!response.shouldExecuteTools()) {
        return response.getContent(); // 文本回复 → 结束
    }

    for (ToolCallRequest tc : response.getToolCalls()) {
        // 权限检查
        if (webSearchDisabled && tc.getName().startsWith("web_")) continue;

        Object result = toolRegistry.execute(tc.getName(), tc.getArguments());
        messages.add(Message.ofTool(result.toString(), tc.getId()));
    }
    // 继续下一轮，LLM 看到 tool 结果后决定下一步
}

```

5.22.6 如何对接新模型

假设要接入 Mistral，只需三步：

Step 1: 创建 `MistralProvider` implements `LLMProvider`

```

public class MistralProvider implements LLMProvider {
    public CompletableFuture<LLMResponse> chat(List<Message> messages, List<JsonNode>
tools) {
        // POST https://api.mistral.ai/v1/chat/completions
        // Header: Authorization: Bearer <apiKey>
        // Body: 与 OpenAI 兼容格式
    }
    public CompletableFuture<LLMResponse> chatStream(...) { /* SSE 流式 */ }
}

```

Step 2: 在 `ProviderFactory.registerDefaults()` 注册策略

```

register(new ProviderStrategy() {
    @Override public boolean supports(String model) {
        return model.startsWith("mistral");
    }
    @Override public LLMProvider create(Config config, String model) {
        return new MistralProvider(apiKey, model);
    }
});

```

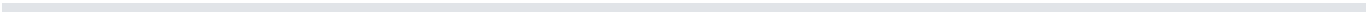
Step 3: `config.yaml` 添加配置


```
providers:
  mistral:
    apiKey: ""
    apiBase: "https://api.mistral.ai/v1"
```

判断工作量的关键点：

差异项	工作量	说明
API 兼容 OpenAI 格式	低	可直接复用 <code>buildRequestBody</code> 和 <code>parseResponse</code> 逻辑
认证方式不同（如 OAuth）	中	需改造 HTTP Header 构造
响应格式不同（如 Anthropic 的 content block）	高	需重写 <code>parseResponse</code> 和流式解析
不支持 tool calling	—	放弃该 Provider 的工具调用能力
流式格式不同	高	SSE 换行策略、delta 结构、 <code>[DONE]</code> 信号均可能不同

本项目已验证兼容的 API：DeepSeek API、OpenAI API。任何 OpenAI 兼容 API（如 vLLM、Ollama、LocalAI）理论上零修改接入。

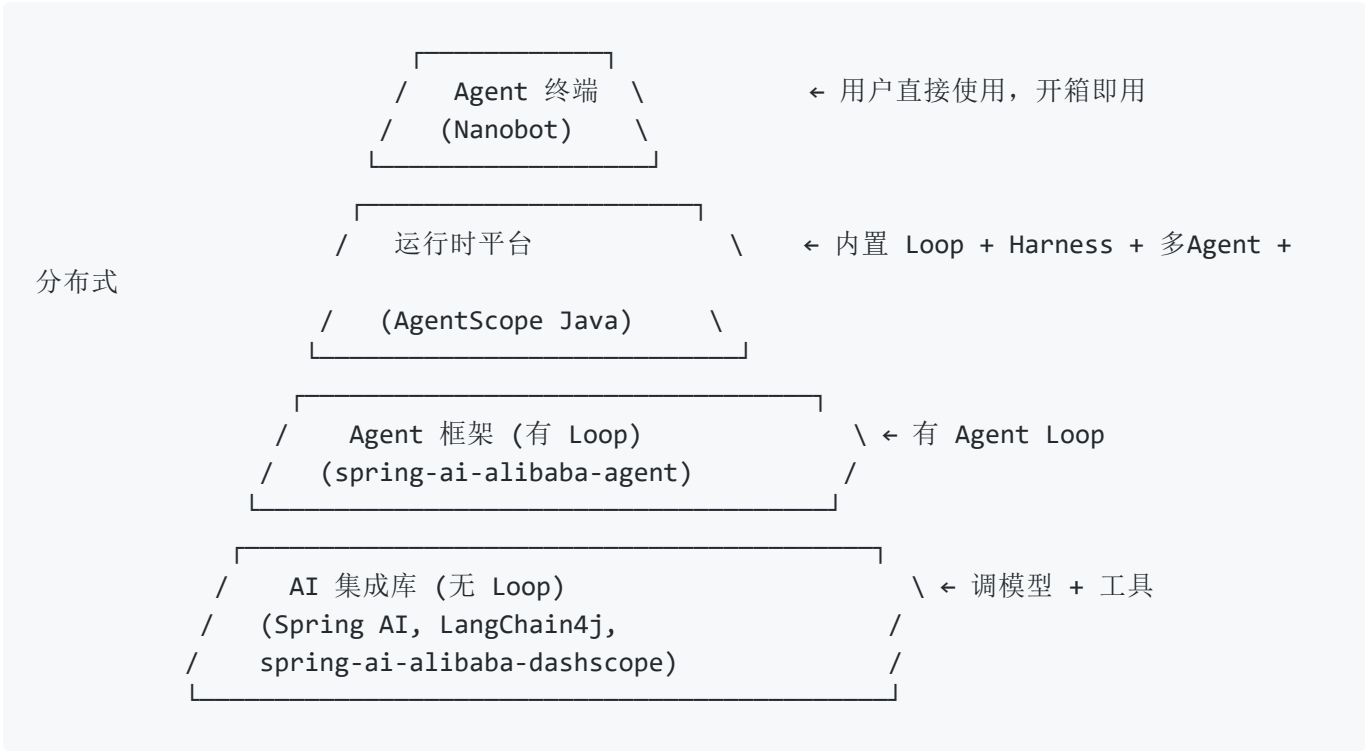


六、主流 AI 开发框架全景对比

阅读目标：理解 Nanobot 在 Java AI 开发生态中的位置，建立从"库 → 框架 → Agent 运行时 → Agent 终端产品"的层次认知。

在深入理解了 Nanobot 每个核心模块的实现之后，本章将目光拉远，把市面上主流的 Java AI 开发框架放在一起比较。这不是"谁更好"的排名，而是帮你理解**每个框架解决什么问题、不解决什么问题**。

6.1 Java AI 开发生态的四层金字塔



每往上一层，框架帮你多接管一个决策。底层告诉你"这是锤子"，顶层直接帮你把钉子钉好。

6.2 五个框架一览

项目	Maven 坐标	定位	一句话
Spring AI	org.springframework.ai:spring-ai-core	LLM 集成库	"把 LLM 调用变成 Spring Bean 之间的对话"
LangChain4j	dev.langchain4j:langchain4j	LLM 集成库	"Java 版 LangChain, Orchestration 框架"
DashScope SDK	com.alibaba.cloud.ai:spring-ai-alibaba-starter-dashscope	LLM 集成库	"通义千问 SDK, Spring AI 的阿里版"
Alibaba Agent	com.alibaba.cloud.ai:spring-ai-alibaba-agent-framework	Agent 框架	"在 DashScope SDK 之上加了 ReAct Agent Loop"
AgentScope Java	io.agentscope:agentscope	Agent 运行时平台	"阿里企业级多 Agent 编排 + Harness 工程化底座"
Nanobot	无 Maven 坐标 — ./nanobot CLI	Agent 终端	"启动即用的 Claude Code 级 AI 编程助手"

6.3 逐框架深度对比

6.3.1 Spring AI — 最底层的 LLM 集成库

仓库: `spring-projects/spring-ai` | **Maven:** `org.springframework.ai:spring-ai-core` | **定位:** 把 LLM 调用融入 Spring 生态

Spring AI 核心抽象:

```
ChatClient.call(prompt)
  → ChatModel (OpenAI/DeepSeek/Ollama...)
  → ToolCallback (@Tool 注解方法)
  → Advisor 链 (拦截器)
  → ChatMemory (对话历史存储接口)
  → VectorStore (RAG 检索)
```

Spring AI 做了什么:

- 统一了不同 LLM Provider 的 API 差异 (OpenAI/DeepSeek/Ollama 同一套接口)
- `@Tool` 注解自动生成 Function Calling Schema → `Method.invoke()` 反射执行
- `ChatMemory` 接口 + 默认 `InMemoryChatMemory` 实现
- `VectorStore` 抽象, 可接入 PgVector、Milvus、Pinecone
- ETL 管道: `DocumentReader` → `DocumentTransformer` → `DocumentWriter`

Spring AI 没做什么:

- **没有 Agent Loop:** `ChatClient.call()` 只做一次 LLM 调用+一次工具执行, 不会自动循环
- **没有会话管理:** 对话历史靠开发者手动拼接 `List<Message>`
- **没有权限控制:** 工具调用无安全检查
- **没有上下文治理:** Token 超限不会自动压缩
- **不独立运行:** 必须有 Spring Boot 应用包裹它

典型使用场景: 在已有的 Spring Boot 业务应用中加一个"智能问答"功能。

6.3.2 LangChain4j — 更丰富的 Orchestration 抽象

仓库: `langchain4j/langchain4j` | **Maven:** `dev.langchain4j:langchain4j` | **定位:** Java 版 LangChain, 编排 LLM workflow

LangChain4j 核心抽象：

ChatLanguageModel ← 同 Spring AI 的 ChatModel
ToolSpecification ← 同 Spring AI 的 @Tool
ChatMemory ← 对话历史存储
AiServices ← ★ 核心创新：接口代理
Chain/Router ← workflow编排
EmbeddingStore ← RAG 向量存储

LangChain4j 独有的亮点：

AiServices — 声明式 AI 服务接口代理：

```
interface Assistant {  
    String chat(String userMessage); // 框架自动处理 LLM 调用  
}  
  
Assistant assistant = AiServices.create(Assistant.class, model);  
String answer = assistant.chat("你好"); // 无需手动拼 prompt
```

LangChain4j vs Spring AI 核心差异：

	Spring AI	LangChain4j
设计哲学	Spring 生态原生	框架无关，可独立使用
Agent Loop	✗ 无	✗ 无（同样是单次调用）
AI 服务代理	无	✓ AiServices 接口代理
RAG 支持	✓ ETL 管道	✓ 更丰富的 Document Loader
workflow编排	基础	✓ Chain + Router + 条件分支
学习曲线	Spring 开发者友好	更通用，概念更多

和 Spring AI 一样没做的： Agent Loop、会话全生命周期管理、权限、长期记忆。两者本质同级——都是LLM 集成库。

6.3.3 spring-ai-alibaba-starter-dashscope — 通义千问 SDK

Maven： `com.alibaba.cloud.ai:spring-ai-alibaba-starter-dashscope` | **定位：** Spring AI 的阿里云版，和 Spring AI 同级

这是最底层——只做一件事：把通义千问（qwen-*）的 DashScope API 封装成 Spring AI 的 ChatModel 接口，让开发者用 Spring AI 统一 API 调通义模型。同时内置阿里云中间件适配（RocketMQ、Nacos、Redis 等）。

```
spring-ai-alibaba-dashscope 做了什么：
DashScope ChatModel      ← 调通义千问
+ 阿里云中间件适配        ← RocketMQ / Nacos / Redis
+ Spring AI 统一接口      ← ChatClient / @Tool / VectorStore
```

和 Spring AI 的关系：同级。Spring AI 默认对接 OpenAI，spring-ai-alibaba-dashscope 对接通义千问。都是 LLM 集成库，没有 Agent Loop。

6.3.4 spring-ai-alibaba-agent-framework — 补上了 Agent Loop

Maven： `com.alibaba.cloud.ai:spring-ai-alibaba-agent-framework` | **定位：**在 DashScope SDK 之上加 ReAct Agent Loop

这是同一个 groupId 下的另一个 artifact，**不替代** dashscope，而是**叠加**：

```
spring-ai-alibaba-agent = spring-ai-alibaba-dashscope (LLM 调用)
                          + ReActAgent (Agent Loop)
                          + ChatMemory (对话记忆)
                          + 工具编排
```

关键跨越 —— Agent Loop：

```
// dashscope 层 (无 Loop): 你需要自己写
while (!done) {
    response = chatClient.call(messages);
    if (response.hasToolCalls()) {
        result = tool.execute(response.getToolCall());
        messages.add(result); // 你管理历史
    } else {
        done = true;
    }
}

// agent 层 (有 Loop):
ReActAgent agent = ReActAgent.builder()
    .model(model)
    .tools(tools)
    .memory(memory)
    .build();
agent.call("帮我分析这份数据"); // Loop 全自动
```

但它仍然是一个“嵌入型 Agent”:

- 两个 artifact 都是库，引入到你的 Spring Boot 应用中
- 你的应用是主体，Agent 是其中一个 Bean
- 去掉 Agent，业务系统照常运行 → AI-Augmented

6.3.5 AgentScope Java — 企业级 Agent 运行时平台

仓库: `agentscope-ai/agentscope-java` | **Maven:** `io.agentscope:agentscope` | **定位:** 阿里开源的生产级多 Agent 编排 + Harness 工程化底座

这是目前 Java 生态中**在架构层级上最接近 Nanobot 的项目**，但设计目标完全不同——AgentScope 是“帮开发者搭 Agent 平台”，Nanobot 是“开发者直接用的 Agent 终端”。

AgentScope 双层架构:	
HarnessAgent (工程化封装层)	
└ Workspace	: 结构化工作目录 (AGENTS.md/MEMORY.md/knowledge/skills)
└ 双层记忆	: 流水账 → 后台合并精炼 → MEMORY.md + SQLite FTS5 检索
└ 上下文压缩	: 四道防线 (阈值触发 → 结构化保留 → 卸载到磁盘 → 兜底)
└ 权限系统	: 三态决策 (允许/需审批/拒绝)
└ Middleware	: 五阶段洋葱模型
└ 多 Agent 模式	: Pipeline/Routing/Handoffs/Supervisor/Subagents/Skills
ReActAgent (纯推理引擎 - 完全无状态)	
└ Record + Sealed Class	不可变消息
└ 28 种类型化 AgentEvent	
└ Project Reactor	全链路非阻塞

AgentScope 比 Nanobot 强的地方:

维度	AgentScope	Nanobot
并发模型	Project Reactor (数万并发)	CompletableFuture (单机并发)
多 Agent	✔ 9 种开箱即用协作模式	✔ 基础 AgentCoordinator
消息系统	Record + Sealed Class 编译期类型安全	POJO 运行时校验
企业部署	GraalVM <200ms、K8s HPA、Redis	单机 JVM
分布式	Dubbo/Nacos/OSS/Redis 共享	本地文件系统
生态	阿里云全系 + Spring Boot	独立运行

Nanobot 比 AgentScope 强的地方:

维度	Nanobot	AgentScope
启动方式	<code>./nanobot</code> 零配置	需写启动类 + Spring Boot
CLI 交互	全屏终端、Esc 中断、实时渲染	依赖外部 UI 或 API
互动性	<code>[y/N]</code> 权限确认、流式打断	通过事件流需要自己实现 UI
学习成本	2.6 万行, 纯手写, 什么都看得见	框架体系庞大, 概念多
安装体验	一个 fat JAR + 脚本	Maven/Gradle 依赖 + 配置

AgentScope 不是终端产品。它是一个强大的 Agent SDK——你得写 `@Tool` Bean、写启动类、配 Spring Boot、部署到 K8s。Nanobot 是 `./nanobot` 一敲就开始干活。

6.3.6 Nanobot — Agent 终端（不是框架也不是库）

仓库： `ztkyaojiayou/mynanobot-java` | **安装：** `git clone + ./scripts/nanobot` | **定位：** 像 Claude Code 一样的独立 AI 编程助手

和上面五个项目有本质区别——没有 Maven 坐标，不引入到其他项目。它不是库、不是框架、不是运行时平台，而是**最终产品**。

Nanobot 的设计前提是“开发者打开终端直接对话”，不是“开发者在 IDE 里引入一个 Maven 依赖然后写 Agent 代码”。这个定位决定了它在架构上的每一个选择：

不做的：

✗ 不需要 Spring Boot 包裹

✗ 不需要写 @Tool Bean

✗ 不需要配置 GraalVM

✗ 不需要 K8s 部署

✗ 不需要 Redis 共享 Session

→ V3 CLI 零依赖启动

→ 17 个内置工具 + MCP 扩展

→ JVM 直接跑

→ 本地单机即服务

→ 本地 JSONL 文件持久化

做到的：

✓ Agent Loop（8 状态 State 模式）

✓ Plan Mode（先计划后审批）

✓ CLI 全屏终端 + Esc 中断

✓ NANOBOT.md 项目记忆自动生成

✓ 4 层记忆自动提取

✓ 4 级权限管道

✓ SSE/WebSocket/HTTP 多通道

✓ Consolidator 自动压缩

6.4 全维度对比矩阵

维度	Spring AI	LangChain4j	DashScope SDK	Alibaba Agent	AgentScope Java	
定位	LLM 库	LLM 库	LLM 库	Agent 框架	Agent 运行时平台	
Agent Loop	✗	✗	✗	☑ ReAct	☑ ReAct	
工具系统	☑ @Tool	☑ @Tool	☑ @Tool	☑ @Tool	☑ @Tool + MCP	I
对话记忆	ChatMemory 接口	ChatMemory 接口	同 Spring AI	ChatMemory	☑ 双层 + FTS5	
会话管理	✗ 手动	✗ 手动	✗ 手动	✗ 手动	☑ Redis 共享	
上下文压缩	✗	✗	✗	✗	☑ 四道防线	
权限安全	✗	✗	✗	✗	☑ 三态决策	
Plan Mode	✗	✗	✗	✗	☑	
多 Agent	✗	✗	✗	✗	☑ 9 模式	
CLI 独立运行	✗	✗	✗	✗	✗	
流式中断	✗	✗	✗	✗	事件流	
并发模型	同步	同步	同步	同步	Project Reactor	C
消息类型安全	运行时	运行时	运行时	运行时	☑ Sealed Class	
企业部署	Spring Boot	框架无关	Spring Boot	Spring Boot	☑ K8s/GraalVM	

维度	Spring AI	LangChain4j	DashScope SDK	Alibaba Agent	AgentScope Java	
代码规模	—	—	—	—	十余万行	
学习门槛	低	中	低	中	高	
AI-Native 程度	AI-Augmented	AI-Augmented	AI-Augmented	AI-Augmented	☒ AI-Native*	
适用场景	业务加 AI	LLM 工作流	通义千问对接	阿里生态 Agent	企业 Agent 平台	

* AgentScope 本身是 AI-Native 框架，但它构建的应用可以是 AI-Native 也可以是 AI-Augmented。

6.5 选型指南

你的目标是什么？

"已有 Spring Boot 应用，想加个智能对话"

→ Spring AI 或 LangChain4j（够用，零学习负担）

"想用通义千问，只需要调模型能力"

→ spring-ai-alibaba-dashscope（纯 SDK，和 Spring AI 同级）

"想加一个能自主循环调工具的 Agent 模块"

→ spring-ai-alibaba-agent（在 dashscope 上加了 ReAct Loop）

"要做企业级多 Agent 平台，支撑数十个 Agent 协作"

→ AgentScope Java（Harness + 9 种多 Agent 模式 + 分布式）

"要一个能直接在终端里用的 AI 编程助手，像 Claude Code 那样"

→ Nanobot（它不是框架也不是库，是成品。./nanobot 直接跑）

"想学习 Agent 底层是怎么实现的，每一行代码都能看懂"

→ Nanobot（2.6 万行纯手写，从 MessageBus 到 Dream 全透明）

6.6 框架演进的必然路径

观察上面的对比矩阵，有一条清晰的演进线：



每一层都在上一层的基础上**多接管一个维度的决策**：

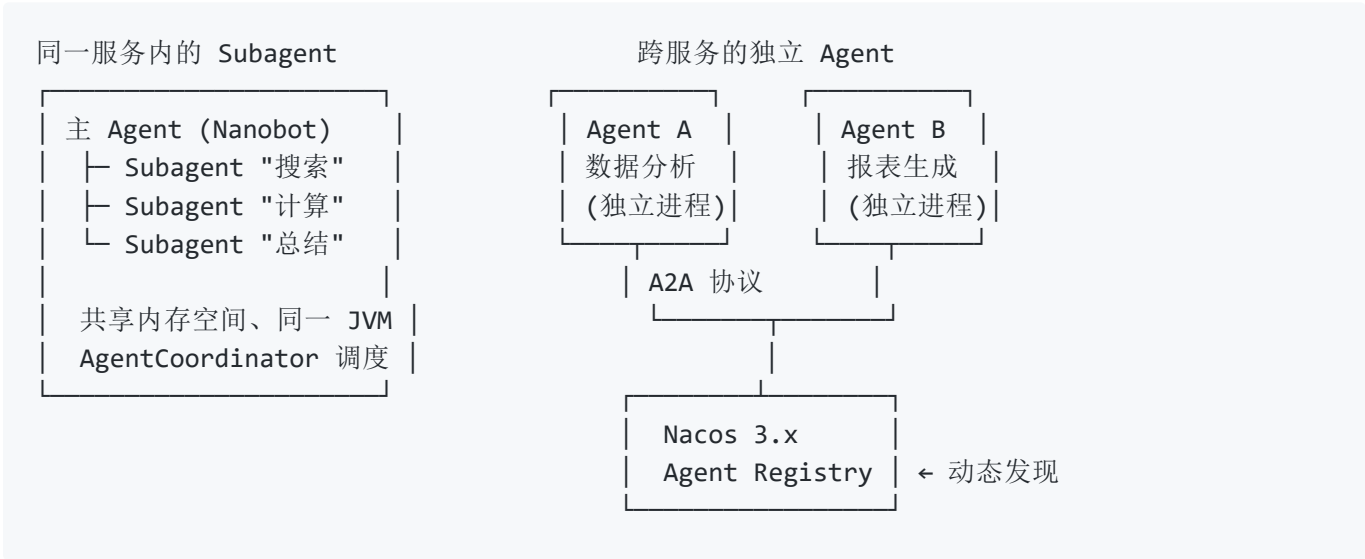
1. **库** (Spring AI / LangChain4j / DashScope SDK) ——让你不用写 HTTP 调用代码
2. **Agent 框架** (spring-ai-alibaba-agent) ——让你不用写 while 循环
3. **运行时平台** (AgentScope Java) ——让你不用管分布式、记忆、多 Agent 调度
4. **终端产品** (Nanobot) ——让你不用写任何代码，打开终端直接对话

这就是从"帮开发者省代码"到"帮开发者省决策"，再到"帮用户省掉整个开发过程"的演进。

6.7 多 Agent 协作：从 Subagent 到 A2A 协议

6.7.1 一个 Subagent ≠ 一个独立 Agent

回顾第五章 5.21（子 Agent 系统）和 AgentScope 的 9 种多 Agent 模式，这里有一个关键概念需要厘清：

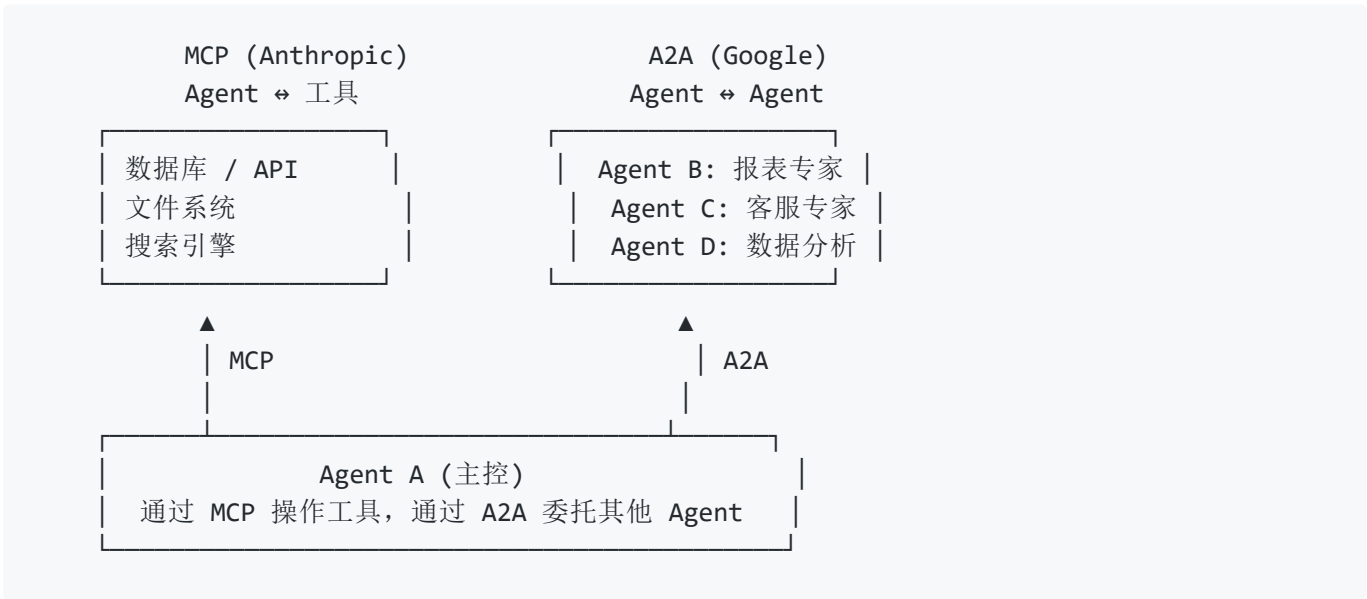


维度	同服务的 Subagent	跨服务的独立 Agent
进程	同一个 JVM 进程	各自独立进程/容器
发现机制	AgentCoordinator 内部调度	Nacos Agent Registry 服务发现
通信协议	方法调用 / 内存消息队列	A2A 协议 (HTTP/gRPC + Task 生命周期)
能力描述	枚举标签 ("search", "calc")	Agent Card (结构化 JSON)
故障隔离	Subagent 崩溃可能影响主进程	天然隔离，一个挂了不影响其他
扩展方式	代码内注册新 Subagent 类型	注册新 Agent 到 Registry
适用规模	单应用内的任务分解	企业级跨团队、跨服务的 Agent 协作

一句话：Subagent 是 Agent 的"多线程"，独立 Agent 是 Agent 的"微服务"。从 Subagent 到独立 Agent 的跨越，本质是从**单进程内部分工**升级到**分布式服务协作**。

6.7.2 双协议标准：MCP + A2A

2025 年，社区在 Agent 通信协议上收敛到两个互补的开放标准：

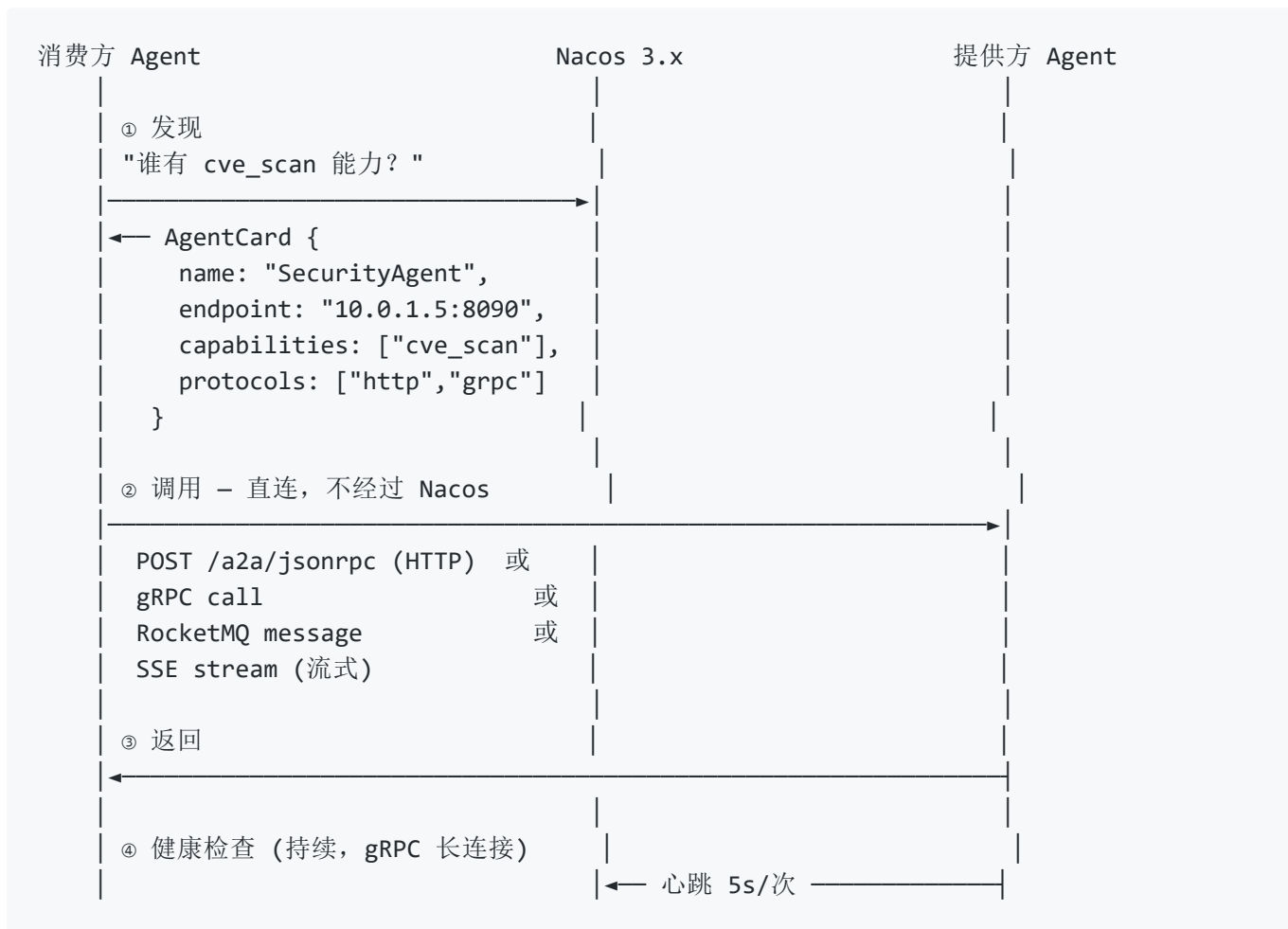


A2A 协议的四个核心要素：

要素	说明	示例
Agent Card	JSON 元数据文件，声明 Agent 身份、能力、接口	<pre>{"name": "DataAgent", "capabilities": ["sql_query", "statistical_analysis"], "endpoint": "https://..."} </pre>
Task 生命周期	每个委派任务有标准状态机	<pre>created → in-progress → completed/failed </pre>
多模态消息	支持文本 + 图片 + 结构化数据	分析结果可以是 JSON + Markdown 混合
安全认证	OpenAPI 授权 + TLS 加密	企业级 Agent 间安全通信

6.7.3 A2A 调用全链路：发现→调用→返回

A2A 协议的本质是**服务发现与业务调用分离**——Nacos 只管"谁在哪、能干什么"，实际通信是 Agent 之间直连：



和微服务的类比：Nacos 之于 Agent，就像 Nacos 之于 Dubbo/gRPC 微服务——注册中心只做服务发现，不代理业务流量。

6.7.4 四种调用方式

A2A 协议不强制单一通信模式，而是提供 Task **状态机** + **多传输层** 的组合，让调用方按场景选择：

A2A Task 状态机（9 个状态）：



模式	协议	工作方式	适合
同步调用	HTTP JSON-RPC	<code>agent.invoke(msg)</code> → Task 全程走完 → 阻塞返回终态	短任务（查订单 100ms）
轮询异步	HTTP JSON-RPC	<code>invoke()</code> 立即返回 <code>taskId</code> → 之后 <code>tasks/get?id=xxx</code> 查询	长任务 + 不想等
流式 SSE	HTTP SSE 长连接	<code>agent.stream(msg)</code> → <code>onDelta(chunk)</code> 逐块推 → <code>onComplete()</code>	实时生成（审计报告逐条输出）
长连接推送	gRPC / WebSocket	注册 webhook → 任务完成时 Server 主动推送	极长任务（压测跑 30 分钟）

同步调用示例：

```
// Spring AI Alibaba - A2aRemoteAgent
A2aRemoteAgent securityAgent = A2aRemoteAgent.builder()
    .name("SecurityAgent")
    .agentCardProvider(agentCardProvider)
    .connectTimeout(Duration.ofSeconds(10))
    .readTimeout(Duration.ofSeconds(30))
    .build();

// 阻塞等待，直到 Task 状态变为 Completed/Failed
Optional<OverAllState> result = securityAgent.invoke(
    Map.of("action", "cve_scan", "file", "pom.xml")
);
```

流式 SSE 调用示例：

```
// Spring AI Alibaba - 流式返回
@GetMapping(value = "stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<ServerSentEvent<String>> stream(@RequestParam("question") String question)
{
    return rootAgent.stream(Map.of("input", List.of(new UserMessage(question))))
        .mapNotNull(output -> {
            if (output instanceof StreamingOutput chunk) {
                return ServerSentEvent.<String>builder().data(chunk.chunk()).build();
            }
            return null;
        });
}
```

多 Agent 并发调用示例：

```
// 并发调三个独立 Agent，等全部返回后汇总
CompletableFuture<OverAllState> f1 = CompletableFuture.supplyAsync(() ->
    securityAgent.invoke(...));
CompletableFuture<OverAllState> f2 = CompletableFuture.supplyAsync(() ->
    complianceAgent.invoke(...));
CompletableFuture<OverAllState> f3 = CompletableFuture.supplyAsync(() ->
    perfAgent.invoke(...));

CompletableFuture.allOf(f1, f2, f3).join();
// 汇总三个结果
```

6.7.5 A2A "异步"的实质——仍是 Request-Response

这是理解 A2A 协议最关键的洞察：A2A 的"异步"只是把一次长调用拆成了多次短 HTTP 请求，每一跳都是 request-response。

A2A "同步"：

```
Client →req→ Server
Client ←res← Server          ← 一次 HTTP round-trip
```

A2A "轮询异步"：

```
Client →req→ Server
Client ←{task_id, status:"Working"}← Server    ← 第 1 次 request-response
...过几秒...
Client →tasks/get?id=xxx→ Server               ← 第 2 次 request-response
Client ←{task_id, status:"Completed"}← Server
```

A2A "webhook 推送"：

```
Client →req + webhook_url→ Server
Client ←{task_id, status:"Working"}← Server    ← 第 1 次 request-response
...过几分钟...
Server →POST webhook_url→ Client              ← Server 反调，仍是 HTTP request-response!
```

本质没变——底层永远是 Client 发 → Server 回 的 HTTP 请求-响应模型。A2A 只是在上面抽象了一个 Task 状态机，用 task_id 把多次 request-response 关联起来，让你"感觉像是异步的"。

这和外卖 App 查骑手位置完全一样——每次刷新都是一个新的 HTTP 请求，后端不会主动推给你。

6.7.6 真正的异步——RocketMQ A2A 传输层

上面三种模式的共同问题：**Client 必须主动发起每一跳**，无论是轮询还是注册 webhook（webhook 本质是"Server 替 Client 发了一次 HTTP POST"）。

真正的消息驱动异步需要中间加一个 Broker：

A2A over HTTP（伪异步）：

Client → Server → Client

← 每一跳都是 HTTP，有来有回

调用方明确知道被调用方是谁 ← 耦合

RocketMQ A2A（真异步）：

Client —publish→ Topic: a2a.task.request

Client 不等回复，立即返回，继续干别的

...过一会儿...

Topic: a2a.task.result —push→ Client (Consumer 回调)

Client 收到结果，触发回调函数

关键差异：

- Client 发完就忘（fire-and-forget）
- Client 不知道谁在处理（完全解耦）
- 没有挂起的 HTTP 连接，没有轮询
- 一个 Task 可以广播给多个 Agent 并行处理

方式	协议层	实质	耦合度
A2A <code>invoke()</code>	HTTP JSON-RPC	一次 request-response，阻塞到 Task 终态	高
A2A <code>tasks/get</code> 轮询	HTTP JSON-RPC	多次 request-response，每次主动查询	高
A2A webhook 推送	HTTP POST	Server 反调 Client，仍是 request-response	中
A2A SSE 流式	HTTP 长连接	一次请求，Server 持续推送事件	中
RocketMQ A2A	消息队列	发完即忘，Broker 推结果——真正的异步	低

结论：A2A 标准协议没有脱离 HTTP 的 request-response 模型。它定义了 Task 状态机和 AgentCard 格式，但不强制传输层。阿里开源的 RocketMQ A2A 传输层是第一个把"真正的消息驱动异步"带入 A2A 生态的实现——适合高并发、长任务、弱耦合的场景。

6.8 实战案例：电商智能客服多 Agent 系统

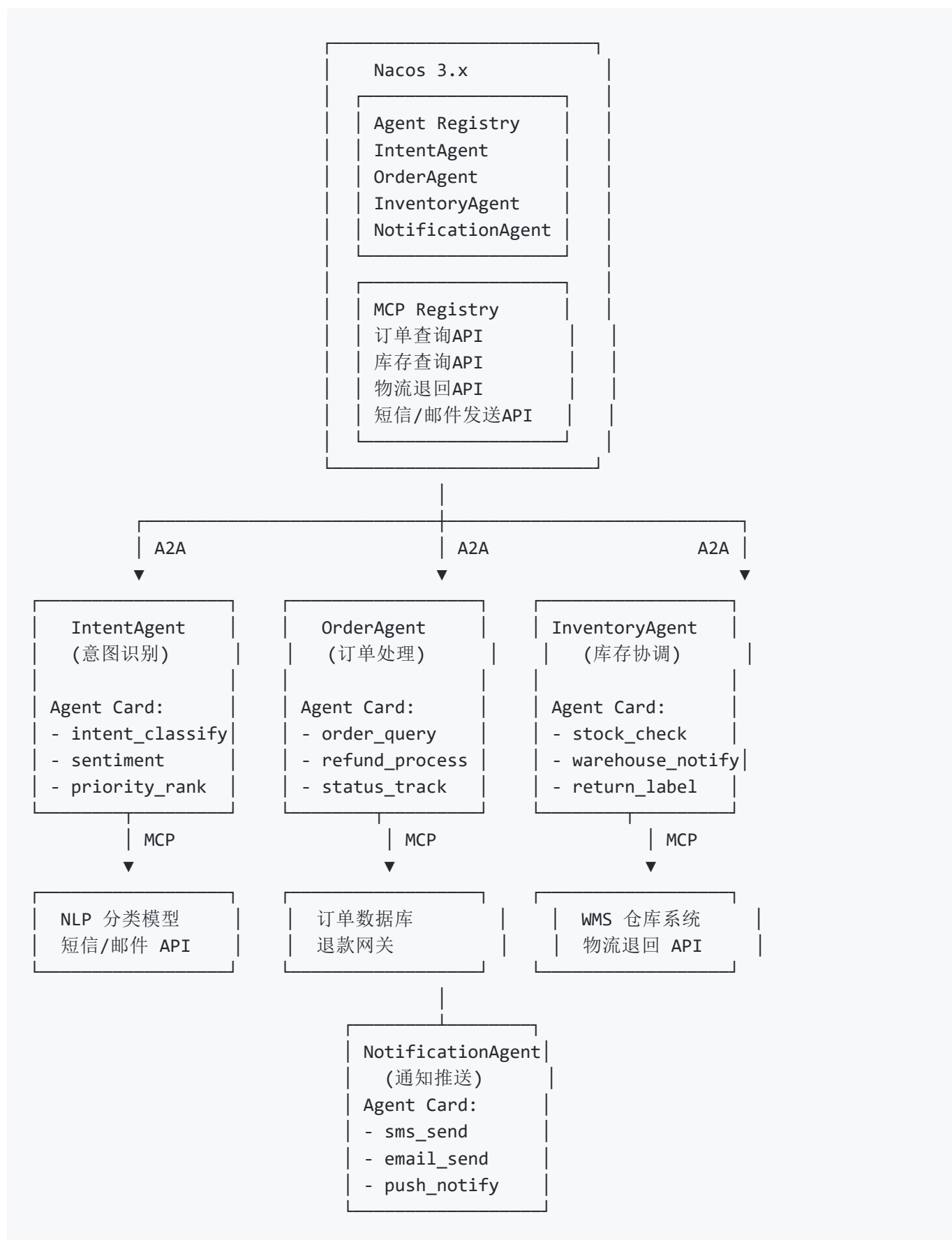
以下是一个真实的生产级多 Agent 协作架构，模拟某电商平台的售后处理流程。

6.8.1 业务场景

用户发起售后请求："我买的手机屏幕有坏点，我要退货。"

这涉及到四个专业领域，单个 Agent 难以全部精通。

6.8.2 架构设计



6.8.3 协作全链路

用户: "手机屏幕有坏点, 我要退货"



Step 1: IntentAgent 接收请求

- Nacos MCP Registry 搜索可用的 NLP 模型
- 调用意图识别 MCP 工具
- 输出: {
 "intent": "refund_request",
 "sentiment": "negative",
 "priority": "high",
 "entities": {"product": "手机", "issue": "屏幕坏点"}
}
- A2A 委派给 OrderAgent
- Task {id: "task-001", type: "order_lookup", ...}



Step 2: OrderAgent 处理售后

- Nacos Agent Registry 查询 "谁管库存?"
 ← InventoryAgent (capability: warehouse_notify)
- MCP: 查询订单数据库 → 订单号 #12345, 金额 ¥3999
- MCP: 调退款网关 → 发起原路退款
- A2A 委派给 InventoryAgent
- Task {id: "task-002", type: "return_label", ...}



Step 3: InventoryAgent 库存协调

- MCP: 查 WMS 仓库系统 → 确认退货仓有库容
- MCP: 调物流退回 API → 生成退货单号 RTN-88921
- Task 完成 → 返回 {return_label: "RTN-88921"}



Step 4: NotificationAgent 通知用户

- A2A 接收汇总结果
- MCP: 发送短信 "您的退货 RTN-88921 已受理, 预计3个工作日内退款"
- MCP: 发送邮件 (含退货单 PDF)



用户收到短信 + 邮件 → 售后完成

关键点: 每个 Agent 只专注于自己的领域。IntentAgent 不懂订单、OrderAgent 不懂库存、InventoryAgent 不懂通知。但通过 Agent Registry 动态发现 + A2A 标准通信 + MCP 工具调用,

它们无缝协作完成了一个单个 Agent 难以胜任的复杂任务。

6.8.4 Nacos 在其中的角色

每新增一个 Agent（比如加一个 PricingAgent 做动态定价）：

1. 注册 Agent Card 到 Nacos Agent Registry
2. 注册它需要的新 MCP 工具到 Nacos MCP Registry
3. 其他 Agent 通过 MCP Router 的语义匹配自动发现新能力
4. 零代码改动即可参与协作

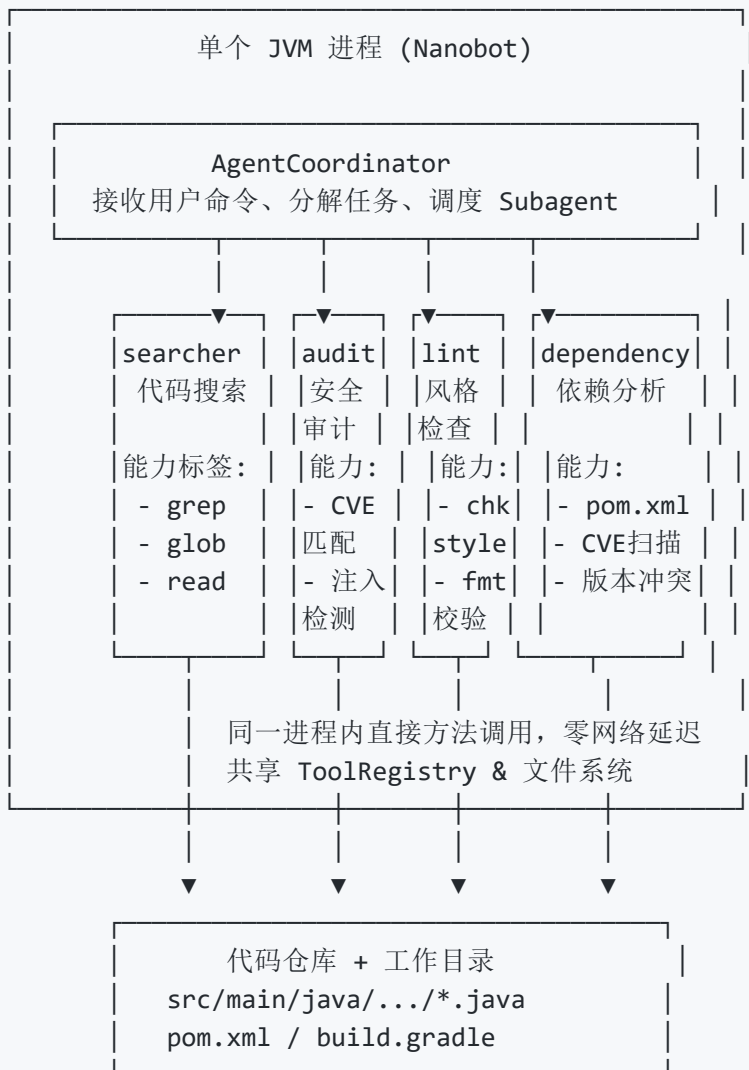
6.9 单服务多 Subagent 实战案例 —— 代码审查助手

上面的电商案例用的是跨服务独立 Agent（A2A + Nacos）。现在看一个**同服务 Subagent 模式**的生产案例——代码审查助手。

6.9.1 业务场景

开发团队提交 PR，需要对代码进行**多维度审查**：安全检查、性能分析、代码风格、依赖风险。

6.9.2 架构设计



6.9.3 协作全链路

用户: "审查这次 PR 的代码, 重点关注安全漏洞和性能问题"

Step 1: AgentCoordinator 分解任务

主 Agent 分析请求 → 拆成 4 个并行子任务:

- searcher: 找出 PR 中所有变更的 Java 文件
- audit: 用已知 CVE 模式匹配变更代码
- lint: 跑 checkstyle 检查代码风格
- dependency: 检查 pom.xml 依赖是否有已知漏洞

(4 个 Subagent 并行执行, 共享 ToolRegistry)

- searcher: `glob("**/*.java") + grep("变更的类")`
→ [OrderService.java, PaymentUtil.java, ...] (6 files)
- audit: `read_file + grep("SELECT.*WHERE.*\+" | "Runtime.exec" | ...)`
→ ⚠ PaymentUtil.java:42 发现 SQL 拼接, 疑似注入漏洞
→ ⚠ OrderService.java:108 发现 Runtime.exec() 调用
- lint: `exec("mvn checkstyle:check")`
→ ⚠ 3 处缩进不规范, 1 处方法名不符合驼峰
- dependency: `read_file("pom.xml") + exec("mvn dependency:analyze")`
→ ⚠ log4j-core 1.2.17 有 CVE-2021-44228 高危漏洞
→ ⚠ 建议升级到 2.17.1+

Step 2: summarizer (另一个 Subagent) 汇总报告

"以下是本次 PR 审查结果:

● 高危 (必须修复):

1. PaymentUtil.java:42 - SQL 注入漏洞
WHERE 子句使用了字符串拼接
2. log4j-core 1.2.17 - CVE-2021-44228 (Log4Shell)
建议立即升级到 2.17.1+

● 中危 (建议修复):

3. OrderService.java:108 - Runtime.exec() 调用
建议改用 ProcessBuilder

● 提示:

4. 3 处代码风格问题 (checkstyle)
5. 1 个未使用的依赖 (maven dependency:analyze)

是否要我帮你修复这些高危问题? "

关键点：4 个 Subagent 在**同一 JVM 内并行运行**，共享 ToolRegistry。searcher 的输出可以直接被 audit/lint/dependency 消费——不需要序列化、不走网络、零延迟。AgentCoordinator 在主线程汇总结果，最后交给 summarizer 生成报告。

6.9.4 同样的需求，用独立 Agent 实现会怎样？

如果用 A2A + Nacos 的独立 Agent 架构做同样的事：

优势：

- ✓ SecurityAuditAgent 可以由安全团队独立维护和迭代
- ✓ 可以水平扩展（PR 量大时起多个实例）
- ✓ 挂了不影响主 Agent，可以重试

代价：

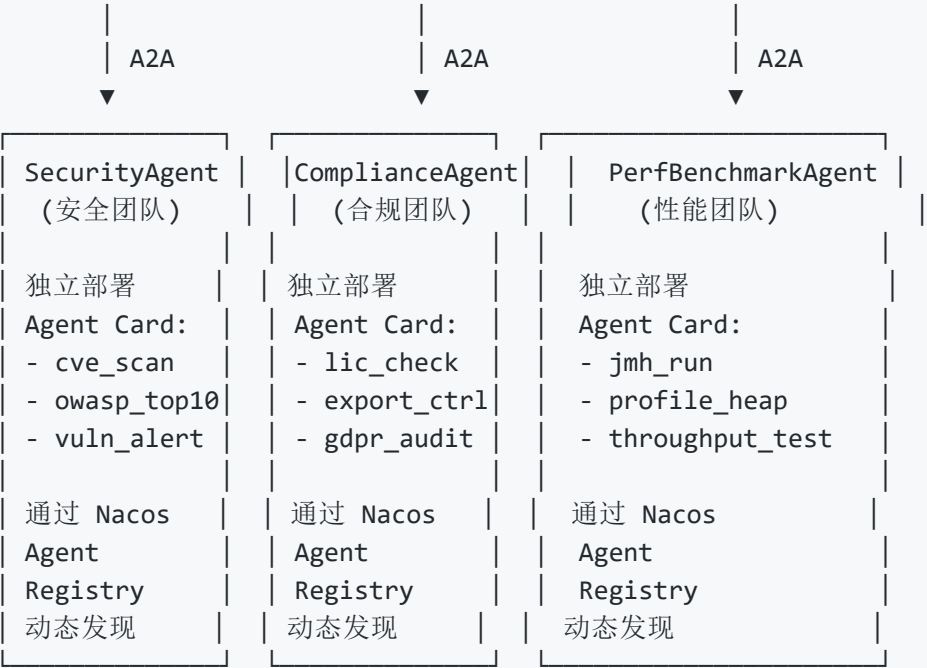
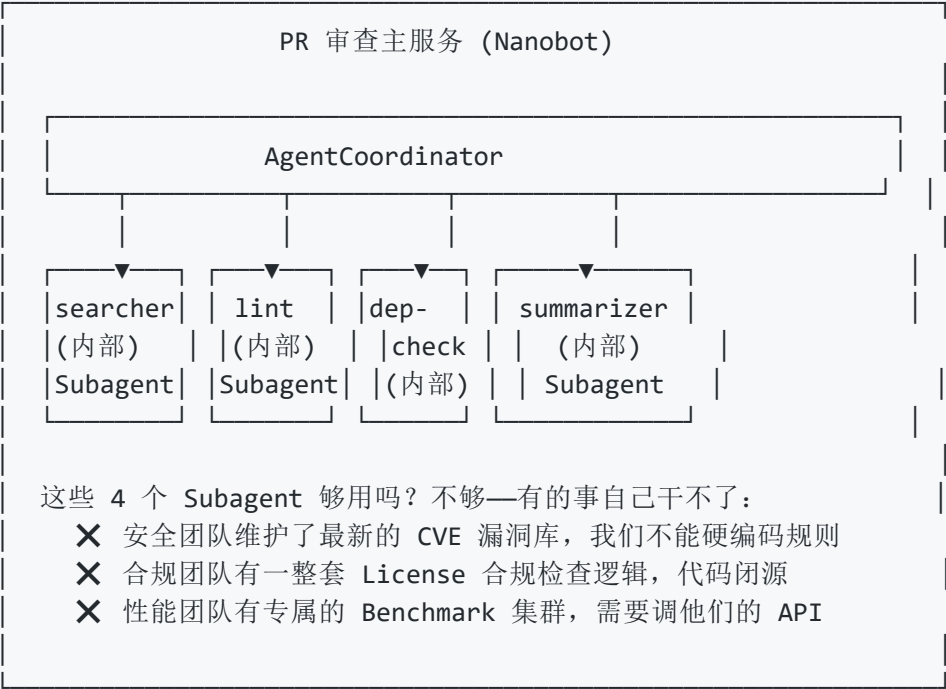
- ✗ 每个 Subagent 调用变成了一次 HTTP/gRPC 网络请求
- ✗ 需要 Agent Card 注册、A2A Task 序列化/反序列化
- ✗ 需要部署 5 个独立服务（而非 1 个）
- ✗ 调试复杂——跨进程链路追踪需要 OpenTelemetry

这就是 Subagent vs 独立 Agent 的核心取舍——简单 vs 灵活，低延迟 vs 独立扩展。

6.9.5 混合架构——同一个服务中既有 Subagent 又调用外部 Agent

实际生产中，**纯 Subagent 和纯独立 Agent 都不是常态**。更常见的是混合架构——主服务内部用 Subagent 做任务分解，同时调用外部团队的独立 Agent 获取专业能力。

还是上面那个代码审查场景，把它升级：



全链路：

Step 1-3 (Subagent): searcher + lint + dep-check 同一 JVM 内并行执行

→ 发现: SQL 注入 + log4j CVE + checkstyle 问题

→ 但 CVE 扫描用的是本地规则库, 半年没更新了

Step 4 (A2A 调外部): AgentCoordinator → Nacos Agent Registry

→ "谁有最新的漏洞数据?" ← SecurityAgent (capability: cve_scan)

→ A2A Task {id:"task-004", file:"pom.xml", context:"log4j 1.2.17"}

← SecurityAgent: "确认 CVE-2021-44228 (CVSS 10.0)。此外还发现:

CVE-2024-xxxxx (上周新披露), CVE-2024-yyyyy (本月新披露)

这些在本地规则库中不存在——外部 Agent 的知识是实时的"

Step 5 (A2A 调外部): AgentCoordinator → Nacos Agent Registry

→ "谁管合规?" ← ComplianceAgent (capability: lic_check)

→ A2A Task {id:"task-005", file:"pom.xml"}

← ComplianceAgent: "发现 com.example:internal-lib (Apache 2.0)

与你们项目使用的 GPL 3.0 存在兼容性问题——需要法务审批"

Step 6 (Subagent): summarizer 汇总内部 + 外部所有结果

→ 生成最终审查报告 (内部 Subagent 发现 + 外部 Agent 补充)

Step 7 (A2A 通知): AgentCoordinator → NotificationAgent

→ 把报告推送到企业微信 / Slack / 邮件

这个混合架构说明了关键设计原则:

选择 Subagent 的条件:

- ✓ 自己能做的
- ✓ 不需要独立扩缩容
- ✓ 不需要独立迭代发布
- ✓ 低延迟要求
- ✓ 共享内存/文件系统更方便

选择独立 Agent 的条件:

- ✓ 别人做得更好的
- ✓ 由其他团队维护的
- ✓ 需要独立扩缩容的
- ✓ 知识需要实时更新的
- ✓ 需要隔离部署/权限管控的

6.10 多 Agent vs 单服务多 Subagent —— 决策指南

场景	推荐方案	理由
个人 AI 编程助手，需要搜索+计算+总结	Subagent	Nanobot 的 AgentCoordinator 足够，单进程零延迟
一个 Spring Boot 应用内，需要多个 AI 模块协作	Subagent	共享内存、直接方法调用、无需网络开销
电商售后系统，订单/库存/物流分属不同团队	独立 Agent + A2A	各自独立部署、独立迭代、独立扩缩容
跨部门协作，Agent 由不同团队开发和维护	独立 Agent + A2A	Agent Card 是团队间的 API 契约
需要水平扩展（某个 Agent 扛不住高并发）	独立 Agent + K8s HPA	独立 Agent 可以单独扩缩，Subagent 随主进程
10 万并发请求分发到数百个 Agent	Nacos + A2A	服务发现 + 负载均衡 + 故障摘除

核心判断标准：



6.11 Nacos 3.x AI 资产管理 —— 核心组件速览

Nacos 3.0 将定位从"微服务注册中心"升级为 "AI Agent 应用的动态服务发现、配置管理和智能体管理平台"。

6.10.1 四维注册表



注册表	管什么	杀手能力
MCP Registry	Agent 可调用的工具/API	存量 HTTP 接口零代码转 MCP 服务；语义搜索自动匹配
Agent Registry	Agent 实例 + Agent Card	A2A 协议原生支持；按能力标签动态发现
Prompt Registry	System Prompt 模板	版本控制、A/B 测试、灰度发布、增量推送（带宽省 72%）
Skill Registry	可复用技能组件	Docker 沙箱隔离、恶意代码扫描（98%+ 识别率）

6.10.2 MCP Router —— Agent 的统一工具入口

Agent → "我需要查订单、发短信、生成退货单"

MCP Router (从 200 个工具中自动匹配 3-5 个)

- └ search_mcp_server("订单查询") → 订单数据库 MCP 服务
- └ search_mcp_server("短信发送") → 阿里云短信 MCP 服务
- └ search_mcp_server("物流退货") → 物流退回 MCP 服务

内置三个标准工具： search_mcp_server （搜索） → add_mcp_server （安装） → use_mcp_tool （调用）。

6.10.3 版本演进路线

版本	时间	新增
3.0	2025 Q2	MCP Registry + 零信任安全 + 存量接口一键转 MCP
3.1	2025 Q3	A2A Registry + Agent Card 注册发现
3.2	2026 Q1	Prompt/Skill Registry + Nacos Copilot + CLI

本章小结：从 Spring AI 的 "给 API 套个壳" 到 AgentScope 的 "企业级 Agent 平台"，到 Nanobot 的 "开箱即用的终端产品"，再到 Nacos 3.x 的 "让 Agent 像微服务一样被发现和管理"——Java AI 生态正在经历从 "能调模型" 到 "多 Agent 自治协作" 的范式转变。理解这个演进脉络，才能看清每个项目的定位和你自己的位置。

七、nanobot CLI 实战指南

阅读目标：掌握 nanobot CLI 的日常使用方式、常用命令和最佳实践，像使用 Claude Code 一样高效。

7.1 启动与环境

```
# 把 scripts/ 目录加到 PATH
export PATH="/path/to/nanobot-java/scripts:$PATH"

# 在任意项目目录下启动
cd /your/project
nanobot

# 指定工作目录
nanobot --workspace /path/to/project

# 恢复之前的会话
nanobot --resume cli-1234567890
```

启动后进入交互界面：

my-nanobot CLI 模式
基于 Java 的 AI Agent助手

输入消息开始对话，/exit 退出，/clear 清上下文，/compact 压缩，/remember 记，Esc 中断

>

7.2 命令速查

命令	别名	功能	示例
<code>/help</code>	—	列出所有可用命令	<code>/help</code>
<code>/init</code>	—	分析项目生成 NANOBOT.md	<code>/init</code>
<code>/mode plan</code>	<code>/plan</code>	进入规划模式（只读分析+出计划）	<code>/plan</code>
<code>/plan approve</code>	—	审批计划并切换到执行模式	<code>/plan approve</code>
<code>/mode default</code>	—	默认模式（读放行，写需确认）	<code>/mode default</code>
<code>/mode accept_edits</code>	—	接受编辑模式（读+文件编辑放行）	<code>/mode accept_edits</code>
<code>/mode bypass</code>	—	绕过模式（全部放行，谨慎使用）	<code>/mode bypass</code>
<code>/resume</code>	—	列出最近 5 个会话	<code>/resume</code>
<code>/resume <key></code>	—	恢复到指定会话	<code>/resume cli-1234567890</code>
<code>/clear</code>	—	清空当前会话上下文	<code>/clear</code>
<code>/compact</code>	—	手动压缩对话历史（LLM 摘要旧消息）	<code>/compact</code>
<code>/remember</code>	—	手动提取长期记忆（无视增量阈值）	<code>/remember</code>
<code>/exit</code>	<code>/q</code> , <code>/quit</code>	退出 CLI	<code>/exit</code>
<code>Esc</code>	—	中断当前流式回复	按 <code>Esc</code> 键

7.3 核心 workflow

6.3.1 工作流 1: 自由对话 (Vibe Coding)

最直接的使用方式：描述需求 → AI 生成代码 → 验证结果 → 不满意就继续改。

> 帮我写一个用户登录接口，Spring Boot + JWT

AI: 分析项目结构 → 生成 LoginController + JwtUtil + 配置 → 流式输出到终端

> 端口改成 9090

AI: 修改 application.yml 中的 server.port

> 帮我跑一下测试看看有没有问题

AI: 执行 mvn test → 分析结果 → 修复 → 再跑 → 全部通过

6.3.2 workflow 2: Plan Mode (推荐)

> /plan

📖 已进入规划模式 - LLM 将只读分析并出计划，不会修改代码

> 实现用户注销功能，包括：清除 JWT、失效 Session、前端跳转

AI 使用只读工具探索项目 → 输出计划：

需求理解 / 影响范围 / 实现步骤 / 注意事项

> /plan approve

✅ 计划已审批，进入执行模式... (AI 开始按计划逐步实现)

6.3.3 workflow 3: 探索式开发

> /init # 第一步：生成项目记忆

> 这个项目的认证是怎么实现的？ # AI 读取 NANOBOT.md + grep 相关代码

> 有没有安全漏洞？ # AI 做安全审查

> 帮我把明文密码改成 BCrypt # AI 已理解项目结构，直接精准修改

7.4 权限模式选择指南

信任度低 ←		→ 信任度高	
PLAN	DEFAULT	ACCEPT_EDITS	BYPASS
(只读)	(默认)	(编辑放行)	(全放行)
/plan	/mode	/mode	/mode
	default	accept_edits	bypass

场景	推荐模式	原因
刚接手不熟悉的项目	PLAN	先读后改，避免盲目修改
日常开发	DEFAULT	读自动放行，写操作需确认
信任的编码会话	ACCEPT_EDITS	跳过文件编辑确认，Shell 仍需确认
CI/CD 自动化	BYPASS	完全信任，无人值守

权限确认快捷键：

```
[!] 工具调用需要确认：
    工具：exec
    参数：{command=mvn test}
    1=允许  2=之后都放行  3=拒绝  [1/2/3]
```

7.5 会话管理

```
> /resume                # 查看历史会话
最近会话 (/resume <key> 恢复)：
cli-1784128421194        12 条消息  07-16 14:30
cli-1784207282340        5 条消息  07-15 09:12

> /resume cli-1784128421194 # 恢复指定会话
> /clear                    # 清空当前上下文
> /compact                 # 手动压缩对话历史
> /remember                # 手动提取长期记忆
```

Web 界面：访问 `http://localhost:8080/sessions.html`，支持查看、重命名、删除会话。

7.6 NANOBOT.md — 项目的 AI 记忆文件

`/init` 命令会在项目根目录生成 `NANOBOT.md`，后续每次对话 AI 都会自动加载它。

nanobot-java 项目概述

这是一个基于 Java 17 的 AI Agent 框架...

技术栈

- Java 17, Maven, Spring Boot 3.2

构建和运行

```
mvn compile && mvn test && ./scripts/nanobot
```

手动编辑建议：补充编码规范、常用命令、目录用途说明。

7.7 实用技巧

1. **善用 Esc 中断：**AI 生成跑偏了，按 `Esc` 立即停止
2. **先 /init 再干活：**进入新项目第一件事
3. **复杂需求用 Plan Mode：**`/plan` → 审查计划 → `/plan approve` → 执行
4. **分步执行：**不要一次性描述过于复杂的需求
5. **信任加速：**频繁执行同类操作时，用 `2=` 之后都放行

八、Claude Code 实战指南

为什么要学 Claude Code？ 本项目全程使用 Claude Code 构建。Claude Code 是 Anthropic 官方出品的 AI 编程 Agent CLI 工具，是目前 Agent-Driven Development 的标杆产品。掌握它，你就能理解本项目的设计目标——用 Java 复刻一个类 Claude Code 的 Agent 框架。

8.1 Claude Code 是什么

Claude Code 是 Anthropic 推出的**终端原生 AI 编程助手**，运行在命令行中，不是一个 IDE 插件。

```
$ claude
> 帮我实现用户登录功能
  → AI 自动探索项目（列出文件、读取代码、搜索关键模式）
  → 制定计划（改哪些文件、加什么依赖、注意什么安全问题）
  → 逐步实现（创建文件、编辑代码、跑测试验证）
  → 提交 git commit
```

与 Copilot / Cursor 的关键区别：

	GitHub Copilot	Cursor	Claude Code
运行环境	IDE 插件	独立 IDE	终端 CLI
交互方式	Tab 补全 + 聊天面板	编辑器内联 + 聊天面板	纯命令行对话
自主性	低（需人触发补全）	中（可对话式编辑）	高（自主探索→规划→执行→验证）
工具调用	受限	部分支持	完整（文件读写、Shell、Web、Git）
项目理解	当前文件上下文	项目级索引	自主探索（glob/grep/read）
权限模型	无	部分	4 级权限（Plan/Default/AcceptEdits/Bypass）

8.2 安装与启动

```
# 安装 Claude Code（需要 Node.js 18+）
npm install -g @anthropic-ai/claude-code

# 在任意项目目录启动
cd /your/project
claude

# 首次使用需要登录 Anthropic 账号
claude login
```

启动后界面：

```
>                                     ← 直接输入需求，AI 开始工作
```

没有繁琐的配置，Claude Code 自动检测项目结构。

8.3 核心命令速查

命令	功能
<code>/help</code>	列出所有可用命令
<code>/clear</code>	清空当前对话上下文
<code>/compact</code>	压缩对话历史（释放 token 预算）
<code>/init</code>	分析项目生成 NANOBOT.md（项目记忆文件）
<code>/doctor</code>	诊断环境问题
<code>/login</code> / <code>/logout</code>	账号管理
<code>/status</code>	查看当前会话状态
<code>/add-dir</code>	添加额外的工作目录
<code>/cost</code>	查看本次会话的 API 费用
<code>/context</code>	查看当前上下文使用情况
<code>/review</code>	代码审查当前改动
<code>/security-review</code>	安全审查当前改动
<code>/pr-comment</code>	将审查结果发布为 PR 评论
<code>Ctrl+C</code>	中断当前回复

8.4 核心 workflow

7.4.1 工作流 1: Vibe Coding（对话式编程）

最直接的使用方式：

```
> 帮我写一个 Spring Boot REST API, GET /users 返回用户列表

Claude: 先读项目结构 → 确认是 Spring Boot 项目
      → 创建 UserController.java
      → 跑 mvn test 验证 → 通过 ✓
```

7.4.2 工作流 2: Plan Mode（先计划后执行）

这是 Claude Code 最特色的工作流：

- > /plan
- 进入规划模式（只读）
- > 实现用户注销功能，包括 JWT 黑名单和 Session 失效

Claude（只读模式）：

1. 探索项目：glob `**/*Auth*.java` → 找到 `AuthController`, `JwtUtil`
 2. 阅读代码：read_file `AuthController.java`, `JwtUtil.java`
 3. 搜索相关：grep `"token\|jwt\|logout"` → 确认没有现有实现
 4. 出计划：
 - ### 实现计划
 - `JwtUtil.java`：新增 `invalidate()` + 内存黑名单
 - `AuthController.java`：新增 `POST /logout`
 - `SecurityConfig.java`：放行 `/logout`
 - 测试：`AuthControllerTest`
- 等待审批

> 继续

Claude：切换到执行模式 → 按计划逐步实现 → 跑测试 → 全部通过 

对比本项目的 Plan Mode：完全相同的工作流，`/plan` → 探索 → 出计划 → `/plan approve` → 执行。

7.4.3 workflow 3：代码审查

> /review

Claude：分析 `staged changes` → 从多个维度审查：

- 正确性：有没有 bug？
 - 安全：SQL 注入、敏感信息泄露？
 - 性能：N+1 查询、不必要的循环？
 - 可维护性：命名清晰？有无重复代码？
- 输出审查报告 + 修复建议

7.4.4 workflow 4：多轮迭代

> 帮我写一个四则运算计算器

Claude: 生成 `Calculator.vue` → 完整的加减乘除功能

> 加个历史记录功能

Claude: 读 `Calculator.vue` → 在现有代码上添加 `history` 列表

> 把历史记录存到 `localStorage`, 页面刷新不丢失

Claude: 再加持久化 → 验证 → 完成

8.5 CLAUDE.md — 项目的 AI 记忆

这是 Claude Code 最核心的功能之一，也是本项目 `NANOBOT.md` + `/init` 的灵感来源。

运行 `/init` 后，Claude Code 自动探索项目并生成 `CLAUDE.md`：

```
# nanobot-java

A Java 17 AI Agent framework built with Spring Boot 3.2.

## Build & Run
- `mvn compile` - compile
- `mvn test` - run tests
- `./scripts/nanobot` - launch CLI mode

## Architecture
- `core/AgentLoop.java` - state machine engine
- `core/AgentRunner.java` - LLM call loop
- `tools/` - 17 built-in tools
...
```

每次对话开始时，Claude Code 自动加载 `CLAUDE.md` 作为系统提示词。你可以手动编辑补充：

- 编码规范 ("用 4 空格缩进""禁止使用 Lombok @Builder")
- 项目约定 ("数据库迁移用 Flyway, SQL 文件放 db/migration/")
- 常用命令 ("启动: `./mvnw spring-boot:run`")

8.6 权限系统

Claude Code 的 4 级权限模式是本项目 `PermissionManager` + `PermissionMode` 的对标对象：

模式	Claude Code	nanobot
Plan (只读)	<code>/plan</code> — 只能读文件，不能改	<code>/mode plan</code> — 完全相同
Default (默认)	读放行，写需确认	<code>/mode default</code> — 完全相同
Accept Edits	读+文件编辑放行，Shell 需确认	<code>/mode accept_edits</code> — 完全相同
Bypass	全部放行	<code>/mode bypass</code> — 完全相同

交互确认：Claude Code 弹出 `y/N` 确认，nanobot 用 `1/2/3` 数字选择（多了"之后都放行"选项）。

📖 详见第五章 5.11（安全系统详解）和 5.8（Plan Mode）。

8.7 Claude Code 的架构思想

Claude Code 的架构直接启发了本项目的核心设计：

Claude Code 特性	本项目的对应实现
CLI 终端原生交互	V3 <code>CliChannel</code> + <code>JLine</code> + <code>Markdown</code> 渲染
Agent Loop (LLM 循环调用)	<code>AgentLoop</code> + <code>AgentRunner</code>
文件读写工具	<code>read_file</code> , <code>write_file</code> , <code>edit_file</code>
搜索工具	<code>glob</code> , <code>grep</code>
Shell 执行	<code>exec</code>
Web 搜索	<code>web_search</code> , <code>web_fetch</code>
Task 系统 (任务追踪)	<code>TaskStore</code> + <code>task_create/list/update</code>
MCP 工具扩展	<code>MCPManager</code> + <code>MCPToolWrapper</code>
<code>/init</code> → CLAUDE.md	<code>/init</code> → NANOBOT.md
<code>/plan</code> 规划模式	<code>/mode plan</code> + <code>/plan approve</code>
Permission 权限模式	<code>PermissionManager</code> + <code>PermissionMode</code>
Stream 流式输出	<code>StreamResponseCallback</code> + <code>SSE</code>
Session 会话管理	<code>SessionManager</code> + <code>sessions.html</code>

8.8 实战建议

1. 先 `/init` 再干活：进入新项目第一件事就是 `/init`，让 Claude Code 理解项目。

2. 善用 Plan Mode：

```
/plan          # 进入规划
（描述需求）   # Claude 探索 + 出计划
（审查计划）   # 不满意就调整
/plan approve  # 满意后执行
```

3. 定期 `/compact`：对话长了之后 token 占用上升，`/compact` 压缩历史释放预算。

4. 用 `/cost` 关注费用：每次对话结束看看花了多少钱，养成习惯。

5. 利用 `CLAUDE.md` 积累项目知识：把每次踩坑的经验写进去，后续对话自动生效。

6. 复杂任务分步拆解：不要一次性描述过于复杂的需求，拆成 2-3 步逐步推进。

7. 命令行 + IDE 配合：Claude Code 做"设计+编码"的脏活，你在 IDE 里 Code Review 和微调。

九、Harness 规范体系 — Claude Code 的结构化使用方式

Vibe Coding 的问题在于"随意"——没有规格约束、没有变更追踪、没有质量门禁。Harness 体系是在 Claude Code 之上叠加一套轻量级开发规范，让 AI 编码从"随意"走向"可控"。

本项目的 `.harness/` 目录就是这套规范的完整实现。它是本项目全程使用 Claude Code 构建的核心方法论。

8.1 目录结构


```

.harness/
├── agents/                # Agent 角色定义
│   └── owner.md           # Owner Agent - 总编排者（身份+职责+流水线+决策边界）
├── rules/                 # 约束规则（AI 的"护栏"）
│   ├── SDD-TDD模式.md    # Spec → Test → Code 三层关系
│   ├── 工程结构.md       # 目录组织 + 包结构 + 命名约定
│   ├── 开发流程规范.md   # 6 阶段流水线 + 门禁 + 变更状态机
│   ├── 编码规范.md       # Java 命名/日志/异常/提交规范
│   └── 运行时可靠性.md   # 工具超时重试/消息持久/优雅关闭
├── skills/                # 可复用技能（6 阶段流水线的执行者）
│   ├── request-analysis/ # ① 需求分析 → 产出 change.md
│   ├── coding-skill/     # ② 编码实现（TDD）
│   ├── unit-test-write/  # ③ 边界/降级测试
│   ├── expert-reviewer/  # ④ 多维度评审
│   ├── unit-test-ci/     # ⑤ CI 门禁
│   └── deploy-verify/    # ⑥ 部署验证
├── changes/               # 变更追踪（Spec Coding 核心）
│   ├── _TEMPLATE/        # 模板（change + review + verify）
│   └── C-001~C-018/      # 18 张已完成的变更卡片（三件套）
└── wiki/                  # 项目知识库（按需加载）
    ├── 业务模型.md / 接口协议.md / 数据模型.md
    └── 架构决策.md        # 10 个 ADR（Java17/State模式/ProviderFactory/...）

```

8.2 核心理念

三重约束：

- `rules/` — 约束边界（AI 不能做什么）
- `skills/` — 执行手段（AI 怎么一步步做）
- `changes/` — 可追溯记录（每一步留档）

6 阶段流水线：

人类意图

- ① `request-analysis` 需求分析 → `change.md`（规格真相源）
- ② `coding-skill` 编码实现（TDD: Red→Green→Refactor）
- ③ `unit-test-write` 边界/降级测试（覆盖率 ≥80%）
- ④ `expert-reviewer` 专家评审（0 个严重问题）
- ⑤ `unit-test-ci` CI 全量通过（`mvn test` 59/59）
- ⑥ `deploy-verify` 部署冒烟验证
- 交付

变更状态机： draft → analyzing → coding → testing → reviewing → ci → verifying → done

8.3 在 Claude Code 中如何使用

```
# 1. 新建变更
mkdir -p .harness/changes/C-019-new-feature
cp .harness/changes/_TEMPLATE/* .harness/changes/C-019-new-feature/

# 2. 对 Claude Code 说
> 按 .harness 流程，帮我完成 C-019：给 AgentLoop 加个新状态
  → Claude Code 自动读取 owner.md（角色定位）+ rules/（约束）
  → 按 ①→⑥ 流水线逐步推进
  → 每阶段产出写入 .harness/changes/C-019/

# 3. 人类只需在关键 Gate 审批
> ① 需求卡 OK，继续 ②
> ④ 评审通过，继续 ⑤
```

8.4 与 Plan Mode 的关系

Harness 流水线	nanobot Plan Mode
① request-analysis	/plan + 描述需求 → AI 探索 + 出计划
② coding-skill	/plan approve → AI 按计划逐步实现
③~⑥ 验证	人工审查 + mvn test

核心价值： Harness 把"AI 辅助开发"从不可控的 Vibe Coding 升级为可追溯、可验证、可断点续传的 Spec Coding 工程实践。你不只是在和 AI 聊天，你是在用 AI 做软件工程。

8.5 Harness 各模块详解

8.5.1 Owner Agent (agents/owner.md)

Owner Agent 是整个 Harness 体系的总指挥。它定义了 AI 的角色定位、工作流水线和决策边界。

模块	说明
身份与使命	"我是 mynanobot-java 的 Owner Agent, 精通 Java、Spring Boot、AI Agent 框架"
核心职责	需求理解、技能编排、约束守护、上下文管理、变更追踪、质量兜底、人机协同
工作流水线	6 阶段严格按序, 不可跳步 (见上文)
决策边界	可自主决定 (命名/重构/测试设计) vs 必须请示人类 (新增依赖/改 API/改架构)
上下文加载策略	按需加载 (地图模式), 禁止全量灌入; 任何任务先读 CLAUDE.md + rules/
技术铁律	JDK 17、Spring Boot 3.2.5、Maven 3.9+、deepseek-chat、JLine 3

8.5.2 Rules 规则层详解

每个 `.harness/rules/` 文件都是对 AI 的强制约束:

文件	核心内容	AI 行为约束
SDD-TDD模式.md	SDD (change.md=真相源) + TDD (Red→Green→Refactor) + Harness (约束→验证→留档)	核心逻辑必须先写失败测试, 覆盖率 ≥80%
工程结构.md	完整目录树、包结构约定、命名约定、新增模块/功能约定	单模块 Maven, <code>com.nanobot.*</code> 分包, 禁止循环依赖
开发流程规范.md	6 阶段流水线 + 门禁 + 变更状态机 + 人机协同协议 + 提交规范	每个 Gate 不通过禁止进入下一阶段; 状态变更即更新 change.md
编码规范.md	Java 命名 (PascalCase/camelCase)、Lombok 策略 (@Data 允许)、日志约定、异常处理、安全红线	禁止吞异常、禁止硬编码 Key、提交前 mvn test 全绿
运行时可靠性.md	Agent 调用保护 (超时/重试/降级)、消息总线可靠性、会话持久化、CLI 交互可靠性、优雅关闭	外部依赖必须有超时+重试+降级; 消息队列有界防 OOM

8.5.3 Skills 技能层详解

6 个技能对应开发流水线的 6 个阶段：

阶段	技能	输入	产出	Gate
① 需求分析	request-analysis	一句话意图	change.md（用户故事 + AC + 边界 + NFR）	AC 可测试、≥3 边界情况
② 编码实现	coding-skill	已确认的 change.md	失败测试 + 最小实现	测试通过 + mvn compile
③ 单测编写	unit-test-write	实现代码	边界/降级/回归测试	核心覆盖率 ≥80%
④ 专家评审	expert-reviewer	代码 + 测试	评审报告（5 维度）	0 个严重问题
⑤ CI 门禁	unit-test-ci	完整变更	CI 报告	mvn test 59/59 全绿
⑥ 部署验证	deploy-verify	通过 CI 的构建	验证报告	冒烟 + 健康检查

8.5.4 Changes 变更追踪详解

每个功能/模块 = 一张变更卡片，三件套：

```
.harness/changes/C-NNN-<slug>/
├─ change.md    - 用户故事 + 验收标准 + 边界情况 + 非功能需求（规格真相源）
├─ review.md    - 设计评审记录（5 维度：代码规范/工程结构/测试覆盖/边界处理/可扩展性）
└─ verify.md    - 验证记录（mvn test 59/59 全绿 + 编译通过）
```

本项目 18 张卡片速览：

编号	模块	状态
C-001	项目脚手架 (Java 17 + Maven + Spring Boot)	done
C-002	消息总线 (MessageBus + Inbound/Outbound)	done
C-003	AgentLoop 引擎 (State 模式七状态机)	done
C-004	AgentRunner 执行循环 (LLM→Tool→递归)	done
C-005	工具系统 (Tool 接口 + 17 内置工具)	done
C-006	LLM 提供商 (ProviderFactory + OpenAI + DeepSeek)	done
C-007	记忆系统 (Dream + Consolidator + NANOBOT.md)	done
C-008	会话管理 (SessionManager + SessionStore + Web UI)	done
C-009	安全权限 (PermissionManager + Guard + RuleEngine)	done
C-010	命令系统 (/exit /help /init /mode /resume)	done
C-011	钩子系统 (AgentHook + Metrics/Validation/Tracing)	done
C-012	身份系统 (SOUL + IDENTITY + USER)	done
C-013	V3 CLI (JLine + Markdown + Esc 中断)	done
C-014	Plan Mode (/plan → /plan approve)	done
C-015	V2 Spring Boot (REST + SSE + WebSocket + sessions.html)	done
C-016	MCP 集成 (StdioMCP + HttpMCP + MCPManager)	done
C-017	设计模式重构 (State + Strategy + Repository)	done
C-018	文档体系 (架构手册 + README + NANOBOT.md + .harness)	done

8.5.5 Wiki 知识库

文件	内容
业务模型.md	Agent 框架核心概念 (InboundMessage→AgentLoop→Tool→Provider 流转)
接口协议.md	REST API (6 端点) + CLI 9 命令 + WebSocket + MCP 协议
数据模型.md	文件存储 5 层 (history.jsonl / MEMORY.md / NANOBOT.md / TaskStore / config.yaml)
架构决策.md	10 个 ADR (Java 17/单模块/State模式/ProviderFactory/文件存储/JLine/Claude Code)

8.5.6 快速上手

```
# 1. 新需求：创建变更卡
cp -r .harness/changes/_TEMPLATE .harness/changes/C-019-my-feature
# 编辑 change.md 填写用户故事和 AC

# 2. 启动 Claude Code，一句话启动流水线
> 按 .harness 流程完成 C-019，先读 rules/ 和 wiki/

# 3. AI 按 6 阶段推进，关键 Gate 等你拍板
> ① OK 继续 ②
> ④ 通过 继续 ⑤

# 4. 收口
mvn test # 59/59 全绿
git commit -m "feat: xxx"
```

十、新手学习路线

9.1 学习阶段规划

阶段	目标	时间	关键知识点
Phase 1	动手跑起来	1-2 天	第三章 Step 1-3 (最小Agent+工具+System Prompt)
Phase 2	理解核心机制	3-5 天	第五章 5.1-5.5 (Agent Loop/状态机/全链路/流式/工具)
Phase 3	深入架构设计	3-5 天	第五章 5.6-5.10 (身份/System Prompt/Plan Mode/上下文/记忆)
Phase 4	扩展开发	3-5 天	自定义工具、新 Provider、MCP 集成

9.2 Phase 1：动手跑起来

- 1. 安装 JDK 17+、Maven 3.9+
- 2. 按第三章 3.0 搭建项目骨架
- 3. 跑通 Step 1 (最小 Agent) → Step 2 (工具调用) → Step 3 (System Prompt)
- 4. 启动 nanobot CLI: `./scripts/nanobot`

9.3 Phase 2：理解核心机制

学习路径：

- 1. AgentLoop.java — 状态机引擎 → 理解消息处理全流程
- 2. AgentRunner.java — LLM 调用循环 → 理解工具执行和递归逻辑
- 3. MessageBus.java — 消息总线 → 理解异步解耦设计
- 4. Tool.java + ToolRegistry.java — 工具系统 → 理解扩展机制

实践：跟踪一条消息从进入到响应的完整流程，添加一个简单的自定义工具。

9.4 Phase 3：深入架构设计

学习路径：

- 1. BuildState.java — System Prompt 构建 → 理解上下文注入
- 2. Plan Mode — 权限 + 工具 + 提示词三层协同 → 理解安全设计
- 3. SessionManager + SessionStore — Repository 模式 → 理解 I/O 分离
- 4. Consolidator + Dream — 记忆压缩 + 长期记忆 → 理解记忆系统

实践：实现一个自定义 Provider，添加会话压缩逻辑。

9.5 Phase 4：扩展开发

1. 添加新工具：实现 Tool 接口 → 注册到 ToolRegistry

2. 添加新 Provider：实现 LLMProvider 接口 → 注册到 ProviderFactory

3. 配置 MCP 服务器：`config.yaml` 添加配置，自动发现工具

十一、关键技术选型

功能	技术方案	理由
运行环境	Java 17 LTS	生产环境主流版本
异步编程	CompletableFuture + ExecutorService	标准异步编程模型
JSON 处理	Jackson	成熟稳定，支持 Schema 验证
HTTP 客户端	HttpClient (Java 11+)	内置，支持异步和流式
终端输入	JLine 3	跨平台原始按键读取（Esc 中断）
配置管理	Jackson + YAML	支持复杂嵌套配置
日志	SLF4J + Logback	Java 标准日志框架
Web 框架	Spring Boot 3.2	V2 模式 HTTP/SSE/WS
构建部署	Maven + Fat JAR + shell/bat 脚本	跨平台一键启动
文件 I/O	NIO.2 (Files/Paths)	现代文件 API

十二、配置说明

11.1 配置文件位置

<code>src/main/resources/config/config.yaml</code>	← 项目默认配置（内置）
<code>~/.nanobot/config.yaml</code>	← 用户自定义覆盖

11.2 配置结构


```

agents:
  defaults:
    model: "deepseek-chat"           # 默认模型（ProviderFactory 自动匹配）
    workspace: ".nanobot"
    maxTokens: 8192
    contextWindowTokens: 200000
    temperature: 0.7
    maxToolIterations: 100
    maxTurns: 0                     # 0=不限制
    maxCost: 0                      # 0=不限制，单位美元
    timezone: "UTC"

providers:
  openai:
    apiKey: ""                      # 优先读环境变量 OPENAI_API_KEY
    apiBase: "https://api.openai.com/v1"
  deepseek:
    apiKey: ""                      # 优先读环境变量 DEEPSEEK_API_KEY
    apiBase: "https://api.deepseek.com"

tools:
  exec:
    enable: true
  web:
    enable: true
    search:
      provider: "baidu_web"
      maxResults: 5
      timeout: 30

mcp_servers:                        # MCP 服务器（可选）
  git:
    command: "npx"
    args: ["-y", "git-mcp@latest"]
    tool_timeout: 30

memory:
  dream:
    maxMemories: 100

```

11.3 环境变量

变量名	说明	适用 Provider
DEEPSEEK_API_KEY	DeepSeek API 密钥	deepseek-chat
OPENAI_API_KEY	OpenAI API 密钥	gpt-4o, o1, o3 等
JAVA_HOME	JDK 路径	启动脚本使用

ProviderFactory 根据 model 字段自动匹配 Provider: deepseek* → DeepSeekProvider, gpt-*/o1/o3/o4 → OpenAIProvider。

十三、编译、启动与部署

12.1 环境要求

依赖	版本	说明
Java	17+	核心运行环境
Maven	3.9+	构建工具

12.2 编译

```
mvn clean compile      # 编译
mvn test                # 运行测试
mvn clean package       # 打包（含依赖）
mvn clean package -DskipTests # 打包跳过测试
```

12.3 启动

V3 CLI 模式 (推荐)：

```
nanobot                # 全局安装后任意目录运行
java -jar target/nanobot-cli.jar # 直接启动 JAR
nanobot --workspace /path/to/project # 指定工作目录
```

V2 Spring Boot 模式：

```
mvn spring-boot:run
# 访问 http://localhost:8080 (聊天) /sessions.html (会话管理)
```

V1 独立模式：

```
mvn exec:java -Dexec.mainClass="com.nanobot.v1.Nanobot"
```

12.4 自动重建 (scripts/nanobot)

scripts/nanobot 自动检测源码变更并重新打包：

```
# 源码有更新 → 自动 mvn package -DskipTests -q → 再启动
```

12.5 常见启动问题

问题	解决方案
API Key 未设置	检查环境变量或 config.yaml
端口被占用	修改配置或停止占用进程
依赖下载失败	<code>mvn clean compile -U</code> 强制更新
内存不足	增加 JVM 堆内存 <code>-Xmx2g</code>

十四、调试与日志

13.1 日志配置

日志配置文件： `src/main/resources/logback.xml`

日志级别： `DEBUG` (详细) → `INFO` (一般) → `WARN` (警告) → `ERROR` (错误)

13.2 调试技巧

- 1. 设置断点跟踪状态机转换
- 2. 查看消息队列的入队出队
- 3. 监控 LLM 调用的请求/响应

4. 使用钩子记录执行时间

十五、扩展建议

14.1 添加新工具

```
public class WeatherTool implements Tool {
    @Override public String getName() { return "get_weather"; }
    @Override public String getDescription() { return "获取指定城市的天气信息"; }
    @Override public JsonNode getParameters() { /* JSON Schema */ }
    @Override public CompletableFuture<Object> execute(Map<String, Object> params) {
        String city = (String) params.get("city");
        return CompletableFuture.completedFuture("天气结果...");
    }
}
// 注册: toolRegistry.register(new WeatherTool());
```

14.2 添加新 Provider

```
public class AnthropicProvider implements LLMProvider {
    @Override public String getName() { return "anthropic"; }
    @Override public CompletableFuture<LLMResponse> chat(...) {
        // 调用 Anthropic API
    }
}
```

14.3 配置 MCP 服务器

```
mcp_servers:
  git:
    command: "npx"
    args: ["-y", "git-mcp@latest"]
    tool_timeout: 30
```

十六、常见问题

Q1: 如何配置 API Key?

通过环境变量（`DEEPSEEK_API_KEY`）或 `~/.nanobot/config.yaml` 配置。

Q2：如何添加自定义工具？

实现 `Tool` 接口，在 `NanobotConfig.registerTools()` 中注册。

Q3：如何扩展支持其他 LLM？

实现 `LLMProvider` 接口，处理特定 API 的格式差异，注册到 `ProviderFactory`。

Q4：项目依赖哪些库？

Jackson（JSON）、SLF4J + Logback（日志）、Spring Boot 3.2（V2 模式）、JLine 3（CLI 终端）。

十七、学习资源

1. **本项目文档**：第四章（架构概览）+ 第五章（核心模块详解）
2. **第三章教程**：从 0 到 1 逐步构建 Agent
3. **测试代码**：参考 `src/test/` 中的单元测试
4. **原始项目**：`../nanobot/` 目录下的 Python 源码

十八、总结

Nanobot-Java 的核心价值在于：

1. **清晰的分层架构**：State 模式状态机 + 策略工厂 + Repository 分离
2. **灵活的扩展能力**：插件化工具系统、可注册 Provider 策略、Command 模式命令
3. **多种设计模式**：State、Strategy、Command、Chain of Responsibility、Template Method、Repository
4. **类 Claude Code 的 CLI 体验**：JLine 终端、Markdown 渲染、Plan Mode、Esc 中断
5. **新手友好**：第三章从 10 行代码开始，逐步构建完整 Agent

建议学习顺序：

1. 第三章：动手构建 Agent（建立感性认识）
2. 第四章：架构概览（建立全局地图）

3. 第五章 5.1-5.5: 核心模块 (Agent Loop / 状态机 / 全链路 / 流式 / 工具)
4. 第五章 5.22: LLM Provider 体系 — 理解 Agent 如何与 DeepSeek API 交互, 学会对接新模型
5. 第五章 5.6-5.10: 进阶模块 (身份 / System Prompt / Plan Mode / 上下文 / 记忆)
6. 第六章: 主流 AI 框架对比 — 建立 "库→框架→运行时→终端产品" 的层次认知
7. 第七章: nanobot CLI 实战 (日常使用)
8. 第八章: Claude Code 实战 (对标学习)